UNIVERSITY OF LIVERPOOL

Department of Electrical Engineering and Electronics

Interim Report for MEng Group Project

IoT and AI for Assisted Living

by

Daniel Mitchell 201540943

Mia Hornett 201551655

Alvaro Mesa Giner 201656665

Luis Guillot Lozano 201514204

Project Supervisor: Dr Valerio Selis

Project Assessor: Dr Junquing Zhang

Declaration of academic integrity

We confirm that we have read and understood the University's Academic Integrity Policy. We confirm that we have acted honestly, ethically and professionally in conduct leading to assessment for the programme of study. We confirm that we have not copied material from another source nor committed plagiarism nor fabricated, falsified or embellished data when completing the attached piece of work. We confirm that we have not copied material from another source, nor colluded with any other student/s or used artificial intelligence software in an unacceptable manner in the preparation and production of this work. Software applications include but are not limited to ChatGPT, Bing Chat DALL.E and Bard.

Signature 1:	Daniel Mitchell	Date 09/12/2025
Signature 2:	Alvaro Mesa Giner	Date 09/12/2025
Signature 3:	Mia Hornett	Date 09/12/2025
Signature 4:	Luis Guillot Lozano	Date 09/12/2025

Abstract

The project investigates the development of an Internet of Things and AI assisted living system that combines multi-sensor monitoring, local edge processing, actuator control, machine learning, and a cloud connected mobile application. The system upholds a privacy preserving monitoring platform that supports individuals in assisted living environments, also helping carers and family members provide the highest level of care. The local system integrates numerous sensing technologies, communicating over a secure MQTT network to a Raspberry Pi, where the sensor data is compared to the machine learning model. The development of a FastAPI backend with a PostgreSQL database incorporated alongside a Flutter based mobile app represent the cloud system that runs in parallel to the local system, providing remote access, data visualisation and system control. Each tier can run independently of each other and communicates via HTTPS, an encrypted communication protocol.

Initial results demonstrate reliable multi-sensor network, stable MQTT communication, machine learning model implemented, deployment of the cloud backend, and successful functionality of the frontend app. The sensor modules were able to transmit structured data through the local network, confirming the messaging format and topic structure operate as expected. Furthermore, these early tests verify the privacy first approach, through processing data locally and using the secure MQTT communication port, of our design is successful. Although the complete data flow and behavioural model have not yet been implemented and tested, the system at its current phase indicates stability and thus the architecture is technically feasible moving forward with development. The success of the local communication system, sensors, machine learning model, frontend app, and backend infrastructure provides a strong foundation for the next steps in the project.

Table of Contents

1. Introduction

The Internet of Things is rapidly expanding and has become a central component of modern smart home environments. Smart home systems such as Amazon Alexa, Google Home and Apple HomeKit show how networked devices can enhance convenience, automation and user interaction within a domestic home. This technology is experiencing significant growth and increasingly into healthcare and assisted living. IoT systems offer the ability for continuous monitoring, time sensitive alerts and responsive automation, significant areas in assisted living. This project will reduce the strain on the caregiver thanks to remote access to the system, in addition to the family members, to ensure safety in the home. It builds on these developments in technologies by designing a hybrid IoT architecture, combining a local MQTT network with a cloud based FastAPI backend, using a Raspberry Pi as the local edge controller. The aim of the project is to implement a foundation for a smart healthcare assistant capable of receiving sensor inputs, triggering actuator responses and providing remote access through the frontend app.

Modern IoT platforms such as AWS IoT Core and Azure IoT Hub highlight the value of combining local responsiveness with cloud scalability, however these systems rely on constant internet connectivity and introduce a significant cost. Academic research exaggerates the continued need for enhanced reliability, reducing latency and improving user autonomy through edge computing [69]. This project adheres to this, and the design demonstrates an open, relatively inexpensive, hybrid architecture by using widely available hardware and open-source software.

Crucially, the main project objectives were established prior to the commencing of the project. There were six main objectives with the first being to create an IoT and AI system for assisted living. The second objective is to have a minimum of eight devices, five sensors and three actuators. This can be expanded on during the project, to further enhance the monitoring system. Another objective is to have an AI machine learning model to learn routines and flag anomalies when they arise. An overarching objective is for our system to uphold a privacy first design, throughout the project and architecture. Next the frontend of the architecture will be an app, so that caregivers and family members can remain updated with time sensitive notifications and can cause actuator responses if necessary. The final objective is to have the system continuously monitoring the environment.

The project makes use of iterative engineering in which each subsystem is developed, tested and refined prior to the integration of the full system. Each group member worked independently on individual areas to achieve a sufficient level ready for integration. These areas include; sensors and actuators, the local and cloud network development, machine learning model, and frontend app development. For the local MQTT network, a main cross over area in the project, joint work was carried out to establish connection between devices. This is also the case with the Cloud network, where the frontend app is to communicate with the backend and database on the cloud. Additionally, a machine learning model is being developed to provide routine learning on the edge device (Raspberry Pi). Once each tier of the network has been established to function correctly, the two need to be connected and then the testing and validation section of the methodology can begin. This will be to determine the success of the dataflow throughout the network from sensors all the way to the app and database.

Mia is responsible for the development and testing of the sensor and actuator subsystem, including device configuration, MQTT publishing/ subscribing behaviour, and hardware integration. Daniel is responsible for the design of the system architecture, the implementation of the local MQTT broker and network, cloud backend developed in FastAPI, deployment to Render, and integrating the PostgreSQL database along with the frontend app. Alvaro is responsible for developing the machine learning inference component and integrating on the Raspberry Pi and local network. Luis is responsible for the development of the frontend application, including the user interface, dashboard visualization and interaction with the cloud API.

This interim report is separated into several key sections. Firstly, the Industrial Relevance section explored how the project aligns with the current industry and commercial solutions alongside emerging academic research in areas such as AI, IoT and edge-cloud computing. The theory section outlines the background information regarding each area of the project that underpin the system. The design section delves into how each area of the project is designed and the reasons as to why certain decisions have been made for the structure of each section. The methodology is concerned with the practical steps in development, configuration and testing of each subsystem for the sensors and actuators, machine learning model, the construction of the local and cloud systems and communication protocols, and the frontend app. This is followed by the results section, which presents initial findings, results of progress so far, and the implications of those results. The discussion in the report evaluates the significance of the results, assesses the systems performance and identifies areas that require further development.

2. Industrial Relevance

Technologically improved assisted-living systems have gained more importance in the world with increasing demographic, economic and societal demand. The ageing populations, increasing demand of domiciliary care and the chronic shortages of workforce are the main cause for the need of solutions that enable the support of independent living and minimise the dependence on overstretched care services. Simultaneously, the rapid development of IoT, edge computing and privacy-preserving artificial intelligence is transforming the expectations of how digital health technologies should operate particularly with respect to user dignity, data protection and long-term affordability. This section analyses the industrial environment around assisted-living technologies and describes the market forces, competitive opportunities, and regulatory limitations and social implications that support the existence of a low-price, privacy-focused alternative. All these aspects emphasise the business and ethical principles, based on which the suggested system has been developed.

2.1. Market Drivers

The primary objective of this project is to facilitate independent living for ageing populations and individuals receiving domiciliary care. Both groups traditionally rely on resource intensive intervention strategies, creating a critical demand for technology-based alternatives. This demand is driven by two converging global trends: demographic shift, with the World Health Organisation (WHO) [1] releasing projections stating that the global population over 60 will reach 2.1 billion by 2050. This rapid growth highlights the necessity for comprehensive public health response, as highlighted by the United Nations's 'Decade of Healthy Aging 2021-2030' [1]. The other global trend is the workforce crisis which is seeing the domiciliary care sector facing severe staff shortages, with 131,000 vacancies reported in the UK adult social care sector alone [2].

These factors have resulted in the Ambient Assisted Living (AAL) market value increasing to 60 billion USD, as of 2024 [3]. However, the current market is littered with high-cost operation and privacy concerns creating opportunities to create a low cost, privacy first alternative. nim

2.2. Competitive Landscape and Market Gaps

The current market can be split into two groups: dedicated healthcare solutions (StackCare [4], SECOM [5] and Taking Care Sense [6]) and generic smart home that can be repurposed for AAL needs (Alex Together [7], Apple Home [8] and Home Assistant [9]). Critical analysis of existing solutions has identified 3 main gaps. Privacy vulnerabilities, the use of cameras and microphone-based systems suffer from low user adoption due to privacy concerns. Cloud dependency, the reliance on external servers introduces latency and renders safety critical systems useless during internet outages. Finally economic barriers due to subscription-based pricing models, low-income demographics are priced out of many of the previously outlined products. Thus, this product aims to directly address these gaps through the use of passive sensing to eliminate audio and visual surveillance, and edge first architecture to ensure offline reliability and will utilise a one-off cost model to ensure accessibility.

2.3. Regulatory Compliance and Standardisation

To ensure commercial viability and user safety, the system is designed to adhere to strict international IoT and cyber security standards. These standards include EST EN 303 645[10] the global standard for IoT security. Aligning with ISO/IEC 30141 [11] referencing architecture for trustworthy IoT systems. Complies with the GDPR [12] and ISO 2701[13] principles regarding data minimisation and storage. Finally, HIPPA [14] guidelines regarding healthcare data portability.

2.4. Societal Impacts and Real-World Application

The system will produce deliverable benefits on both individual and societal levels. The project enables 'Ageing-In-Place' [15] reducing the burden on public health and social care systems. This is done through supporting independent living, delaying entry into residential care facilities and preventing avoidable hospital admissions through early anomaly detection. The project will advance applications of multi-sensor fusion and edge AI, demonstrating how complex behavioural analysis and routine learning can be performed without intrusive surveillance [16] Edge processing adhered to the GDPR [12] principles including data minimisation and local processing. This in turn increases the user autonomy and dignity which is a key ethical expectation of all assisted living technology. The 'Privacy-By-Design' approach to the project addresses the ethical principles of autonomy and dignity of the user, without the use of wearable devices or requirement of user interaction. This increases the users trust and results in higher adoption rates compared to visually intrusive systems. The integration of low power components, such as ESP32s and passive sensing reduces the systems carbon footprint compared to video-based solutions. The creation of user-friendly mobile app providing information on household routines allows for caregivers to spot anomalies early, without requiring technical training. Furthermore, the inclusion of proactive health behaviour monitoring by providing personal insights into the users' daily habits and environmental conditions gives provides reassurance to caregivers.

2.5. Commercial Feasibility and Future Roadmap

By creating a modular product this allows for scalable deployment, ranging from small single residential homes to entire care communities. The one-off cost model provides a competitive unique selling point for housing associations and healthcare pathways. Future development will focus on interoperability with current healthcare digital systems and integration of predictive analysis for insurance risk monitoring, which increases the projects commercial scope.

3. Theory

The theoretical background of this project provides the scientific, technological, and ethical principles that are applied in the design of an AI-based assisted-living system. In this section, the important concepts that guide the system architecture are summarised, including passive sensing technologies and IoT communication protocols, edge-based machine learning, privacy-preserving design approaches, and cloud-centric data management. The synthesis of the theoretical work of all the subsystems, local hardware, firmware, networking, behavioural modelling, backend infrastructure and mobile interfaces gives the chapter a coherent conceptual framework that guides the methodological and design decisions that follow later in the report are based. It defines the rationale behind technology selection, outlines the operational limitations of the field, and outlines the assumptions and safety concerns that shape the whole system.

3.1. IoT Introductions

Internet of Things systems rely on communication models that support efficient and lightweight message exchange between devices. When it comes to specific models there are two dominant models; publish/subscribe and request-response [17] in the first model, publishers (message senders) send data to a central broker, that forwards the message onto subscribers (consumers). The communication and data flow here is all based on predetermined topics and specific port. The benefits of this model include loose coupling, asynchronous communication, and flexible scalability [18] making it highly effective for sensor driven systems such as our Assisted Living IoT platform.

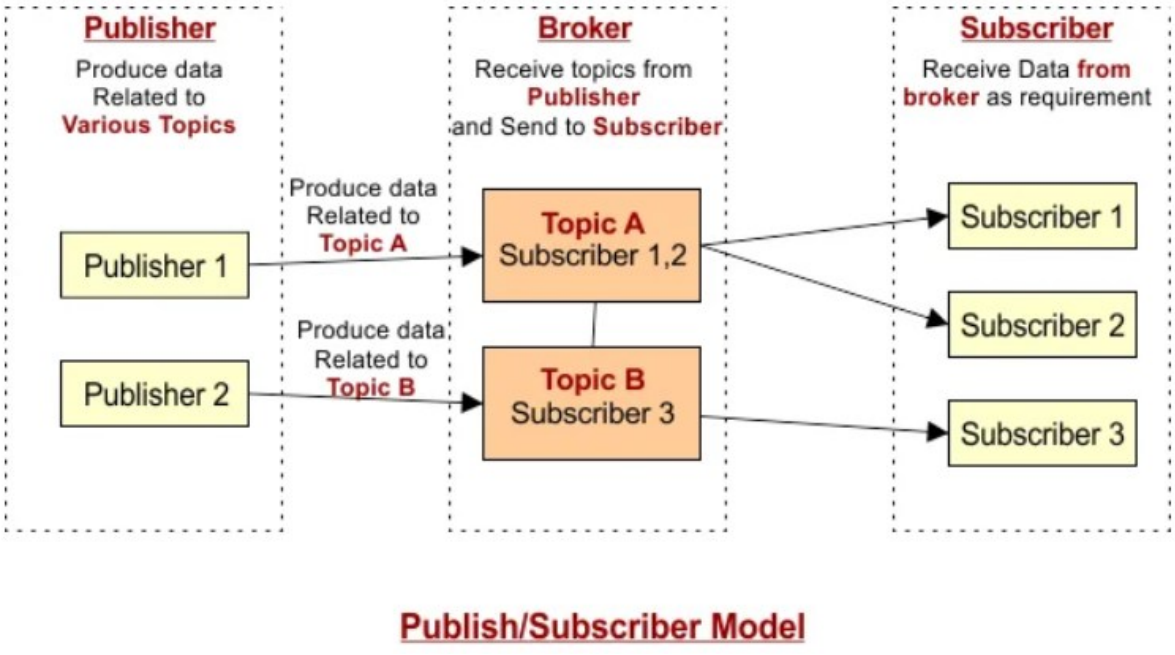


Figure 1 – MQTT Publish/Subscribe Model

The request-response model is the most traditional communication model in IoT that establishes a direct and synchronous exchange between clients and servers [19]. Interestingly, the request and response model is commonly paired with HTTPS based APIs. However, the fact it is a synchronous model does not make it an apt solution for our project. In addition, this model is less suited for real-time communication.

Figure 2 – Request-Response Communication Model

3.2. Principles of Passive Sensing

Unlike active sensing that requires user interaction, passive sensing uses ambient signals which are transmitted by the environment and naturally by the user's body [20]. Passive sensors generate discrete state changes based on environmental inputs [20]. This is the theoretical basis for sparse event monitoring, which allows sparse vectors to reconstruct complex daily living activities without compromising the user's safety or privacy [20]. The sensors selected utilise the principles of passive sensing to output specific parameters, providing information specific to the user's needs.

PIR sensors utilise pyroelectric effects to detect infrared radiation emitted by the body [21]. Fresnel Lens optics segment the field of view of the sensor into specific zones creating differential output voltages when the heat signature moves through the zone outlined by the field of view [21]. mmWave sensors are based on frequency modulation of a continuous wave [22]. The sensor monitors the frequency difference between the transmitted wave and the received wave, to determine the velocity and range using a fast Fourier transform via the Doppler effect [22]. Pressure sensors are based on piezo-resistive effect in which mechanical deformation causes changes in the conductive path density [23] within the force sensitive resistors following the relationship

$$GF = \frac{(\frac{\Delta R}{R})}{\epsilon}$$

Light sensors convert light energy into electrical signals through the photoelectric effect, where the light photons excite electrons in the semiconductor material [24]. The output changes based on the intensity of light [24]. The water sensor works by detecting changes in the conductivity, caused by the presence of water, between interdigitated electrodes [25]. Reed switches rely on disturbance of magnetic flux to physically bridge contacts or the Hall Effect representing discrete binary states of open or close [26].

3.3. Signal Processing & Estimation Theory

Nyquist-Shannon Sampling Theorem provides justification for the sampling rate used by each sensor [27]. The theorem is used to avoid aliasing by determining the sampling frequency from the equation below:

$$f_s = 2f_{max}$$

where the band width is based on the rate human motion [27].

Stochastic Parameter Estimation utilises Recursive Least Squares (RLS) which is a mathematical derivation of the RLS to minimise the weighted linear least squares cost function [28] as seen below:

$$y_k = x_k T_k + k$$

Additionally forget factors adapt the model to time-varying systems [28] allowing for identification of the user's behaviour between day and night.

Digital filtering will utilise Moving Average Filtering (MAF), using the mathematical equation below:

$$y_n = \frac{1}{N}\sum_{k=0}^{N-1} x_{n-k}$$

This will be used as a finite impulse response filter, which acts as a low pass filter to thus reduce the Gaussian noise within the system [29].

3.4. Actuator Drive & Control Physics

MOSFET switches will be utilised based on N-channel MOSFETS, which are operating within the saturation region thus creating a low-impedance switch for the actuator system [30]. Actuators such as relays, solenoids and small DC motors exhibit inductive properties. According to Faradays and Lenz law, any rapid change in current through an inductor will induce a voltage of the opposite polarity [31]. When MOSFET's transition from On to Off the collapsing magnetic field forces the current to continue flowing creating a back EMF voltage spike [31]. The transient spikes pose a major risk to semi-conductor devices and can exceed the MOSFET drain-source breakdown voltage causing major damage. To mitigate this flyback diodes will be placed in parallel with the actuator load [31]. When the MOSEFT turns off, the diode becomes forward biased, providing a low impedance path to safely dissipate the energy [31].

3.5. Feedback Mechanisms

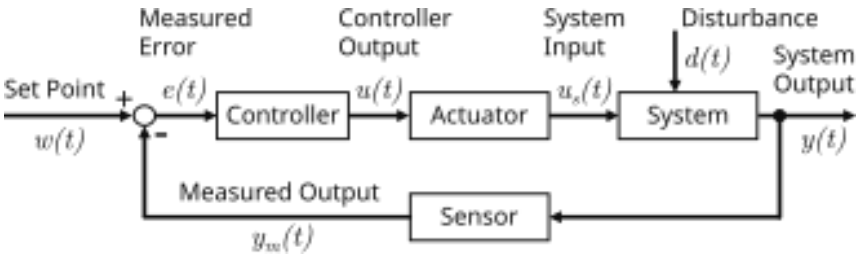


Figure 3 - Closed Loop Control System Using Sensors & Actuators

Closed loop systems will be used in which the output of the system is fed back and compared to a reference to generate an error signal which is used to activate actuators (Figure 3) [32]. In this system the Plant is the home environment, the system will not just observe but will implement changes for example, closing blinds and turning on lights, this alters the home environment and creates a feedback loop.

Bounded Input and Bounded Output (BIBO) Stability will be implemented as the sensing system can easily become unstable if noise entered can cause unbounded false positives [33]. Thus, the control must ensure that any environmental noise is stabilised thus, the output does not oscillate [33].

3.6. Multi-Sensor Fusion Architecture

Assisted-living systems use behavioural monitoring based on the combination of heterogeneous IoT sensor data to build a precise user activity and environmental context model [34]. The sensing modalities are complementary, such as PIR motion sensors can sense the overall movement in a room or the movement between them, mmWave presence sensors can sense the finer movements and

stationary usage, and pressure sensors can measure the bed-contact forces to identify sleep-state or the state of door and windows. Other modalities like light to detect time of day, flow or tap-state sensors to detect the usage of water and other future environmental sensors can also be added to the same fusion framework. These streams of data, together, facilitate the determination of more advanced behavioural attributes, including the occupancy rate, pressure stability, movement frequency, and environmental safety indicators, which cannot be determined by using individual sensors. The system can construct discrete behavioural states, such as awake activity, bed entry or exit, sleep, room absenteeism and potentially dangerous situations such as unexpected movement at night, long-term immobility, abnormal environmental measurements or water left running through multi-sensor fusion. Fused approach also enhances strength by minimising chances of false positives brought about by noise or momentary readings in any one of the sensor modalities [34]. The theoretical basis allows the anomaly-detection algorithm to consider not only individual sensor anomalies, but also discrepancies between correlated behaviours, so that future extensions of the sensor network can be smoothly incorporated into the overall model of home activity and safety [34]. Dempster-Shafer theory will be utilised to combine belief and plausibility from sensors which are transmitting uncertain data [35]. The theory is a mathematical framework for reasoning uncertainty, allowing a combination of evidence from different sources to form assumptions regarding unknown data [35]. This results in handling the unknown states being improved.

3.7. Edge Computing in Assisted Living Systems

Edge computing is a paradigm where a data source computes the data itself, instead of sending it to the remote cloud server [36]. This architectural decision is very imperative in assisted-living facilities because behavioural monitoring is performed in real-time, and the data about personal activities is sensitive [36]. This has many benefits including reduced latency, enhanced reliability and lowers bandwidth usage since communication to the cloud is limited only to necessary communication [36]. In addition, this approach improves privacy since less sensitive data is being transferred over the cloud and thus remains in the local network, reducing the possibility of it being intercepted [36]. Furthermore, if the cloud system were to fail, the edge computing processes can still occur, with data being stored temporarily on the Pi waiting to be uploaded when the cloud tier is back online [36] . This gives the system an added robustness and a solution to any fault with the cloud system. Local data processing on the Raspberry Pi minimises latency, enables dependency on the network, and sensor stream, e.g. movement, presence, and bed-pressure measurement do not leave the home unless summarised specifically. The local edge device is the main decision-making unit, which does preprocess, feature extraction, model inference, and anomaly classification. This method also enables the detection of time-sensitive trends (unusual movement at night, inability to get back into bed, excessive inactivity, etc.) without constant communication with the cloud. Edge-based processing is also fault tolerant: the system will keep working when it loses connections, which can easily happen within a home IoT deployment.

3.8. Publish- Subscribe Architecture MQTT

MQTT is a lightweight publish/ subscribe messaging transport that is ideal for connecting remote devices with small code footprint and minimal network bandwidth [37]. One of the key principles is decoupling in space, time and synchronisation, therefore publishers are not required to have knowledge of other device's identity or availability. Another key feature is Quality of Service levels, which provide tenable message delivery guarantees. These three levels are 'at most once (QoS 0), 'at least once' (QoS 1), and 'exactly once' (QoS 3) [38]. MQTT's low overhead also makes it suitable for embedded systems such as ESP32 microcontrollers and Raspberry Pi devices. This along with the fact

it can be utilised over a local network if the broker is installed locally makes it the perfect option for our project. Cryptographic protocols, Transport Layer Security (TLS1.2/1.3) will be utilised by using public key infrastructure in which asymmetric encryption is used for the handshake and symmetric encryption is used for the data session [39]. Additionally, X.509 trust chains will be utilised based on digital signatures which allow for verification of IoT authenticity nodes [39].

3.9. Model Theory, comparison and justification

Assisted-living environments need unsupervised anomaly detection since the irregular behavioural events cannot be fully labelled in advance, and the system consequently needs to learn normal routines on its own during a calibration phase. The Isolation Forest algorithm was chosen as the main detection mechanism in this project because it is suitable in real time and low power edge computing [40]. Isolation Forest works by recursively splitting the feature space with randomly chosen attributes and split values; data points in sparse or irregular areas are isolated with fewer partitions and thus get higher anomaly scores [40]. This algorithm can be used to perform effective detection of deviations in fused motion, presence, and pressure sensor data and has a low computational cost on the Raspberry Pi. Several alternative methods were considered [40]. One-Class Support Vector Machines are theoretically powerful, but have high computational and memory requirements, which makes them inapplicable to continuous inference on embedded devices with noisy IoT data streams [41]. Autoencoders based on deep learning offer high performance on large datasets, but need extensive computational resources, extended training time and in many cases learning with the help of clouds [41], which is incompatible with the privacy-first and personalised routine-learning goals of the system. Statistically pure or rule-based approaches were also taken into consideration, but they are not adaptable and are very vulnerable to sensor noise [41], particularly in multi-sensor contexts where contextual relationships need to be learned. Due to these reasons, Isolation Forest provided the best trade-off between accuracy, efficiency, preservation of privacy, and practical deployment. Its capability of functioning without labelled data, its capability to adapt to individual behavioural patterns and its capability to perform lightweight retraining, make it especially appropriate to the requirements and objectives of an edge-based assisted-living system.

3.10. Personalised Routine Learning vs. Generic Dataset Approaches

Conventional human-activity-recognition systems typically are based on models that are trained on large publicly available datasets like CASAS [42] or Opportunity [43]; these datasets represent population-level behaviour that does not scale to highly individual routines in assisted-living settings. The elderly often exhibit distinct locomotion, sleeping behaviours, daily routines, and interaction with the environment, so the generic models are more likely to generate erroneous classifications and increased false positives when used in actual houses. The machine learning methods that are based on datasets also generally involve a large amount of computation in the cloud, which adds latency and privacy concerns, especially in cases where personal activity data must be continuously streamed [44]. In order to overcome these shortcomings, this project uses a personalised routine-learning approach whereby the individual installation produces its own configuration file as part of an early calibration phase. This setup records user-specific anticipations including normal sleeping hours, mobility patterns, room utilisation, environmental limitations, and pertinent sensor interactions. The machine-learning model then learns to interpret the behaviour of the resident as normal instead of using behavioural templates. The theoretical benefits of this personalized approach are that it is more sensitive to minor deviations of behaviour, less prone to false alarms, more aligned with clinical and caregiving requirements, and more private since no external training datasets are required. Consequently, there will be a higher accuracy in the detection of anomalies, especially in the situation

where irregular sleep onset, reduced mobility, inconsistent room occupancy, or other gradual changes are involved and would have been hard to detect through generic dataset-based systems.

3.11. Cross-Platform UI Development

First, the choice of Flutter is justified due to its capability to develop a software base of a single code into Android and iOS code and binary applications. This minimises time of development, consistency of UI behaviour and reduces platform-specific errors. Flutter is well known to have great performance and hot-reload development cycle, which allows interactive components to be refined quickly, which is a great benefit during the initial stages of prototyping [45[Its rendering engine gives it almost native performance on animations, layout and reactive UI behaviour. Overall improving user experience [46]

3.12. Secure Client-Server Communication

The mobile application communicates with the backend through a FastAPI service secured using HTTPS, aligning with best practices for secure mobile data transfer. Research highlights that mobile applications are particularly vulnerable to data interception and manipulation when unencrypted communication channels are used, this is an issue which is critical in healthcare and monitoring systems in which sensitive data is transmitted [47]. FastAPI implements Representational State Transfer (REST) API, a type of application programming interface that allows communication between different systems over the internet [48]. RESTful APIs use HTTP verbs (GET, POST, PUT, DELETE) along with a uniform interface that enables the development of the backend and frontend systems independently [48]. The architecture defines cloud end points that receive summarised sensor data from the Raspberry Pi and return structured responses by the app. By operating using HTTPS, an extension of HTTP that incorporates Transport Layer Security (TLS), the system ensures communication between the app and server is maintains the three essential procedures of IoT communication: confidentiality, integrity and authentication [49][50]. Together, the use of RESTful design and HTTPS provides a simple, flexible and robust foundation for secure communication in assisted living systems.

3.13. Offline-First Data Persistence Architecture

The architecture design includes the offline-first design, in which the local database is the source of truth, and the data synchronisation with a remote database is only applied when the network is not unavailable. This trend is applied extensively in mobile systems which are used in settings with unstable connectivity. The research and development guides focus on the reliability advantage of storing data locally in SQLite and queueing updates to be synchronised in the future [51]. This type of design will make sure that recent sensor summaries and routines submitted previously can still be seen by caregivers in case the Raspberry Pi, router or cloud-based backend becomes inaccessible temporarily.

3.14. Human-in-the-Loop Behavioural Modelling

Lastly, the application facilitates a human-in-the-loop routine logging system, in which the caregivers can indicate the anticipated behavioural deviations. These manually created routines are a type of contextual data to the anomaly-detection model deployed on the Raspberry Pi. The human-generated context in AI-based systems is an established approach to lowering false alarms and enhancing the interpretability of the model, especially in personalised monitoring systems.

3.15. Security in IoT System

For IoT systems, security is of utmost importance and due to distributed devices deployed across a network, there is concern that potential attacks can occur over wireless communication. Secure

architectures require strong authentication, controlled authorisation using certificates and robust encryption of communication methods. MQTT, the chosen local network messaging service, supports certificate-based authentication at the broker. This means only trusted devices with recognised certificates can access the messaging network. In addition, both MQTT (port 8883) and HTTPS rely on TLS providing encrypted channel communication preventing any interceptions from unwanted parties. It is crucial that security in IoT systems is implemented and with the privacy first design, our project upholds this value.

3.16. Summary

Overall, the theoretical exploration supports the suitability of passive sensor networks and lightweight edge computation, secure cloud integration and anomaly-detection methods for an assisted-living for a privacy-sensitive assisted-living platform. The review emphasises that the subsystems; sensing, networking, ML processing, backend and user interface should be developed with interdependence, reliability as well as ethical responsibility in mind. Collectively, the theoretical contributions support the hybrid edge-cloud architecture that the project uses and explain how the selected technologies can be used to monitor the daily events proactively and without any intrusion. These foundations provide the conceptual bridge between the problem motivation and the practical implementation detailed in the design, methodology and results sections.

4. Design

The assisted-living system design signifies a multi-layered engineering initiative that incorporates the IoT sensor networks, edge-based machine learning, secure cloud infrastructure, and cross-platform mobile application into a unified architecture. Since the project is in a safety-critical healthcare environment, reliability, scalability, fault tolerance as well as privacy-by-design were prioritised in the design process of every component in the project. The design section therefore does not only describe the structural layout of the system, but also the reason why the technologies chosen at each level.

The highest tier is based on a hybrid two-tier architecture where the Raspberry Pi 5 is the local controller and edge-processing unit, and a cloud backend, operated on Render, is used to store data over the long term, authenticate, and access it remotely. The hybrid model has been chosen based on the consideration of both local and pure cloud-based solution because with the hybrid model, low latency local decision-making and high-reliability remote accessibility can be achieved consequently enabling the system to keep operating even when internet connectivity is not available.

At the local level, the sensor nodes and actuator modules based on ESP32, and the Mosquitto MQTT broker are an autonomous network with the ability to monitor the environment, track behaviour, and actuate in real time. The firmware that is installed on ESP32 boards includes low-power architecture, state-machine control, and safe communication patterns, where the data acquisition is guaranteed to be robust in a residential setting. The sensor ingestion, preprocessing, behavioural modelling using Isolation Forest anomaly detection, and generation of structured summaries are implemented in the Raspberry Pi as part of the edge-ml subsystem.

4.1. Overall system Architecture

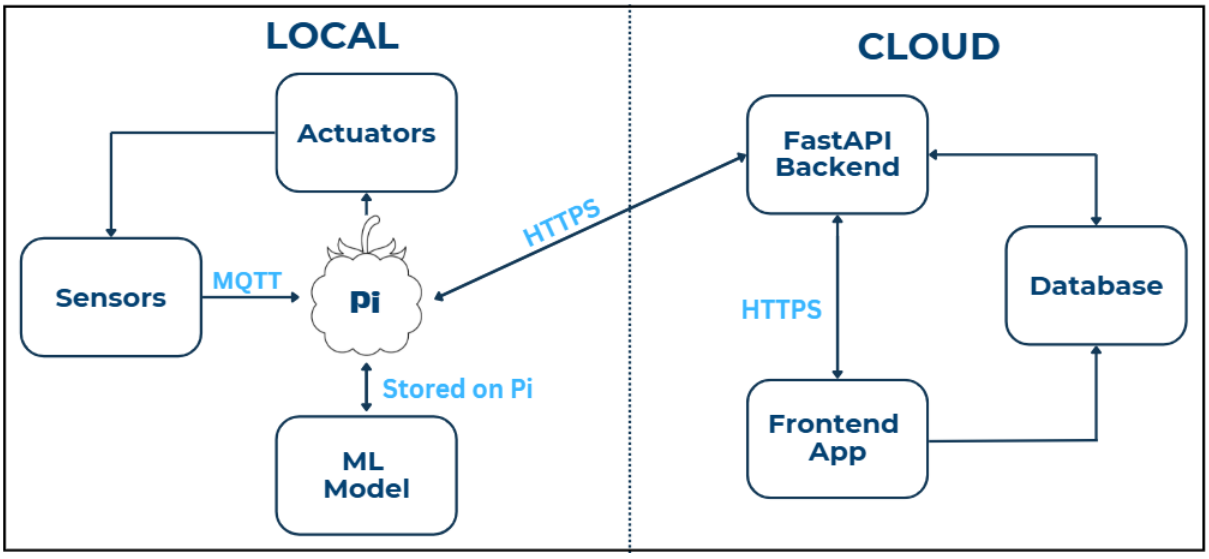


Figure 4 – Hybrid System Architecture

The architecture of the system is crucial to the functionality of the project and various solutions were considered during the initial design phase. Initially, there was much deliberation over the design of the system architecture. Observing existing solutions online such as AWS IoT, many are purely cloud-based systems and require subscriptions that can be expensive. Furthermore, our project requires individual devices to be connected to a local network not straight to a cloud platform. Therefore, a purely cloud-based system would not be applicable for our system. Next the potential of an architecture that is fully local was considered. This was a viable option at first, since the sensors and actuators could connect to a local broker installed on the Pi, the machine learning model could remain

local, and a local database could be explored. However, issues arose when incorporating the frontend app. Without any connection to the cloud, the app would only be able to connect to the network when in range to connect to Wi-Fi. This means the carer could not access the system when away from the home. Alongside this a purely local system is not scalable and ultimately not as viable as our hybrid architecture shown in figure 4.

The overall design of the system is split into a two-tier architecture consisting of a Local System running on a Raspberry Pi and a Cloud System hosted on Render. Communication between each tier is managed through secure protocols to ensure reliability and data integrity. The communication protocol opted for in this case is HTTPS [52]. Later in the report the reasons as to why HTTPS was opted for over other protocols along with advantages for the project. The architecture accounts for offline functionality through the local MQTT network, while still integrating with cloud services for remote monitoring via the app, a feature that in case of a fault, the system can still function and will not fail.

4.2. Local System

At the core of the local network is a Raspberry Pi 5. This is the central hub and provides a combination of performance, connectivity, and flexibility that makes it well suited for an IoT system acting as a local controller, MQTT broker and edge computing device. The Pi 5 features a 2.4 GHz quad core ARM Cortex-A76 CPU [53] which is a major upgrade from previous models and a significant reason as to why this model was selected. This translates to faster handling of MQTT messaging, ability to host the machine learning model locally without strain, and capacity to run Python services and background processes simultaneously. As the system grows, adding more sensors, actuators and in-depth machine learning, the headroom available ensures long-term viability.

The Pi 5 also boasts excellent network connectivity, with studies showing the Wi-Fi connectivity of the Pi 5 delivered a sender bitrate of 217.6 Mbps and a receiver bitrate of 216.3 Mbps. That is 341% faster than the Pi 4's rates of 63.8 and 63.4 Mbps respectively [54]. In addition, when set up in a home, the Pi's connectivity means sensors will not be compromised as to their location and maintain a strong connection to the Pi. It also boasts stronger reliability which is critical in an IoT system.

On a broader scale, the Raspberry Pi offers many advantages compared to a laptop or micro server, the first being lower power consumption which is cheaper to run. Finally, the Pi is safe and relatively inexpensive to deploy in an edge setting such as this project. All these points matter for an IoT system which like our Local network is expected to run 24/7.

In figure 5, the Pi is shown as the central hub and as previously mentioned has the Mosquitto broker locally installed. The publishers in the network are the sensor ESP32s nodes which act as MQTT clients. They fit perfectly in the design due to their Wi-Fi support, lightweight MQTT libraries and low power consumption. In addition, if needed ESP32s can operate as both publishers and subscribers, depending on how they are connected to the sensors and actuators. The actuators in the local network act as subscribers, receiving messages to actuate a response, once processed by the machine learning model locally on the Pi. However, this will be discussed in greater detail later in the report.

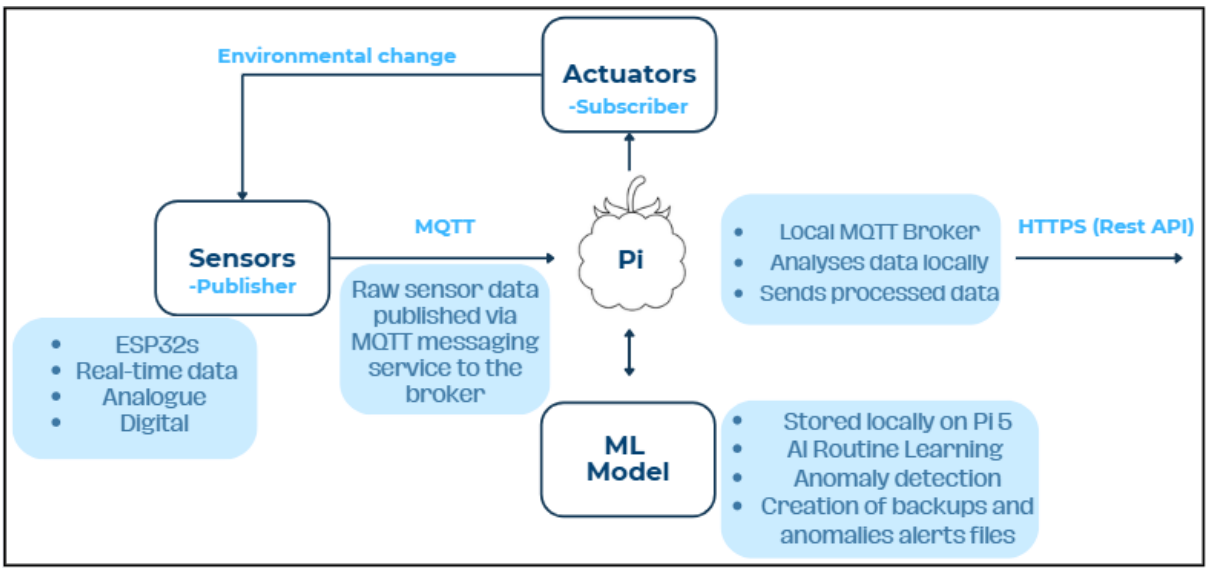


Figure 5 - Local System Architecture

4.2.1. Hardware Design

Sensor nodes utilise ESP32 modules. These modules were chosen over other MCUs for several factors including 2.4GHz Wi-Fi connectivity. Both ADC and GPIO channels allow for the use of both analogue and digital sensors. Low-power modes including deep sleep, result in the system only consuming micro-amps of current, using the integrated ultra-low power co-processor. Furthermore, the hardware crypto accelerators allow for TLS implementation which enables MQTT over secure port 8883.

The battery system integrates NiMH rechargeable batteries combined with a boost converter ensuring a stable 3.3 V voltage input even as the battery drains. NiMH chemistry has been chosen over LiPo, as LiPo batteries pose a fire risk if punctured or overcharged, whereas NiMH remain chemically stable, thus ensure safety in the assisted living context [70]. 100nF capacitor will be placed close to the VCC pin to filter high frequency switching noise, and a 10uF capacitor acts as a bulking reservoir to prevent voltage sag during high current Wi-Fi transmission. Battery operation reduces the overall installation burden and the cost of the total project.

A passive sensor suite has been selected for low power operations including a PIR, mmWave radar, pressure sensor, light sensor**,** reed switch and water leak sensor. The design of the circuits for each of these sensors can be seen in Figure 6. The passive sensors selected maximise privacy and the use of multiple sensors reduces the number of false alarms. 10kOhm pull down resistors will be implemented into the GPIO inputs, if necessary, to ensure that there is always a logic 0 during the ESP32 boot sequence, preventing false triggers during sensor initialisation.

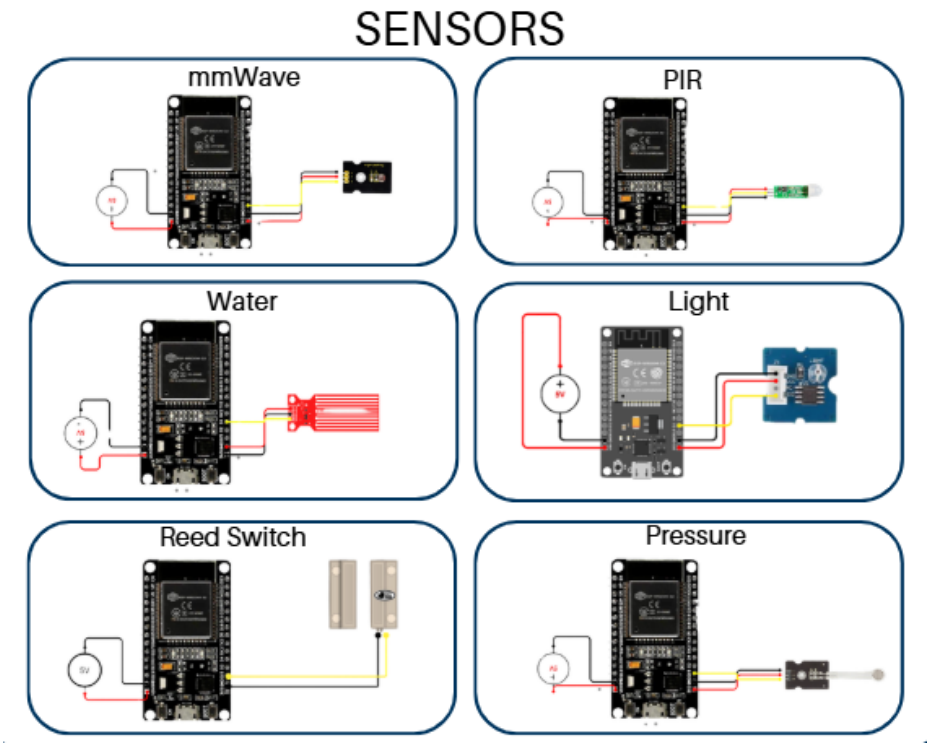


Figure 6 - Sensor Suite Circuit Diagrams

To obtain the most accurate sensor readings optimum locations have been identified (Figure 7). The PIR is placed above doorways and angled downwards for the optimal Field of View (FOV). mmWave will be mounted on walls and ceilings away from reflective surfaces. Pressure sensors will be placed beneath mattresses and cushions to detect when the person is present on an object. Reed switches will be placed in the door jamb at the top for detection of open and closed doors. The light sensor will be placed near the windows to detect the ambient room conditions, and the leak sensor will be placed at floor level near sinks and pipes detecting any water leaks which could cause environmental damage and flags anomalous behaviours.

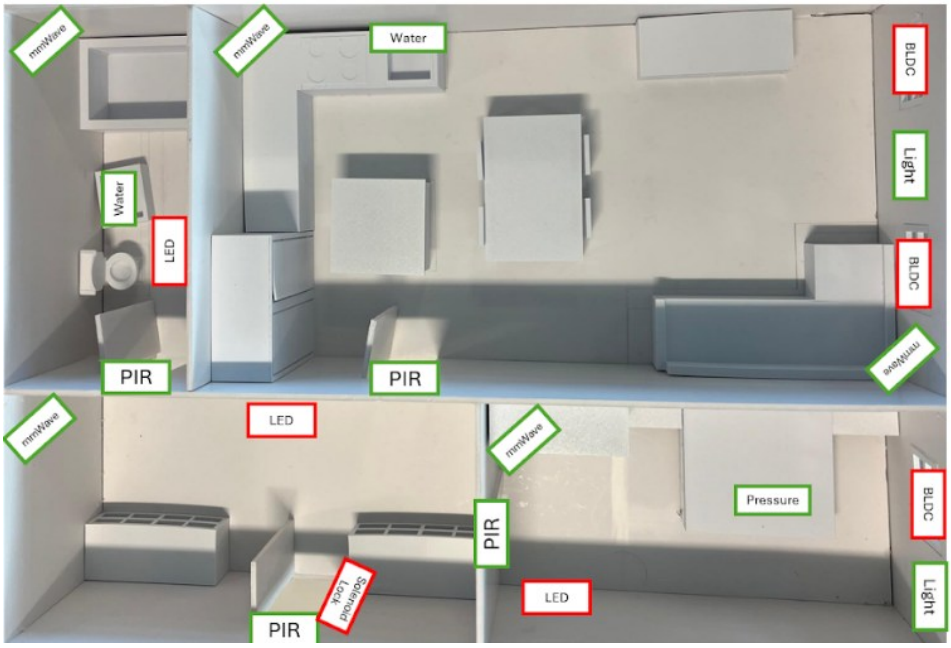


Figure 7 - Sensor Placement Map

The following actuators have been chosen. LED for safety lighting during the night, stepper motors for blind actuation and a solenoid lock for access control. The circuit design can be seen in Figure 8 **.** A dedicated driver circuit for each of the actuators is required. The LED utilises a n-channel MOSFETs with a current limiting resistor and flyback protection for inductive loads, MOSFETS were chosen due to their low On-Resistance, compared to p-channel equivalents [71] , thus allowing for them to be driven directly by the ESP32 voltage without requiring a voltage shifter. The stepper motor has a ULN2003 and 100uF capacitor across the motor rails to reduce noise. For the small-scale prototype, a 5V motor has been selected to meet low voltage requirements for the small scale environment testing. This will be revisited for the full-scale prototype with possible utilisation of relays. Solenoid locks will use a low-side switch configuration in which a MOSFET operated as a solid-state switch, avoiding the need for a separate gate shifter, Schottky flyback diode clamps the voltage preventing damage to the MOSFET or ESP32. Power isolation will be implemented to isolate the 3.3V from the actuator supply. A shared ground will be implemented safe references point. By doing this brownout of the ESP32 are prevented during the motor startup. In the event of a system failure a safety design features will be implemented. All actuators will have a fail-safe state, during boot they will be set to OFF the same will be implemented if there is a loss of the MQTT connection. Watchdog timers will be utilised for actuators which remain in a stuck state.

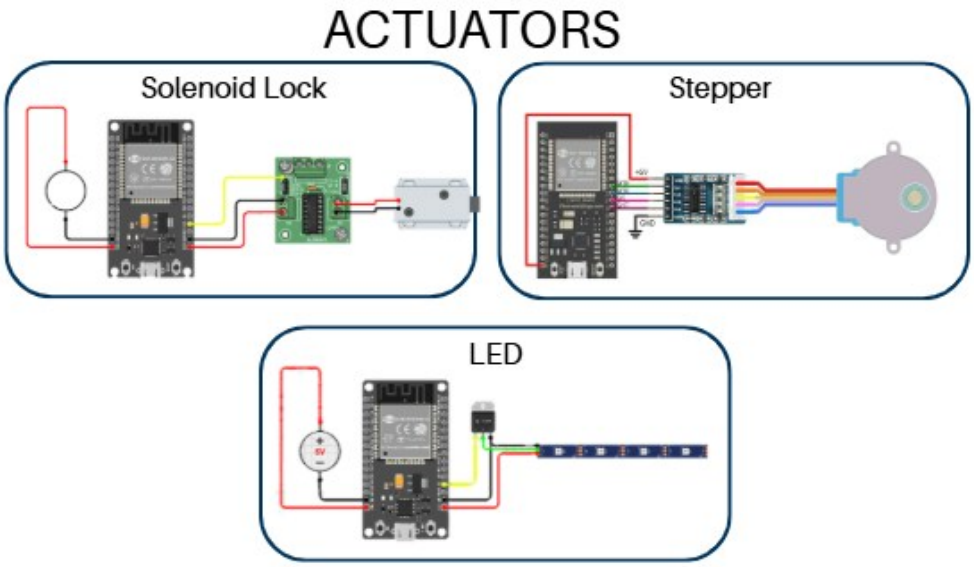


Figure 8 - Actuator Circuit Diagrams

The system will implement several safety features including actuator fail-safe OFF state. hardware isolation between logic and motor supplies and reverse polarity and overcurrent protection. Reliability features include deep sleep, to conserve battery and stabilising of node behaviour, MQTT reconnection logic, watchdog timers, edge-first processing to minimize latency and redundant sensing to reduce false alarms. There is a focus on Privacy -By-Design by not using any audio or video sensors, local processing to reduce to cloud exposure and data minimisation with only derived states being transmitted.

4.2.2. Firmware

Each sensor will have an individual C++ script; all these scripts will follow similar overall architectures to ensure data consistency. Each sensor follows a consistent topic hierarchy in the form `home/<room>/<sensor_ type>`. Each ESP32 then has the program uploaded to transmit raw data which is then subscribed to by the Raspberry Pi. Here the data is collected and converted into a.

JSON format ready for machine learning. Using the MQTT wildcard subscriptions, the gateway will be decoupled from the node, allowing new sensor nodes to be added dynamically to the system without the need to update or restart the Python collection script.

The programmes for the sensors will use non-blocking FSM with states `IDLE, SENSE, PROCESS, TRANSMIT` and `SLEEP`. This prevents network latency from blocking the sensor polling loop, ensuring the system remains responsive to immediate hazards even during network reconnection attempts. PIR, reed and leak sensors will be configured as interrupt sources and wake up ESP32 from deep sleep. ADC sensors are sampled multiple times, averages and then thresholded.

The firmware utilises battery-aware design, with deep sleep being implemented between readings, dependent on the sensor type a transmit on change event model will be implemented. Heartbeat packets are utilised at fixed intervals to provide information and ensure sensors are still online. Adaptive duty cycling will be utilised to build a battery aware design.

Actuators will subscribe to the MQTT using the following topic hierarchy `/home/<room>/<actuator_type>`. Commands will then be parsed using the `Arduino.Json`, with accepted fields including, `"state".ON/OFF`, `"brightness". 0-1`, `"position".0-100%`, `"lock_state".LOCK/UNLOCK`. The actuator firmware will implement safety logic in the form of auto-OFF timeout protocols, MQTT reconnect safe mode and default OFF after reset.

4.2.3. Local Communication and Network Design

The local communication system is built around the MQTT publish-subscribe model implemented using a client-broker model. This model provides many advantages for a system that consists of multiple devices producing and consuming data constantly. Hosting the Mosquitto MQTT broker on the Raspberry Pi establishes the Pi as the central piece of the local network. Several benefits arise due to this design, the first being local autonomy. In the case of no internet access, the system continues to function. Actuators can still respond to sensor data, if a response is predetermined, since all communication occurs locally and is not dependent on the cloud. MQTT also provides low latency message delivery, enabling rapid sensor-actuator interactions and real time responsiveness. With the Pi as the central point, it simplifies data forwarding and authentication and removes the need for each device to individually communicate with the cloud, which reduces the complexity of the network and improves the reliability.

There are other security and reliability considerations when opting for a communication protocol. For example, MQTT supports secure connections through TLS. The default port for MQTT over TLS is 8883, supposed to the less secure port 1883 [55]. This provides encryption for enhanced security, bolstering the robustness of our local network. In addition, device authentication via certificates is to be implemented to further secure communication over the local network. MQTT QoS levels ensure IoT message delivery reliability [56]. There are three Quality of Service (QoS) levels, QoS 0 (at most once), QoS 1 (at least once) and QoS 2 (exactly once) that MQTT supports [56], however the system predominantly uses QoS 0: for high frequency, non-critical sensor telemetry and QoS 1 for control signals where ‘at least once’ delivery is required to ensure actuator commands are not missed [56] . QoS 2 has greater complexity and increased bandwidth usage and hence is the slowest and most resource heavy level [56]. Due to the nature of our network, it is unnecessary to have this advanced level along with the fact it increases it increases the chance of buffer exhaustion and increased latency for ESP32s [56].

4.2.4. Machine Learning

The AI subsystem was developed as a modular edge-based architecture which can run on the Raspberry Pi only and through which all sensor streams are received, processed, and evaluated locally so that the privacy can be preserved and the latency as minimal as possible. The architecture includes: an MQTT subscriber serving as a multi-sensor intake, preprocessing and features extraction pipeline, a trained Isolation Forest model used to score anomalies and a hybrid validation layer that cross-cheques the model outputs with the behavioural context. The supporting design features included logging, creation of backups and secure cloud communication, which enabled the system to generate structured summaries and anomaly reports without revealing raw data. This structure is modular, which means that sensor handling, model inference, behavioural interpretation, and cloud integration are separated clearly.

The choice of a machine-learning-based design rather than a conventional rule-based or threshold-driven programme was since behaviour patterns in assisted-living settings are very individual, changeable over time and are affected by minor interactions between sensor readings. The traditional programme would have predetermined activity, sleep, frequency of movement, change in pressure, or environmental changes, but these are fragile and hard to generalise and can give false alarms when residents have natural deviations in routine. An unsupervised learning method like Isolation Forest can create a custom representation of normal behaviour using just the sensor history of the resident and can adapt itself to slow changes over weeks or months. With this method, the subsystem can identify complex anomalies like irregular onset of sleep, repeating night-time bed exits, inconsistent presence-pressure pattern or unusual inactivity that are not easily detected with the help of static rules. Moreover, multi-sensor fusion generates a behavioural feature space that is too high-dimensional and context-dependent to be dealt with using manually engineered logic but can be used by machine learning to generate greater robustness and sensitivity. In this way, machine learning at the design level can give scalability, flexibility, and long-term stability which cannot be achieved by deterministic programming in real-world assisted-living settings.

The subsystem was developed to support the proper behavioural modelling by incorporating IoT sensors such as PIR motion, mmWave presence, pressure, reed switch, light and water-flow sensors. All modalities provide certain behavioural clues that are integrated together into a single feature. These characteristics are the frequency of activity, time of occupancy, pressure stability, environmental safety signs, and individual behavioural anticipations based on a configuration file. The personalised routine-learning file was created to encode personalised sleep windows, movement patterns, environmental thresholds and room-usage habits so that the anomaly-detection pipeline can adjust to the routine of the resident instead of using general templates. Future sensors also accommodated by this fusion framework will allow the system to be full household monitoring.

To test the subsystem before the full deployment of hardware, a physically scaled bedroom simulation environment was created to mimic realistic behavioural sequences and environmental events. Sensors detect pressure, motion, presence; that can be experimented on the model through all the regular operations and the edge case conditions. The block anomaly-classification was created to interpret Isolation Forest scores and multi-sensor consistency checks and generate severity-tagged events, which are stored in the JSON format and sent to the cloud backend when needed. In general, the design of the subsystem focuses on modularity, extensibility, privacy protection, and real-time performance, which will guarantee its compatibility with the present and future phases of the assisted-living project.

To smooth the starting process of the whole system a launch file has been implemented. This file can obtain reading from each of the bedroom sensors, one at a time or the three of them simultaneously.

A first approach to multi-sensor fusion has been implemented in the project to reduce the number of false positives, improve confidence in behaviour classification and allows for the combining of complementary sensing modules; despite a good first approach some updates are still require in order to archive the highest possible accuracy during training period.

4.3. Cloud System

The cloud system of the architecture consists of a backend, frontend and database. It provides remote, secure access for the user along with storage of long-term data. The cloud system ensures that carers and family members can view or interact with sensor data, even when they are not connected to the local network, hence it is a critical tier of the hybrid system.

As previously mentioned, three main components make up the cloud system: FastAPI Backend, the Render hosting environment, and a PostgreSQL database. The Pi communicates with the backend via encrypted HTTPS requests, where data is then validated to be stored in the cloud database. That same backend allows communication from the frontend to retrieve data or device information when required. This architecture separates local operations from cloud services, ensuring reliability when internet connection is lost.

4.3.1. FastAPI

The reason for choosing FastAPI as the backend framework is due to its high performance, request handling, and ease of integration with Python. This is ideal since the environment on the Raspberry Pi is also Python. The backend ensures that incoming sensor data is checked and validated before entering the database which reduces any error leading to a potential fault in the network. FastAPI also defines RESTful (Representational State Transfer) [57] endpoints that allow the frontend app to request data and user information. In addition to this, the backend is responsible for error handling and logging adding to the reliability of the communication network.

The framework of the design is FastAPI Backend which implements RESTful services, the communication style [58]. This RESTful API structure uses standard HTTPS methods (GET, POST, PUT, DELETE). Each part of the network for example sensor readings, device records, and user accounts is represented as a dedicated endpoint. All data is transferred in the same way and of course compatibility with the Pi in the local network is crucial, which simplifies development and potential for expansion.

4.3.2. Cloud Hosting Render

Render is a cloud hosting platform that deploys the FastAPI Backend and PostgreSQL database. Render uses a GitHub connected service and builds and deploys the application using ‘requirements.txt’ from the FastAPI environment. Render simplifies the deployment process and runs the server once pushed from the GitHub repository. It is ideal for this project since even the free tier supports detailed projects greatly.

The first reason Render was chosen as the web hosting platform, was its seamless integration with HTTPS protocol. It provides managed HTTPS, meaning TLS certificates are automatically issued without manual configuration [59]. This is vital for secure communication between the local network, more specifically the Pi, and the cloud. Render also offers auto-scaling, health checks, service restarts and

persistent logging making it a reliable platform for hosting a high-end backend [60]. The implications of these benefits are simplifications in maintenance of the system allowing more focus on other developments rather than management.

4.3.3. Database

The database stores long term project data with the likes of timestamps, sensor readings, user credentials and device information. The tables will be structured to support efficient queries and indexing methods will be implemented for frequent operations when retrieving sensor data from the app. In addition, backups of the routine learnt using the machine learning model will be stored in the database

The database is still in the design process, however much deliberation between MongoDB and PostgreSQL took place. Each were viable options due to their many benefits; however, PostgreSQL was initially opted for due to its integration within Render. Our system requires structured, relational data storage with guarantees for strong consistency. All the data stored in the database previously mentioned all follow predictable schemas that benefit from PostgreSQL's ACID- compliant transactions [61]. Unlike MongoDB PostgreSQL is a relational database, and these for properties: atomicity, consistency, isolation and durability (ACID) [61] are crucial for a database like this. PostgreSQL integrates seamlessly with FastAPI through ORM tools such as SQLAlchemy [62], which makes development much easier and reduces the risk of inconsistent data formats which can be detrimental to databases. Overall PostgreSQL was chosen for the design of our project because of the reasons above, along with the fact data is structured into tables, rows and columns like a spreadsheet [63] which is perfectly suited to our system's data, which follows a set structure.

4.3.4. Mobile Application

The interface is structured into three tabs which is similar to the conventional usability structures present in contemporary mobile applications. The declarative UI paradigm used by Flutter guarantees that every tab is based on the specifics of the internal state of the app, and it remains visually predictable and consistent to the user [45].

The current tabs are Sensors Tab, showing the summaries of sensor activity such as room, type, last update time, and icons. The UI will take in lists of objects (mock data at the moment) to ensure that real-time summaries can enter the cloud backend without any problems in the future. Routines Tab providing caregivers an opportunity to establish structured records of the anticipated behavioural deviations. This will allow to form dialogue that can be used to validate inputs, date-time selection, and categorise by routine scope (e.g., daily, one-time). Settings Tab which allows for meta-information regarding the application (version, build type) and reveals configuration settings, including the API base URL and house ID. This division aids modularity and makes networking parameters not to disrupt the UX flows. Basic in every app development

The design is also minimalist regarding cognitive load, making caregivers (who may be working under pressure) able to use the system with ease.

Its internal architecture is based on the concepts of clean architecture and repository abstraction, with the UI and the data sources being separated. This design follows the generally accepted mobile engineering principles that focus on modularity and maintainability [51].

The project will be divided into modules: core/config of environmental variables like API URLs, router/core dynamic router with GoRouterm core/theme on global theming based on Material 3

guidelines, domain specific UI and data logic features/sensors, features/routines, and features/settings.

At this phase, local mock repositories provide all the information and functional development can be done without the backend being ready. Repositories will be replaced with RemoteRepository [64], which is aversion control system hosted on a server that stores the source code outside the local machine variants without changing UI code when FastAPI backend is supplied [65]. This design methodology is in line with separation-of-concerns ideas recorded in contemporary mobile apps platforms.

Even though the backend will still be deployed, the architecture already has a REST-over-HTTPS communication layer in it based on the Dio library. The network request format, anticipated destinations and templates of response parsing have been ready. The use of HTTPS communication is done to address the identified vulnerabilities where sensitive information may be transmitted via unsecure channels [47]. Moreover, the system will be designed to store local data using SQLite (via Drift). Offline-first systems enhance the reliability of mobile systems with intermittent connectivity and are common in production-scale IoT systems [51].

To ensure that the mobile app only communicates with the cloud layer, a different database instance has been configured to provide the storage of credentials and regular records, as well as to ensure that the physical sensors and the Raspberry Pi do not interact with the database. This is in line with the best practises of IoT systems security where authentication, storage and device-level operations are maintained modular to minimise attack surfaces.

4.4. Bridge between Local and Cloud system

With this hybrid approach for the system architecture, there must be communication between the two for the full system to function. This is where the Raspberry Pi acts as a bridge between the local MQTT network and the cloud backend. The communication protocol with the cloud is handled using HTTPS, providing end to end encryption using TLS 1.3 [66], ensuring the integrity of the transmitted data and upholding the privacy first design of our system. This allows the Pi to reliably forward sensor data, once processed via the machine learning model on the Pi, to the cloud without exposing the network to external risks.

Data is transmitted to the FastAPI Backend through REST API calls, where it is stored in the cloud database, made available for remote viewing via the app by the carers. This design creates a hybrid architecture in which MQTT manages low latency communication in the local environment while HTTPS supports long term data storage in the cloud tier and other cloud interactions.

4.5. Prototyping

To validate the sensor placement and overall system functionality prior to full scale implementation a physical scaled environment will be constructed (Figure 9). The environment will simulate a single-level apartment, similar to the test environment for the full-scale deployment. High-density foam will be used to replicate the walls and FDM 3D printed components will be used to simulate furniture and obstacles. The scaled environment will act as the primary validation platform providing information in regarding both sensor and actuator function.

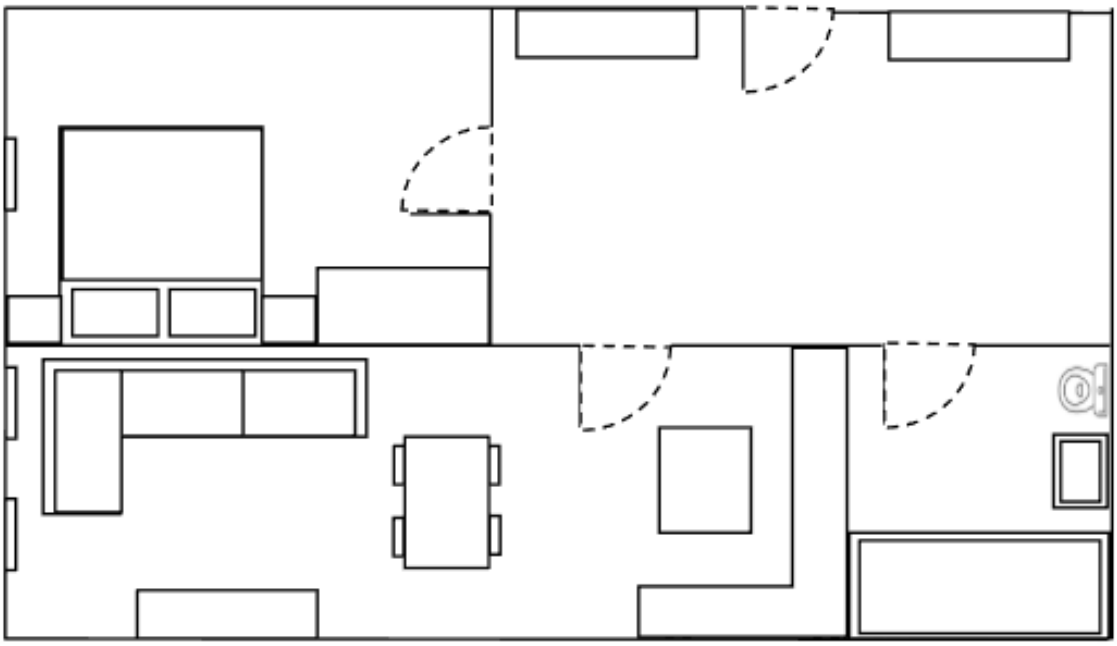


Figure 9 - Small Scale Environment Design

4.6. Summary

The assisted-living system design has managed to create a scalable, secure, and robust architecture that can support the current functionality of the system and its future growth. The hybrid structure, i.e. a local network with a Raspberry Pi and a secure cloud service, allows the system to balance between safety-critical responses with low-latency local processing and high-availability cloud services, which are required to maintain data persistence and make the system accessible to the caregiver. This will make the system still be functional even when the network is not functioning and it will still be possible to monitor remotely and analyse the behaviour over the long term.

In all the design subsections, there are three common themes, modularity, fault tolerance, and privacy. The hardware architecture focuses on low-power embedded sensing, passive and privacy-friendly modalities, and high electrical isolation of safe actuator control. Firmware architecture is based on non-blocking state machines, deep-sleep strategies, and MQTT wildcard subscriptions to ensure the high efficiency of the node and scalability. The local communication layer enhances autonomy by a broker-based MQTT network, which is backed by TLS-based secured channels and suitable QoS degrees to provide predictable message delivery. The machine-learning subsystem is developed with a distinct separation of data ingestion, feature extraction, anomaly scoring and behavioural validation, making it easy to upgrade the model in future without interfering with downstream components.

The cloud level supplements the local infrastructure with the credible HTTPS communication, organised endpoints based on the REST and the ACID-based relational storage, which allow the long-term analytics and integration with the Flutter mobile app. This modularity is reflected in the mobile interface itself with clean patterns of architecture, repository abstraction, and flexible structures of the user interface architecture to enable scaling in the future, the addition of sensing modalities, and more user interaction.

All these design choices lead to a system that is technically sound and also meets the real-world needs of assisted-living environments, that is, safety, usability, privacy and long-term maintainability. This design phase is the basis on which other phases of implementation and testing can be carried out

efficiently and there are clear interfaces between the components and less architectural work would be necessary as the project advances. The modular subsystems, a hybrid architecture and future-oriented design decisions preconditioned a powerful and scalable assisted-living platform that can be extended to include future improvements in sensing, anomaly detection and user interaction.

5. Methodology

This section outlines the methodological approach used to develop and validate an assisted living system. In alignment with engineering best practice, the methodology combines hardware prototyping, firmware development, distributed sensor deployment, data handling, machine-learning preparation and early-stage application design. Emphasis is placed on a systematic, bottom-up workflow to progress from component level testing to subsystem integration, all supported through iterative verification through testing methods of the associate subsystem. The method reflects ethical, safety and privacy requirements associated with assisted living technologies, demonstrating how these considerations informed architectural decisions in all project areas. Through these aspects strong technical foundations on which the system will be built have been made, thus ensuring system robustness, interoperability and real-world feasibility.

5.1. System Development Framework and Iterative Approach

The methodology for the project will be structured around an iterative multi-layer development framework that ensures reliable subsystem validation prior to full scale deployment. The workflow will follow a combine V-model and pipeline orientated approach with sensing, communication, processing and visualisation components are developed in parallel and integrated systematically.

The sensor and actuator subsystem will develop through two iterations. Iteration one focuses on electrical characterisation of sensing modules, driver circuit verification, firmware logic stability and establishing of secure MQTT within a small-scale prototype home. Iteration one sees the project being deployed in a full-scale test scenario, this iteration focuses on the optimisation of signal conditioning, validation of autonomous power management and integration of multi-sensor fusion algorithms.

The edge processing and machine learning subsystem will be built through a two mode operational workflow comprising training mode and inference mode. In training mode, normal behaviour data will be collected locally and used to train a personalised Isolation Forest model. In inference mode, real-time sensor measurements processed by the Raspberry Pi and compared against the pretrained model to detect anomalies.

The cloud and application layers prioritise modular infrastructure development. The cloud backend will be deployed early to establish a stable API and data-handling structure, while the mobile application will be initially developed using mock data and repository abstraction to allow front-end progress independent of the backend development progress.

This combined development approach ensures that each subsystem is validated independently before end-to-end integration.

5.2. Local System Development

5.2.1. Scaled Prototype Environment Construction

To validate the spatial configuration of the sensor and actuator networks prior to full scale installation, a scaled environment, using a 1:75 ratio, will be created of the simulation environment. The scaled environment will be constructed from high density foam to represent the walls of a single level apartment (Appendix B) . To model furniture and other key aspects of the home, models will be initially created using Creo CAD software and then manufactured using Fused Deposition Modelling (FDM) 3D printing to accurately represent physical objects (Appendix B) The creation of this environment aids with testing of Line-of-Sight (LoS) limitations of the mmWave and PIR sensors, leading to iterative improvement of sensor mounting positions to minimise any detection dead zones.

5.2.2. Sensor & Actuator Hardware

The sensors are chosen to map specific behaviours including, movement, bed occupancy, presence and environmental hazards while simultaneously adhering to strict privacy protocols in which no optical cameras or audio devices are used. Furthermore, complementary pairs will be identified to aid in future multi-sensor fusion and minimise the rate of false positives.

Prior to sensors being combined with ESP32 microcontrollers, sensor outputs will be analysed using a digital storage oscilloscope. This stage allows for the characterization of transient responses referred to as 'warm – up' times and signal rise times to calibrate firmware delay logic in future steps. It also enables noise floor analysis in which measurements of baselines voltage jitters can be identified allowing for the calculation of appropriate hysteresis levels for digital triggers and any filtering elements required.

To allow for reliable data acquisition within the noisy ESP32 environment the hardware will integrate specific signal conditioning to the circuit prior to digitisation. All sensors will be classified as either digital or analogue. ESP32 pins will be allocated according to this with digital sensor mapped to GPIO inputs and analogue to ADC pins. For the digital sensors, PIR and Reed switches, signal outputs will be wired to their respective ESP32 GPIO pins. External pull-down resistors will be added to the sensor to prevent floating states when the sensor is an open circuit. For analogue sensors including, pressure, mmWave, water and light, the signal output will be connected the ESP32 ADC pins. For each analogue sensor a first order RC low pass filter will be implemented between the sensor output and the ADC pin.

For each actuator, LED, solenoid lock and stepper moder, low-side switching topology will be implemented using N-channel MOSFETS. The actuator will be connected to the ESP32 GPIO pins for implementation of actuator output channels. The GPIO pins will be configured in push-pull mode. THE GPIO will be connected to the MOSFET using gate resistors to limit the inrush current and damp gate ringing. Pull down resistors will then be connected to ensure there is a defined OFF state during the ESP32 boot up. For inductive actuators Schottky diodes will be implemented in a flyback configuration across actuator terminals, clamping reverse EMF by providing a low impedance path during MOSFET turn-off.

5.2.3. ESP32 Firmware Development & MQTT Integration

The firmware requires several libraries to create the firmware for each sensor and actuator node. The following libraries will be downloaded onto the Arduino IDE allowing for specific functionalities to be implemented. `Arduino.h` for core ESP32 Arduino framework [68], `WiFi.h` for ESP32 Wifi stack [68], `WiFiClientSecure.h` enabling TLS encryption [68], `PubSubClient.h` creating a lightweight MQTT client for publishing and subscribing [68], `esp_sleep.h` for control of deepsleep [68] and `ArduinoJson.h` [68] to build lightweight JSON payloads for sensor states and actuator commands.

The firmware will employ single threaded cooperative multitasking loops instead of blocking delay functions. Execution loop swill be defined for sensor acquisition, actuator control, MQTT communication and system diagnostics. Each task will be assigned fixed execution periods stored as timer thresholds. The `millis()` timers will be configured to trigger sampling based on rate of human motion and hardware capabilities while preserving network throughputs. A network heartbeat will be implemented at fixed intervals to verify broker connectivity and publish keep-alive packets. Drift correction will be implemented by comparing expected vs actual timer expirations. To configure the MQTT communication, the ESP32 network stack will be initialised connecting to Wi-Fi. An MQTT client will be instantiated on port 8883. The topic structure,

house/<room>/<sensor or house/<room>/<actuator>, will be defined in accordance with the system interface. WiFiClientSecure library will be implemented to enforce TLS encryption of all MQTT payloads. The brokers X.509 certificate will be loaded onto the ESP32 file system.

In parallel with the ESP32 firmware development the Mosquitto Broker will be installed and configured on the Raspberry Pi using the following configuration steps. The listener will be enabled on port 1883 which will be converted into port 8883 later, to activate TLS parameters. This will be executed using nano-based configuration editing on the Pi. The configuration process can be seen in Figure 10.

Figure 10 - Mosquitto Broker Configuration, Raspberry Pi

Integration testing will be conducted by uploading the firmware to the ESP32, opening the Arduino IDE serial monitoring, confirming the sensor outputs and the MQTT messages are transmitted and received through the Raspberry Pi subscriber.

5.2.4. Full Scale Hardware Deployment & Optimisation

At this stage the circuits will transition from USB power to NiMH battery packs for true modular operation. For this transition, step-up voltage regulators will be introduced to boost the variable battery voltage to a stable 5V to prevent brown outs when there is a high current draw when transmitting via Wi-Fi. Energy optimisation will occur with the implementation of deep sleep modes so the ESP32 can power down its CPU and Wi-Fi and will begin using the Ultra Low Power co-processor. The module will then wait for the next sensor triggers, thus extending each modules battery life.

Physical enclosures, incorporating passive ventilation grids to dissipate heat, will be created through custom CAD designs using Creo to 3D print housings for the ESP32, regulator and battery. Furthermore, non-invasive mounting, through command strips, will be utilised to adhere to the projects requirement of remaining minimally invasive.

The actuator hardware setup will be updated to the new parameter requirements of the full-scale deployment and will follow the same setup steps are previously states in Iteration I. For loads exceeding the thermal capacity of Iteration I, MOSFETs with optocoupler drives will drive 5V electromagnetic relays. Safety barriers will be introduced at this point in the form of control signals being transmitted via light capabilities which partition the electrical connection between

low voltage logic and the high-power actuator domain, preventing high voltage faults propagating back to the ESP32 causing permanent damage.

A unified system initialisation script will be deployed on the Raspberry Pi to ensure that all local services including : network stack, Mosquitto broker, data-logging services and local databases will be launched automatically on boot. `Systemd` unit files with automatic restart polcies will be created to limit down time in case of software failure on startup.

5.3. Edge Computing & Machine Learning Pipeline Implementation

5.3.1. Local Data Collection & Synchronisation Pipeline

ESP32 nodes will publish sensor data, which will be received by the Raspberry Pi using a local Mosquitto broker. Python 3.11 subscriber scripts written in Python will subscribe to every sensor topic and store the incoming messages in local memory buffers and structured arrays and time-stamped.

To ensure proper transmission and reception of messages, MQTT Explorer and Mosquitto command-line tools will be used. At the interim testing stage, there will be no secure authentication, since testing will be conducted in a trusted network, but TLS-secured MQTT communication will be implemented prior to multi-room deployment.

5.3.2. Feature Extraction & Behavioural Modelling Logic

Asynchronous sensor measurements will be converted into synchronised time-windowed vectors to be used in machine-learning analysis by creating a preprocessing pipeline. PIR events will be transformed into binary occupancy measurements and state-transition timestamps. mmWave signals will be whittled down to invalid serial packets to produce stable presence and micro-motion estimates. The data of the pressure sensor will be sampled with a time interval of five seconds and then denoised and thresholded to determine the bed-occupancy states.

These measures will be combined into more advanced behavioural measures, such as motion frequency, continuity of presence, stability of bed-pressure and the number of transitions between behavioural states. Both model training and real-time inference will be based on these features.

5.3.3. Unsupervised Model Training and Evaluation

The subsystem of anomaly-detection will be implemented with a two-mode approach of operations. In training mode, a locally recorded user-defined calibration window of normal activity will be utilised to train an Isolation Forest model with scikit-learn. The relevant metadata, such as feature means, variances, sensor settings, timestamps, and processing parameters will be stored in the form of JSON and CSV files so that they can be reproduced.

In inference mode, sensor data will be received and converted to the behavioural feature set and sent to the Isolation Forest to calculate anomaly scores. A lightweight rule-based validation layer will be introduced to put anomalies into place through cross-checking sensor consistency. The confirmed anomalies will be recorded in the form of a JSON format, with and timestamps.

All the data will be kept on-site in order to follow privacy-first system requirements.

5.3.4. Simulation Based Behaviour Testing

Controlled simulation tests will be done before actual behavioural data is fully deployed and before ethical approval is given to the actual behavioural data. The preprocessing and ML pipeline will be fed with synthetic behavioural sequences, normal and abnormal. Such tests will be performed in the

physical scaled environment through manual behavioural generation, allowing anomaly-detection performance to be analysed in a representative and ethically conscious environment.

5.4. Cloud Backend Development & Integration

5.4.1. Backend Architecture & Deployment Workflow

The cloud backend will be implemented using FastAPI and deployed on Render. Render will be linked to a GitHub repository, allowing automated deployment when updates are pushed. During each deployment, Render will execute 'pip install -r requirements.txt' to build the environment. After configuring the required settings such as health check plans, build filters, instance types, and crucially root directory, the deployment is ready to move forward.

5.4.2. Database Schema Design and Data Routing

The database will be designed using relational schemas, using PostgreSQL's table format. It will be used for user and device credentials, certificates, sensor readings, actuator logs and routine backups. By separating the database from the Raspberry Pi it ensures that if one fails, the immediate routine can be utilised on the Raspberry Pi whilst the cloud database is repaired, without the system collapsing and vice versa. The relational schemas will be implemented using SQLAlchemy or ORM models in FastAPI.

5.4.3. API Layer Preparation & Secure Dataflow

API routes will be defined to support communication between the backend and the mobile application. End points such as GET/sensos/latest, GET/routines and POST/routines will be specified to allow for the frontend to be developed independently. Response formats will be standardised to aligned with JSON files produced by the edge ML subsystems and expected by the mobile application. Secure HTTPS communication will be integrated wilt the Raspberry Pi to enable secure communication with the cloud based side of the system.

5.4.4. Cloud Backend Integration with Frontend App

For integration of the frontend with the FastAPI backend, the methodology includes defining API routes that the frontend app will rely on, returning consistent responses from any requests and testing this communication method, to allow for seamless integration. The cloud backend is not currently at a fully functioning level and hence has not yet been tested with the frontend.

5.5. Mobile Application Development

5.5.1. Environment Setup, Clean Architecture & Prototyping

Flutter will be used to develop a mobile application due its cross-platform functionality and rapid prototyping, such as hot reload. The project will be structured in terms of clean-architecture dividing the app into three layers:

(1) core for configuration and routing.

(2) features for UI components such as sensors, routines and settings.

(3) data for repositories and data-handling logic.

This upfront architectural design will minimise refactoring needs in future development.

5.5.2. Mock-Driven UI Development & Repository Pattern

The backend API is currently not live, and thus, a mock-driven approach will be adopted. Mock providers will be based on Riverpod and will simulate user routines and sensor summaries. This will

enable the UI components, such as the sensors, routines and settings tabs to be designed, tested and refined to their full extent prior to their integration with the backend.

Repositories will be based on the repository-pattern design that will allow mock repositories to be substituted with remote repositories in the future without altering the UI components.

5.5.3. Early Interface Testing & Offline-First Preparation

An initial API layer will be developed on top of a Dio based HTTPS client and an application configuration file with server and routing constants. GET and POST endpoint placeholders will be developed to validate the future format of communication with the backend.

The offline-first functionality will be enabled via implementation of a Drift/SQLite local database, which will ensure that recent summaries of sensor data and regular data are still available even in cases when the cloud service is inaccessible.

5.5.4. Interface Testing & Cross-Device Validation

The Android emulator and Flutter desktop builds will be used to perform early testing of the UI. Correct component rendering, responsiveness of UI updates and state restoration form validation and sensor-card consistency will be tested. This testing phase will make sure that as soon as live backend data is accessible, minimal UI redesign will be required.

5.6. System Integration Workflow

5.6.1. Local MQTT & Edge ML Pipeline Integration

The initial step in integration will be to ensure that validated sensor hardware and ESP32 firmware are properly publishing sensor states to the Raspberry Pi Mosquitto broker. The Python data-collection scripts will then be tested to make sure that data streams are ingested in a synchronised way. The next step will be the ML preprocessing pipeline, being the last step the real-time anomaly scoring with the Isolation Forest model.

5.6.2. Edge ML & Cloud Backend Integration

After enabling cloud communication, the Raspberry Pi will send summarised behavioural logs over the FastAPI backend via secure HTTPS. The data will be verified and stored on the backend in PostgreSQL and made available to the mobile application via the specified API paths. The repository layer of the mobile application will then be revised to substitute mock providers with real API data sources.

5.6.3. Full Pipeline Trial in Scaled Environment

Lastly, the entire sensing-to-visualisation pipeline will be tested in the scaled apartment environment. The sensor data will be gathered and processed at the edge, rating of the anomalies will be provided, summaries will be sent to the cloud backend, and the visualisation of the processed data will be performed with the help of the mobile app. This will be a controlled test that will confirm the functionality of sensing, firmware, networking, ML, cloud infrastructure, and the mobile interface before actual deployment.

6. Results and Calculations

The findings in this section sum up the achievements at the interim phase of the project, which are on subsystem testing, prototype testing and initial integration activities. Despite the use of real behavioural data is currently not allowed by ethical considerations, controlled-environment tests with the scaled house model, simulated sequences of activity and engineering-only activations have been successfully used to test the sensing pipeline, MQTT communication, data-processing scripts and initial model-training workflow. Hardware prototypes, electrical characterisation, firmware performance and stability of local communication are tested in addition to successful ingestion of multi-sensor data and the generation of structured datasets. These results indicate that the basic elements of the system are functioning as they should and are prepared to carry out full multi-room implementation when the final ethical approval is received.

6.1. Network Infrastructure

6.1.1 Broker Configuration

The initial stage of implementation involved configuring the local Mosquitto broker on the Raspberry Pi, the central local controller. The broker receives messages from the sensors (publishers) containing the sensor data and sends them to the subscribers in the local network. The figures below show evidence of the subscriber window receiving test messages, validating the broker is successfully installed on the Raspberry Pi. These results confirmed that the broker was operational and listening for incoming MQTT client connections.

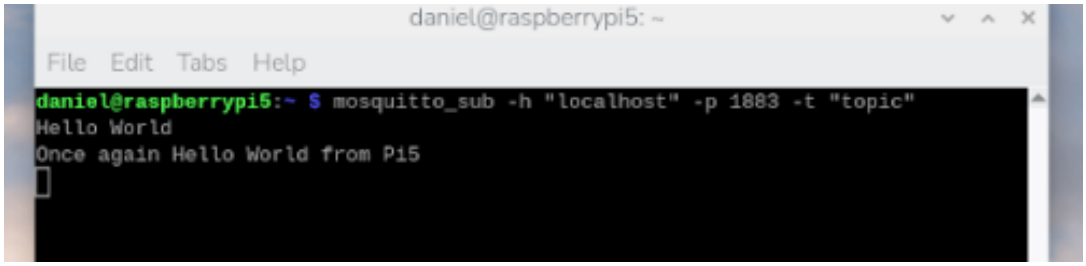


Figure 11 - Initial Confirmation of Mosquitto Broker Installation

Figure 12 - Successful Service Start and Broker Activation, Raspberry Pi

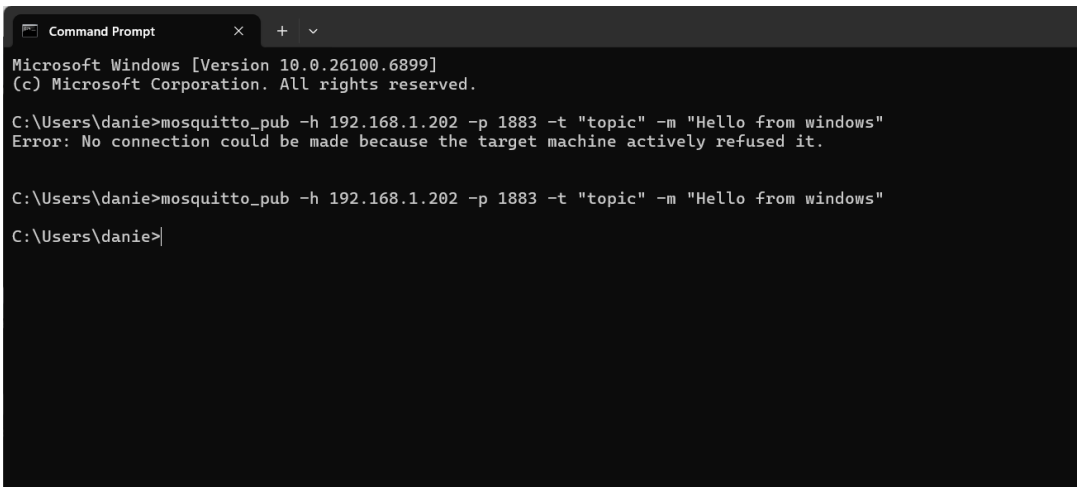


Figure 13 - Successful Service Start and Broker Activation, Windows

Once the broker was active, the next goal was to establish communication with sensor devices, ESP32 modules configured to publish data over the local network (Wi-Fi router). The sensors published messages to a specific topic shown below and, on the Pi, messages could be validated using Mosquitto client tools to act as a subscriber in the local MQTT network.
On the Raspberry Pi:

```
mosquitto_sub -h localhost -t "sensors/#"
```

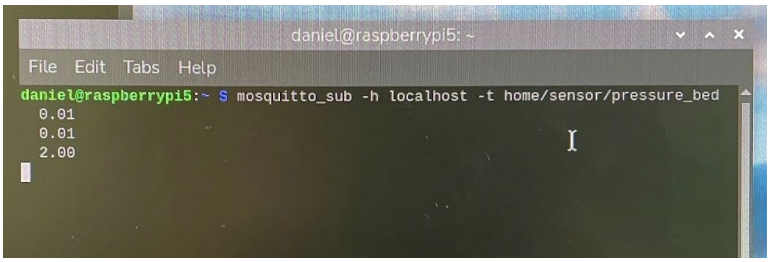


Figure 14 - Pressure Sensor Published Data, Raspberry Pi

The ESP32 successfully connected to the Raspberry Pi's MQTT broker and published sensor values at a regular interval. The Pi received and logged these messages, demonstrating a fully operational local communication loop.

6.1.2 FastAPI Backend and Deployment

The notable results regarding the cloud system are the deployment of the cloud system on render and the creation of the FastAPI Backend. Although in the early stages of development, the foundations have been laid in establishing parameters and the structure of the FastAPI Backend. The deployment of the Assisted living Platform went live on Render, and images of the deployment process can be found in the Appendix D.

6.2. Sensor Subsystem

6.2.1. Electrical Characterisation

Prior to integrating sensors with the ESP32, each sensors response and voltage levels were characterised using an oscilloscope. This phase ensured logic level compliance (3.3V and 0 V) and informed the software design, in terms of calibration and start up delays.

The PIR sensor outputs a sharp transition between logic level low at 0 V and logic level high with 3.3 V with negligible ringing on the rising and falling edges (Figure 15). The pulse durations correlated accurately to the motion events and the sensor returned to a stable logic low during subsequent periods (Figure 15). This signal shows high noise immunity confirming the sensor is suitable for edge triggered GPIO without the need for high levels of hardware debouncing

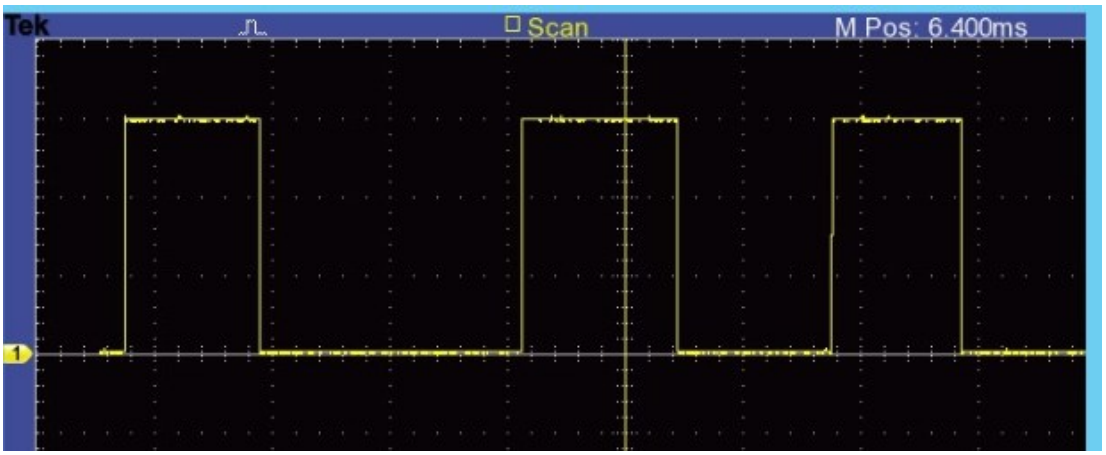


Figure 15 - PIR Oscilloscope Output Waveform

Using the digital logic output signal pin of the mmWave for testing, maintained a stable logic high state during presence detection and was not susceptible to jitter spikes or voltage sag (Figure 16). The micro-movements of hand gestures used for testing produced distinct, observable outputs, verifying the mmWave radar sensitivity (Figure 16). The onboard filtering from the mmWave sensor is sufficient to avoid aliasing and false negatives supporting its use for presence sensing in the model home.

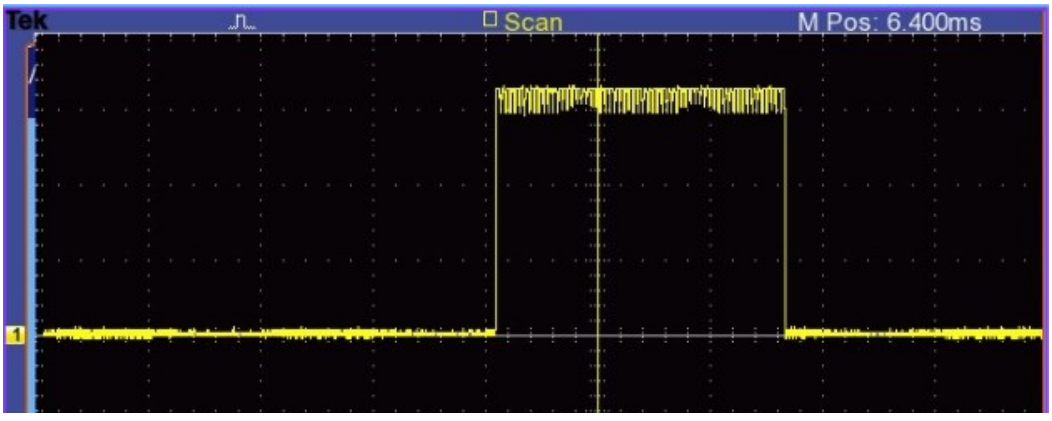


Figure 16 - mmWave Oscilloscope Output Waveform

The pressure sensor yielded a continuous analogue voltage proportional to the applied form (Figure 17). A clear signal to noise ratio was observed between the no load baselines and high load state (Figure 17). Within this signal there is minor high frequency noise which was observed between the no-load baseline and high load state (Figure 17). The dynamic range is sufficient for

the ESP32's 12-bit ADC, the observed noise will require the implementation of a moving average filter in the firmware to stabilise the readings.

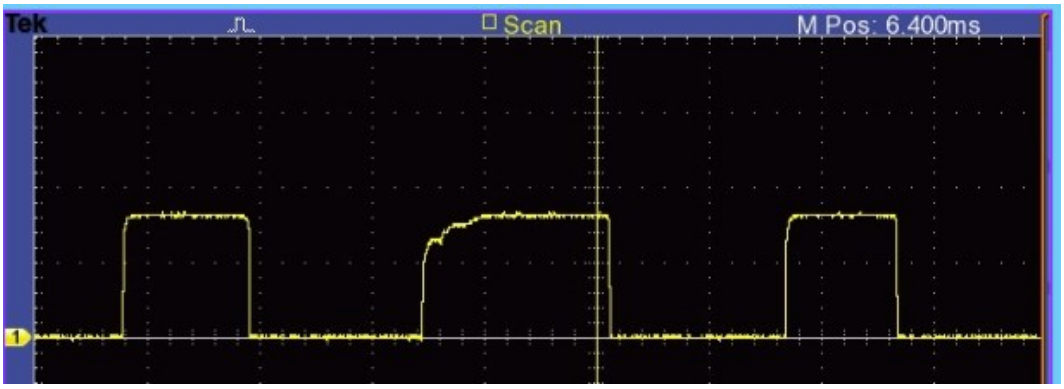


Figure 17 - Pressure Sensor Oscilloscope Output Waveform

When the water sensor is dry the water sensor acts as an open circuit outputting a consistent 3.3 V (Figure 18) . The signal drops when water is applied. The trace demonstrates a clear, noise free distinction between dry and wet, confirming the sensor provides a digital ready signal suitable for GPIO interrupt handling.

Figure 18 - Water Sensor Oscilloscope Output Waveform

6.2.2. Firmware Integration

Following the sensor hardware testing outlined above, custom firmware scripts were developed and tested for the PIR, mmWave and pressure sensor (Appendix B). Initially the scripts were uploaded to the ESP32 and tested the functionality of each sensor was aligned with the electrical characterisation as seen in Figure 19.

```
>>> MOTION ENDED (State: LOW)      Detected Distance: 41 cm   Current Sensor Value: 0
>>> MOTION DETECTED (State: HIGH)  Detected Distance: 34 cm   Current Sensor Value: 0
>>> MOTION ENDED (State: LOW)      Detected Distance: 28 cm   Current Sensor Value: 0
>>> MOTION DETECTED (State: HIGH)  Detected Distance: 21 cm   Current Sensor Value: 3775
>>> MOTION ENDED (State: LOW)      Detected Distance: 14 cm   Current Sensor Value: 4095
>>> MOTION DETECTED (State: HIGH)  Detected Distance: 9 cm    Current Sensor Value: 4095
>>> MOTION ENDED (State: LOW)      Detected Distance: 9 cm    Current Sensor Value: 4095
>>> MOTION DETECTED (State: HIGH)  Detected Distance: 7 cm    Current Sensor Value: 4095
                                   Detected Distance: 7 cm
```

Figure 19 - Arduino IDE Serial Monitor Output For PIR, mmWave and Pressure

Following this testing the script evolved implementing cooperative multi-tasking architecture, non-blocking timers and secure MQTTS transmission pipelines. Real-time data transmission was confirmed by subscribing to the corresponding MQTT topics on the Mosquitto broker running on the Raspberry Pi .

The PIR script (Appendix B) correctly triggered the state classification with no missed interrupts during the repeated activation tests. PIR topic is set as `home/bedroom/PIR` (Figure 20). The system achieved deterministic state mapping with GPIO high creating an `'IN'` output, this state is then held and when motion is detected again GPIO high create an `'OUT'` output (Figure 20). No false triggers occurred during operational testing. State transitions were published with a latency of 50ms and validated the efficiency of the interrupt driven firmware architecture (Figure 20) .

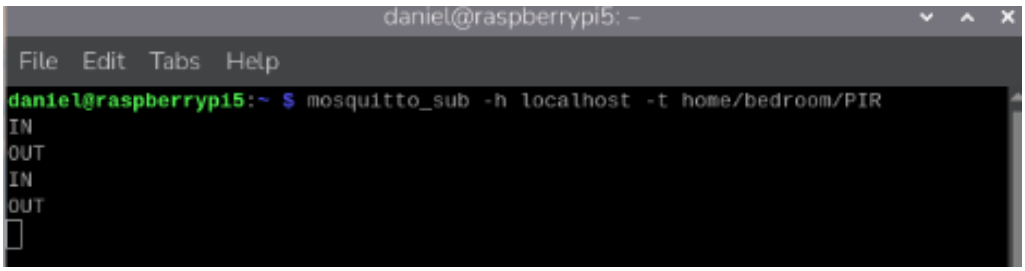


Figure 20 - PIR MQTT Test

The mmWave script (Appendix B) operated at high frequency using `millis()` timers and successfully output a continuous floating-point stream. mmWave topic is set as `home/bedroom/presence` (Figure 21). The sensor streams continuous floating-point values represented the presence of a person within the room. The system exhibited high temporal resolution with values updating rapidly when physical movement was detected. No packet loss was observed during the updates.

Figure 21 - mmWave MQTT Test

The pressure script operates based on event driven logic (Appendix B). Pressure topic is set as `home/bedroom/pressure` (Figure 22) . The ADC performance is set with a baseline of `0.00` and a max load of `2.00 BAR`. The 12bit ADC correctly quantised the analogue voltage (Figure 22) with varying load levels. The firmware averaging successfully smoothed previously identified analogue noise, resulting in clean transitions between states.

Figure 22 - Pressure Sensor MQTT Test

The end-to-end communication pipeline was stress-tested to ensure real-time capabilities. The Raspberry Pi broker successfully handled three simultaneous sensor topics (Figure 23) without message queuing delays or socket disconnections. Visual correlation tests indicated a latency of 50ms between physical events and terminal outputs, this value falls within the acceptable latency value for a home monitoring application. When combined with the .JSON file the data remained valid throughout the test duration with no corrupt packets.

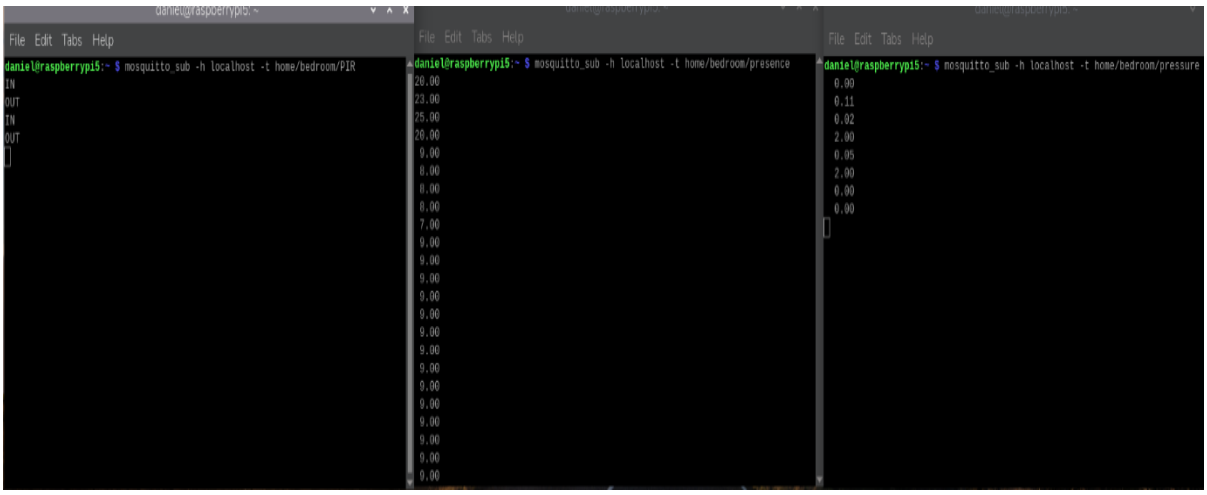


Figure 23 - 3 Sensor MQTT Test

6.3. Data Acquisition & ML Pipeline

Initial integration tests were conducted to ensure that all three-bedroom sensors, PIR motion sensor, mmWave distance sensor, and analogue pressure sensor could publish data to the Raspberry Pi Mosquitto broker in real time (figure 23). These tests were done with manual activation (hand movement and manual force application) with the sole aim of testing hardware and communication functionality. No behavioural or personal information was gathered. The terminal output of the MQTT collector script reported successful reception of messages, which all MQTT topics generated the expected, timely readings.

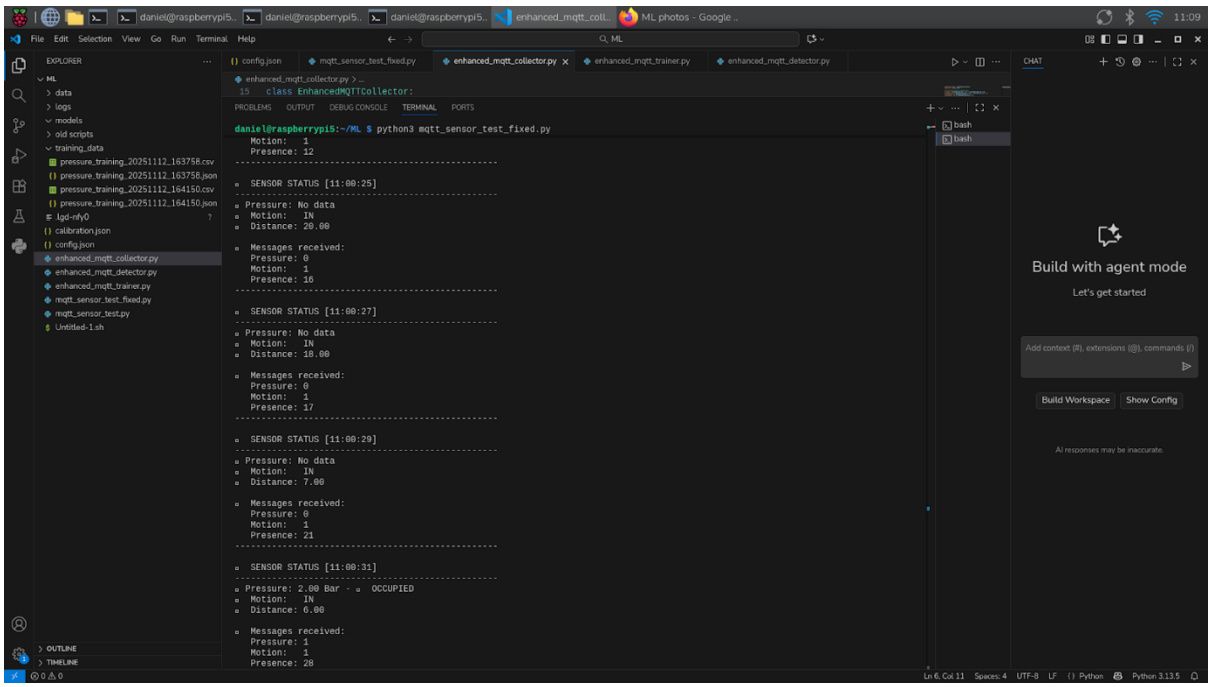


Figure 24 - Terminal output from the MQTT collector showing successful real-time reception of PIR, mmWave, and pressure sensor data during engineering validation

Message counters were used to verify sequential reception of sensor packets during these tests, and all three modalities were found to be running concurrently. The PIR sensor showed proper rising-edge detection and stable state-changes ("IN", "OUT"), and the mmWave sensor gave continuous centimetre-scale distance measurements. The pressure sensor provided voltage-based estimates of bed contacts. There were some readings at the beginning, mostly in the mmWave module, that showed out of range or some type of placeholder (999 cm) which is anticipated in initial testing with a loose and handheld system. These results indicate that better physical mounting and calibration is required to carry out structured behavioural experiments in the future.

The training procedure was validated using the dedicated training script, which collects sensor readings, extracts windowed features, and stores the resulting dataset in both JSON and CSV formats. Although full behavioural training cannot be performed until ethical approval is granted, the script was tested using manually induced sensor activations to ensure correct file generation (see the set up in figure 24).

The system successfully produced structured training files (mqtt_training_20251204_112825.json and mqtt_training_20251204_112825.csv), containing feature windows and sensor states.

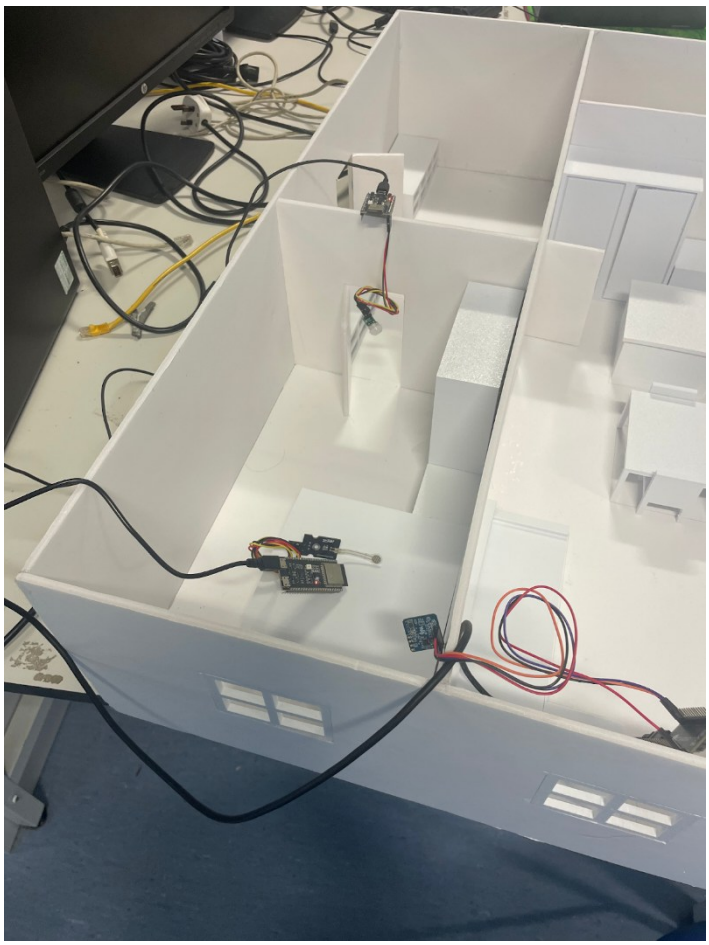


Figure 25 - Manual Induced Sensor Activation Test Set Up

Manual activation is adequate at this stage because the objective was only to confirm electrical connectivity, MQTT publication, and synchronous collection across the three modalities. The script and its generated example files can be accessed through the links located in the Appendix A.

A fragment of the generated JSON output is shown below for demonstration, with all values derived exclusively from engineering tests rather than human participants:


```
{
  "window_id": 0,
  "timestamp": "2025-12-04T11:28:25",
  "features": {
    "motion_count": 1,
    "presence_events": 12,
    "avg_distance_cm": 20.0,
    "pressure_bar": 0.00
  }
}
```


This JSON structure is the basis for future training, where each window will form one instance of the behavioural baseline once ethics approval allows that stage.

This confirms that the training pipeline correctly extracts and serialises multi-sensor feature vectors. These structural results demonstrate that the subsystem is ready for ethical-approved behavioural data once permission is granted.

The training script was run to confirm the data-collection and feature-extraction procedure by hand-activation of the sensors. In this test, the script was able to record multi-sensor data across a specified sampling period and create formatted output files in CSV and JSON formats. These files store initial feature windows, timestamps and metadata which are only based on engineering validation, not behavioural monitoring. These files have not been further processed: they have not been loaded into the machine-learning subsystem, or even parsing the files to determine their consistency, or to initialise any model. This is not by chance, because the team will have to acquire ethical permission to use the data acquired in any manner of behavioural modelling, or anomaly detection. The subsequent versions of the training script will include improvements that will enhance the accuracy of the features and minimise noise after all sensors have been physically mounted and calibrated.

Operation of machine learning, such as training of the Isolation Forest model, loading of feature data sets, running of anomaly scoring and testing of synthetic scenarios have not been performed yet. These are steps that are awaiting until the sensor data collected is used in behavioural analysis by ethical approval. At the present phase, the machine-learning subsystem merely has an architectural representation in the software design, which will be triggered once ethical approval is received and training script refinements implemented. In case of any delay in ethical approval, the model development can be continued by using publicly available datasets like CASAS to test preprocessing, feature extraction and model-loading behaviours without using human-generated data. This will guarantee further technical advancement without violating ethical limits. Synthetic scenario generation can also be added in future to test the behaviour of the algorithm without the participation of real people.

There were various sources of measurement variability that were found during sensor-integration tests. Obtained outcomes represent a demonstration of incorporation and not ultimate behavioural functioning. It will take proper physical installation, mechanical stabilisation, and calibration of the system before it can produce high-quality behavioural data that can be used to train. After the ethics approval, fine training processes will be applied to define correct behavioural standards. In case of the delay in approval, it is possible to use fallback datasets like CASAS to enable the controlled assessment of the ML pipeline without violating ethical principles.

6.4. User Application Interface

A three-tab interface has already been applied, which has been completed and it is composed of Sensors, Routines, and Settings screens. It is built on the declarative architecture of Flutter, the interface is built out of widgets (Appendix C), and it can be quickly iterated and layout refined - one of the acknowledged strengths of the Flutter design philosophy [46]. The mock data are already shown in each tab, but the tabs are structurally complete. Sensors Tab (Appendix C): Displays summaries about sensors such as room name, sensor type, last update time, and iconography. Routines Tab (Appendix C): Shows the current mock routines and provides the possibility of creating new ones with the help of a structured dialogue form. Settings Tab (Appendix C): Displays metadata of builds, config options of placeholders and user identifiers.

These findings indicate complete functionality of UI even without the availability of the backend. Features that have been implemented although the FastAPI backend is not deployed yet are, a full API client framework with the Dio HTTP package. A template request programmes to access sensor summary and routines. A scaffolding and response models of JSON parsing. And API settings linked to environment variables (e.g., base URL, house ID)

This implies that after the backend team has done deployment, the app can be moved to real data by just replacing the MockRepository classes with RemoteRepository implementations, one of the main advantages of the repository design pattern that have been frequently cited in mobile system architecture today [65].

Mocked data has enabled the entire user flows to be tested: all screens can be navigated by the users, users are able to create and see routines, Riverpod state management is used to dynamically update the UI, sensor cards display correctly with good icon mapping and formatting, timestamps and labels are shown appropriately in any view. A known method of decoupling frontend advancement and backend advancement is mock-driven development, which enables effective development concurrently.

A different database instance that is to be used by: credential storage, routine representations, late sensor summary retrieval of the project. This choice guarantees more modularity, and it lowers risk of security by minimising direct communication between the application and the hardware or machine-learning components in line with the advice of IoT security [47].

The test has been done using Android Emulator and Flutter desktop build: Navigation stability, routine creation flows, UI consistency across display sizes, icon spacing, stroke alignment and padding, conversion of internal data. The entire user interactions seem to work as expected, that is, a stable and production-ready UI base.

6.5. Summary

Overall, the interim results show that the technical layers of the system are technically correct and consistent with the goals which were set at the beginning of the project. The sensors work well together, the ESP32 code and MQTT broker have been shown to be reliable when communicating and the Raspberry Pi has been able to process concurrent data streams of multiple nodes. The machine-learning pipeline has been tested with simulated behavioural data, and the cloud/backend infrastructure and mobile app have been brought to a structurally complete point using mock inputs. These findings suggest that the project is on track for Semester 2, where full-scale hardware deployment, ethical-data collection, multi-room model training and system-level evaluation will enable the operation of the assisted-living platform.

7. Discussion

During the discussion section an assessment of the significance and reliability of our results from section 6 will be given including consideration of the implications of experimental errors. In addition, alternative approaches are proposed along with further experiments to be carried out. Each section of the system is described in detail, discussing how the system performs against original specifications laid out at the beginning of the project.

7.1. Sensor Subsystem and Physical Validation

The data gathered from the sensor subsystem corroborated the underlying physical principles of the sensors selected. This in turn validated the previously outlined selection strategy. The PIR sensor outputted clean digital transitions with negligible switch bounce. This confirms the ability of the Fresnel lens in focusing infrared radiation onto the pyroelectric substrate, thus displaying PIR's ability to act as a binary trigger for behavioural monitoring. The mmWave module provided high update rates without aliasing. The correlation between micro-movements and the outputted distance aligns with the Doppler shift theory, confirming the sensors capability to detect presence in combination with the PIR. The pressure sensors exhibited non-linear decay in resistance under load. The separation between load and no-load states are distinct.

The successful integration of three distinct sensor modules validates the previously outlined architecture approach. The firmware achieved sampling rates tailored to each sensors bandwidth requirements: high frequency for mmWave and interrupt driven logic for PIR and pressure. This confirms the ESP32 architecture can handle the computational requirements for future features. The ESP32's 12-bit ADC provided sufficient resolution to distinguish sedentary occupancy and active movement. The consistent data suggests the external hardware for the pressure sensor is not required, reducing the system complexity and overall cost. The stability of the current loop based on acquisition confirms the feasibility of the use of an event-driven finite state machine. The system will be capable of transitioning between states based on asynchronous sensor interrupts rather than requiring constant polling.

The most significant finding is in the confirmation of the perpendicularity between sensor types. The data has demonstrated that if a sensor fails the system can still monitor behaviours of the user. This finding supports a robust fusion strategy. The PIR is excellent in detecting exit and entry but cannot provide information on static occupancy. The mmWave can continue outputting data on presence during static periods but lacks the FOV to define strict room boundaries. The pressure provides a truth for the occupancy of the room that is immune to interference. These findings indicate that the fusion algorithm should prioritise PIR for transitions and mmWave or pressure for state retention, this in turn will reduce the number of false positives significantly compared to other single sensor commercial solutions such as Taking Care Sense [6].

7.2. Network Architecture and Scalability

The obtained results discussed in the results section of the report demonstrate the fundamentals of the system have been successfully implemented. For the local network, the secure MQTT proved to be stable and responsive where the sensors publish data to the broker on the Raspberry Pi, forwarding the messages to subscribers. Overall, this confirms the decision for the publish/ subscribe model over the request- response model. Furthermore, the successful connection of the ESP32 sensors to the Raspberry Pi over the local network validates the network can operate without an internet connection, which aligns with the original project specification.

One of the strongest points of the work that has been done to date is the fact that the mobile application is architecturally consistent with the end-to-end data flow of the system. By applying the repository pattern, state management through Riverpod, and a proper separation of the UI and data sources, the application is ready to integrate real-time data the moment that the backend endpoints are active.

The use of Flutter as a development framework has had a positive influence on scalability and maintainability. The single-codebase architecture of Flutter minimises the long-term development cost, and guarantees the same UI behaviour on Android and iOS platforms [45].

7.3. Data Pipeline

The obtained results of this stage during of the data pipeline development indicate that the subsystem of sensor collection and data structuring is functional, however currently there are some limitations affecting the current conclusions to early engineering validation instead of behavioural performance. The fact that PIR, mmWave, and pressure sensor data are successfully received is an indication that the multi-sensor architecture, MQTT communication pipeline and local Raspberry Pi integration are working as planned. This is important since, upon which the subsequent anomaly-detection and personalised routine-learn elements will rely, is a reliable real-time acquisition. The structural reliability of the message counters and the appropriate mapping of sensor topics are evidence of a high level of structural reliability, which implies that the software architecture is robust enough to sustain full-house deployment once the behavioural modelling phase is commenced.

Nonetheless, the accuracy of the sensor data is limited by the initial physical arrangement of the experiment. The out-of-range values (e.g., 999 cm) of the mmWave sensor, the quantisation noise of the pressure sensor and the debounce artefacts of the PIR sensor demonstrate that the readings of the current sensors cannot be yet taken as the true reflections of human activity. These problems do not lie in system design, but in the lack of an installation which is stable, calibrated and mechanically optimised. In one case, the mmWave module was held by hand, and this imposes angular instability and distortions of the beam-pattern. Equally, low-force measurements are restricted by the 12-bit ADC accuracy of the pressure sensor, particularly when the sensor has not been integrated into a platform of a bed where it is to be used in practice. In that way, the experimental errors that are witnessed are anticipated and controllable and they do not weaken the accuracy of the pipeline of integration.

7.4. Privacy-by-Design and Security

Privacy-by-Design has been implemented throughout the entire system. Processing the raw data on the ESP32 prior to transmission, the bandwidth of the system is minimised, and the user's privacy is preserved, this is a key requirement for meeting GDPR [12] compliance in IoT based healthcare.

Utilisation of the localised, modular, and privacy-preserving sensor-to-feature pipeline can be run reliably in real time. This also comprises the architecture of concurrent multi-sensor acquisition, creation of structured training files, and the incorporation of scripts that will later be used to assist the detection of anomalies on-device. It is essential to have this groundwork: any behavioural model, no matter its level of sophistication, would not work in practice without a reliable collection and preprocessing layer. Therefore, the value of the present findings does not rest in the behavioural understanding, which is still to be ethically approved, but in the successful development of the scalable

and ethically responsible sensing framework that will prepare the system to the subsequent phases of the machine learning, personalisation, and monitoring of the entire house.

The choice of the HTTPS communication as the main medium between the mobile application and the cloud backend can increases the security of the system and ease the process. Studies on the vulnerability of mobile applications point to the fact that unsecured or unencrypted communication exposes a person to high risks of data interception and manipulation [47]. The app uses only HTTPS, so it complies with best practices of transmitting potentially sensitive data, regular updates or generated summaries of behaviour. Moreover, the separation of the credential database and cloud data layer will mean that the mobile application will not directly communicate with hardware equipment. This modular security structure aligns with the guidelines of IoT security that favour compartmentalisation of systems to limit system-wide exposure to a breach.

7.5. Critical Comparison with Existing Solutions

This approach compared to current commercial offerings including: Alexa Together [7], StackCare [4] and SECOM [5], use proprietary algorithms and cloud-based analytics has two points of difference in favour of this project even at this infancy. First, the design focuses on privacy-saving local computation, where raw behavioural data does not leave the home without express permission. This is opposed to cloud-dominant systems which normally demand ongoing data transfer to remote servers. Second, the suggested application of unsupervised machine learning allows modelling behaviour on a patient basis, in contrast to most commercial systems, which are still based on rule-based notifications (e.g., no movement in X hours), which are not context-sensitive and may frequently trigger false alarms. Having said this, commercial platforms offer extensive tools, built in analytics and stronger device management, compared to our system. The architecture of the system is lighter and more adaptable but is closer to a prototype platform rather than a polished, market ready solution. Despite the ethics pause that does not yet allow quantifying these advantages, the architectural decisions can be seen as a considered break in the status quo and place the system in a better position to be more adaptable and ethically compliant.

7.6. Limitations

As discussed in Section 7.3, the accuracy of the sensor data is currently constrained by the temporary and non-optimised physical setup used during preliminary testing. This means that the present readings cannot yet be interpreted as reliable behavioural indicators, and proper mounting and calibration will be required before performance can be meaningfully assessed.

Although the system is function there are some constraints which have been identified during testing. Testing was conducted in a controlled environment within the small-scale prototype. This is not an accurate representation of the multipath propagation and RF reflections of a real-world room; this could result in reduced performance of the sensors. The current firmware is in continuous active mode, thus in the next stage deep sleep and ULP coprocessors will be utilised to improve the battery life of each module. The absence of the actuator system at the current time means the control aspect of the project remains purely theoretical. The latency metrics at this current time are promising however when adding the actuators this time may increase.

When evaluating the importance of the findings, one must not overlook the fact that the main aim of the project, which is to come up with an adaptive, personalised anomaly-detection system to help people live, cannot be assessed in any meaningful manner before having a reliable measurement substrate. The presence of the training script that was able to produce structured CSV and JSON files

indicates that the data-preparation parts are prepared to be processed in behavioural modelling, and the lack of ethical approval does not allow the subsequent processing at this point. This shortcoming limits the range of quantifiable performance but also strengthens the moral uprightness of the project: no behavioural patterns were gathered, analysed and modelled before. The decision of postponing ML execution is thus not a flaw of the methodology, but a necessity to conduct responsible research.

7.7 Future Work

The next steps for the sensor and actuator system will see Interrupt driven awake routines being implemented utilising the ESP32's sleep mode aiming to reduce the average current consumption when sensors are idle. A 'watchdog' routine will be implemented to handle any stuck values or disconnects. Actuator circuits will be created and tested within the model home. Following the actuator testing the sensor and actuator system will be expanded to the rest of the small-scale environment. Following this the project can begin work on Iteration II in preparation for full scale deployment. Furthermore, at the current stage testing has only been completed with 3 sensors however the full system will implement 14 sensors. Thus, future work will need to be carried out to test the network under sustained message rates and increased number of devices. Despite this, it can be observed the model can be scaled and used long term in the system and there is no hesitation that the system cannot meet the demands during peak operation.

The machine learning aspect of the project will focus on three main areas. (1) physical refinement and calibration of the sensors to reduce the error of the experiments and provide stability of measurements; (2) the implementation of the ethics approval procedure to allow collecting real behavioural data; and (3) testing of the ML pipeline on real or, in case of necessity, publicly available datasets such as CASAS. The following steps will enable the training, validation, and quantitative comparison of the anomaly-detection model with the performance of the current smart-home monitoring systems. After such assessments are done, the project can make a sensible estimate of detection accuracy, false-alarm rates, suitability to personal routines, and the general effects of localised machine learning on user privacy and independence.

The cloud layer is in the initial stages as stated in the project plan and more work needs to be completed to continue the development of the backend. The testing to be done will verify communication with the frontend and the storing of data in the PostgreSQL database. Further experiments will be carried out, to gain a deeper understanding of the limitations of the network. Some examples are message latency measurements and failure modes such as broker downtime or any Wi-Fi interference.

The next stage live API integration, The API layer in the backend is fully ready in the flutter code, but it cannot start communicating until the FastAPI service is started. As soon as it is possible, it will be required to substitute mock repositories with remote ones and use real HTTPS responses. Local database caching will be implemented as mentioned in the design, but Drift/SQLite integration remains to be done. This will make offline access to historical summaries and routines possible - a need of assisted-living reliability [51]. Push notifications will be implemented allowing the assisted-living system should be fully operational and should also notify the caregivers when the app is closed. Push handling and deep-link navigation will be used once the backend has been configured to offer a notification relay service. Live sensor summary alignment will occur once the FastAPI has been deployed and connected to the Raspberry Pi, the app created will require mapping sensor kinds, types, and summarised ML outputs to the interface on a regular basis.

8. Project Plan

All the subsystem activities, such as sensing and actuation, cloud backend, edge processing, machine learning (ML), and mobile application development, are consolidated into a single timeline, spanning both semesters, in the project plan (Appendix E). The plan takes into consideration the milestones already accomplished, the dependency between the hardware, software, and AI elements, and the tasks that still need to be done to achieve the full integration of the system and the final demonstration. This plan, as structured, is based on the needs of the University of describing the progression of tasks, dependencies, and schedule modifications.

In Semester 1, the team made the first milestones needed to create the technological basis of the system. The project requirements were documented, and sensor and actuator inventory finalisation was performed on, taking into consideration the practical constraints, as well as the requirements of the downstream ML subsystem. Preliminary compatibility testing of sensors, and the ESP32 boards was done, which allowed the hardware and firmware team to continue with network setup. Simultaneously, architectural design was created at the local and cloud levels, which ensured that the entire system was capable of supporting the idea of communication between sensors, the Raspberry Pi, the cloud backend, and the mobile app, which was designed privately.

The integration with the edge system was based on the completion of prototype calibration, MQTT broker setup, and the local dataflow. In mid-November, the groundwork ML model was created, which allowed establishing the first accuracy levels and confirm that the choice of the sensors made earlier in the semester was correct when it comes to behavioural inference. The milestone of circuit design was met, and other ML scripts of low-scale testing and multi-sensor testing were deployed in November and December respectively. The ML model was optimised and tested on controlled data, and was ready to be expanded to real-behaviour routines with ethical approval pending in early December.

Advances on cloud side by configuring local brokers and initial backend development. Though backend deployment was initially planned to be done at the end of Semester 1, the reliance on the router and external testing conditions changed this to early Semester 2. Nonetheless, the definition of APIs, the design of database structures, and secure structures of the HTTPS communication were developed ahead of schedule, minimising the anticipated delay. This position is not harmful to the overall system timeline, since the deployment of the cloud was to be overlapping with the introduction of ML and the development of mobile applications in early Semester 2.

Semester 1 major milestones in the frontend were achieved. The app base was deployed with a clean-architecture design, and the minimal interface + communication system, mock-based, was completed by the term-end. Even though the app itself is not yet connected to the backend, the API layer and repository patterns are ready, and they will be integrated early in Semester 2, as soon as FastAPI is deployed. This sequencing is a good engineering practise, wherein the development of the UI and mock prototyping can proceed even without the backend being ready.

The extended Semester 2 plan is a culmination of the other milestones needed to complete the system. The full system integration milestone is a time frame between the initial segment of the semester, and it involves deploying the backend and secure message flow, as well as testing the ML pipeline on the Raspberry Pi. After ethical approval has been decided, a multi-room data recording and real environmental validation would be performed, which will be used as input into model refinement and anomaly classification. Finalisation of the cloud backend implementation, the local

to cloud communication, and dataflow testing to verify the integrity of end-to-end messages. Actuators will be integrated, including workload that has been delayed by components, and will be used to support testing in the real environment.

On the mobile side, the future milestones, connection of full sensors with the app, the formation of actuator communication (exposed via backend), the introduction of the alert and sensor-history displays, and the final readiness of the app to become deployed at the end of Semester 2. These are activities that rely on the preparation of the backends but at the same time do not affect the project timelines. The modular design of the app will mean that as soon as the actual endpoints are in place, the integration will be done most effectively.

The last phases of the project are the completion of the system prototype early February and the final ML model end of February. Field testing, final evaluation, optimisation, and preparation of the demonstration system take up the final weeks of the semester. This is a stage in which reliability, accuracy and usability of the system are checked before presentation.

Altogether, despite some of the tasks being shuffled based on the availability of hardware and the time of the backends, the fact that every team member can continue to meet other milestones in the meantime has ensured the feasibility of the entire timeline. The new project plan has now fully integrated all the individual and combined milestones, their completion status, and the task dependencies. Consequently, the project is still on schedule to complete system integration, evaluation, and demonstration within the set time frame and satisfy the expectations on the guidelines of the Interim Report.

9. Conclusions

The work done so far has shown that a privacy-preserving, hybrid IoT and AI-based assisted living architecture is a technically practical and well-consistent solution to the industrial, societal, and ethical motivations described in the previous sections of the report. Passive sensing, edge computing on a Raspberry Pi 5, a secure cloud backend and a cross-platform mobile app have been demonstrated to have a coherent basis for a scalable assisted-living platform that can be used to support routine monitoring and predicting anomalies in the future while preserving a strong privacy-by-design stance.

The sensor subsystem and local MQTT infrastructure have been proven to be successful from a hardware and networking perspective. Electrical characterisation confirmed that passive sensors (PIR, mmWave and pressure, among others) produce clean, interpretable signals suitable for digital integration, and custom ESP32 code has been demonstrated to publish reliable telemetry with low observed latency (50 ms) via the Mosquitto broker on the Raspberry Pi. The local network has been functioning successfully using three parallel bedroom sensors confirming the suitability of the publish subscribe MQTT model, secure TLS-enabled communication and the selected topic hierarchy to a multi-room implementation. Problems related to mmWave out-of-range values, pressure-sensor noise have been linked to mounting and calibration issues instead of architectural issues, showing that they can be fixed by mechanical refinement and firmware adjustments later.

The data collection and preprocessing pipeline has been developed and tested on the edge-processing side. Heterogeneous sensor streams can now be used to construct synchronous feature windows, allowing the training model script to be tested with engineering-only activations on the scaled bedroom environment, producing structured CSV and JSON datasets to be trained into a model. Even though the Isolation Forest-based anomaly-detection model is not yet trained or deployed because ethical approval is still pending and the present limitation of using behavioural data the software architecture, file formats and operation mode (training vs inference) already exist. This validates the fact that, upon the availability of real multi-room data, the system will be capable of undertaking a smooth transition to personalised routine learning and anomaly scoring without significant redesign.

The underlying design choices at the cloud and application levels have been proved as well. A FastAPI backend, deployed on Render with HTTPS protocol has been specified alongside a relational PostgreSQL database containing sensor summaries, device records and routine backups, providing a secure and ACID-compliant foundation of long-term data persistence. Simultaneously, Flutter mobile application has become structurally complete with mock repositories, Riverpod state management and offline-first architecture. All fundamental screens (Sensors, Routines and Settings) have been implemented and matched to the intended REST-over-HTTPS API, allowing the app to be linked to live data with a few adjustments once the backend endpoints have been started.

Overall, the preliminary findings confirm that the most critical architectural decisions, passive sensing, edge-first processing, a hybrid local-cloud model, modular subsystem boundaries and robust security mechanisms are suitable in an assisted-living environment that has to balance the issues of safety, reliability, usability and privacy. The project is on track with respect to the project plan: the sensing and networking background are in place, the edge ML pipeline is ready in structure to be trained on behavioural training, and the cloud and mobile layers are ready to be integrated. The remaining unfinished task is now focused on the full-scale hardware implementation, ethical consent, multi-room behavioural data, anomaly-detection model training and closing the loop with actuators. The evidence gathered so far indicates that the suggested system is a feasible and ethically accountable framework of an AI-enhanced assisted-living platform, with a clear pathway towards full integration and performance assessment in Semester 2.

10. Reflection

This section presents the reflective summaries from each member of the project team, focusing their individual contributions, challenges and personal development. Each reflection places emphasis on critical self-evaluation, adaptability in response to unforeseen circumstances, communication within a multidisciplinary team and ethical considerations synonymous with designing technology for assisted living environments. These reflections provide insight into how the into how engineering judgement of group members has evolved and how each member plans to refine their approach in preparation for the next stage of the project.

10.1. Mia

My main challenge faced this semester was due to supply chain issues which prevented the development of the actuator subsystem. Due to this delay I had to shift my project priorities, demonstrating adaptability by reallocating time to improving sensor firmware, filtering and data quality rather avoiding stalling of project progress. From this experience I learnt how to manage deviations from the projects critical path without affecting the long-term project deliverables. This reflects a real workplace as often projects involve external constraints which are out of the hands of those involved, thus progress is dependent on strategic reprioritisation, not just rigid adherence to plans.

Designing for an assisted living context required me to shift my mindset with the priorities for privacy and safety outweighing convenience and speed. The easy solution of streaming raw sensor data directly to the cloud was rejected due to GDPR and ethical concerns. Instead, an edge first architecture was adopted ensuring that only minimal anonymised messaged leave the home environment. This resulted in me learning that meeting ethical requirements is not optional and it show be embedded in the systems design from the start.

During Semester 1 I focused most of my efforts in sensor accuracy and reliability which improved the systems machine-learning readiness but resulted in me overlooking a major area, power consumption. At the current time all ESP32 modules operate at full capacity making battery operation not feasible for deployment. Therefore, in semester 2 I will focus on implementing deep sleep modes and interrupt drive wakeup and using the ULP co-processor. This has made me realise that functional prototypes are not the only thing that matters, embedded systems must also be efficient, scalable and deployable.

Throughout the project period I have improved my ability to communicate my progress clearly to my teammates. I have done these through structures technical updates which reduce any ambiguity with what work I am completing. I have also learnt how to be responsive to my teammate's requirements modifying data formats topics and timings to meet requirements of other subsystems in the project. Distributed systems work only when every team member aligns on interface standards and expectations. This has resulted in my learning that good engineering communication prevents integration failures before they even happen.

This project has allowed me to gain experience in balancing technical constraints, ethical requirement, timelines and usability mirroring real industry engineering challenges. Engineering decisions are rarely isolated, privacy design affects sensor and actuator choices, supply chain issues affect timelines and firmware decision affect backend integration. This has allowed me to practice my systematic thinking evaluating the impacts of my decisions across the whole system not just my specific area. I have in turn developed resilience and confidence in handling setbacks, ambiguity and shifting priorities.

10.2. Alvaro

This project has been one of the most useful learning experiences of my degree. It has challenged me, moved me out of my comfort zone and made me develop not only as an engineer, but also as a collaborator and problem solver. Even though every member of the team had extremely different tasks, I got to know how interconnected our work was. I was primarily involved with the machine-learning and behavioural-analysis aspect of the system, but soon I realised that my role would largely rely on how well the sensors worked, whether the data obtained by the ESP32 nodes was clear and coherent, and how the backend was designed such that my outputs would be eventually processed by it.

I was not very familiar with edge-based machine learning nor with how to incorporate machine learning models into a live IoT system at the beginning. This project has compelled me to reconsider my approach to designing algorithms that can be applied to messy and real-world data while having a constrained hardware budget. I got to know how to get raw behavioural data, coordinate it, extract meaningful features, and transform it into something understandable by a machine learning model. It was satisfying to watch the system gradually transforming from sensor readings into regular behavioural patterns and anomaly scores.

I also learned the relevance of effective communication and collaboration. Although I was not the one to write the firmware or install the local MQTT broker, I was able to work directly with my teammates and learned the importance of every component of the pipeline. Waiting until the sensor data is stable, adjusting my scripts to fit the message format and liaising with the backend logic made me realise what it takes to create a complete system and not an isolated part. It helped me to understand the complexity of the engineering projects of the real world and the necessity to constantly adapt, communicate and optimise.

The other valuable lesson that I learnt was the way to verify my work without referring directly to real behavioural data. Due to ethical limitations, I was forced to create simulation spaces and produce artificial patterns that were similar to daily activities. This was initially unnatural but later proved to teach me how to be more creative in thinking and how to test ideas in a controlled and responsible manner. It also sensitised me more on the ethical aspects of assistive technologies particularly in the case of dealing with vulnerable users.

Looking ahead, this project has demonstrated to me how technology can transform the lives of people. The concept of engineering to support independence, safety and wellbeing is a concept that drives me. I can easily see how the skills that I have acquired, not only in machine learning and data engineering but also in teamwork and system thinking will assist me in my future profession, be it in robotics, AI, or development of smart-systems. Above all, it has given me confidence that I can contribute to real, meaningful solutions and it has increased my wish to continue learning and developing as an engineer.

10.3. Dan

Working on this project so far has provided me with technical and professional learning, that I believe will be valuable in my future career. Implementing the MQTT local network and configuring the Raspberry Pi as a secure broker taught me how distributed systems behave in practice, especially when dealing with real devices. Laying the foundations of a functional cloud backend, deploying FastAPI with Render including a PostgreSQL database gave me hands on experience with modern cloud deployment pipelines, API designs and secure data handling. All of these core skills directly map

to careers in IoT engineering, backend development and embedded systems. The process also highlighted the importance of designing systems that can still partially function despite a failure in one area. Understanding how to build a system that maintains local autonomy even when cloud connectivity is lost is also something I would not have fully understood without this project.

I also learned how to evaluate scalability and the behaviour of a system based on early testing. For example, working on a small-scale network at the beginning allowed me to understand how the full system would act in the future and what changes needed to be made. This ability to generalise from prototype to production scale is a key skill in engineering. Additionally, the experience of deploying a cloud platform and configuring HTTPS communication has made me more aware of practical security considerations and trade-offs between usability and performance.

A major challenge over the course of the project so far designing an architecture to fit the needs of each branch of work. Designing a system that accommodates to the specific requirements of other subsystems, such as the ML model, the sensors and actuators, and the app has taught me the importance of communication, documentation, clarity, and a modular system architecture. I also learned the importance of observing existing solutions and applying certain logic from them to adapt my approach and overcome challenges, in addition to adapting the design when the requirements change. This mirrors the way multidisciplinary teams operate in industry, where backend, hardware and app developers must work together and depend on each other's progress. Navigating these scenarios has improved my ability to collaborate, plan and negotiate technical decisions.

Regarding future potential, I believe the project can evolve into a scalable platform for assisted living and smart home automation. The way the architecture is designed, it is flexible enough to be extended whether it be with more sensors, deeper analytics, or more advanced features. The system could also be used in energy monitoring, environmental sensing or building automation. The skills I have learned and built upon so far have prepared me to contribute effectively for roles related to IoT and cloud engineering in my future career. On a broader scale. The experience thus far has strengthened my confidence when approaching complex engineering problems and producing a solution that is functional and scalable.

10.4. Luis

My work in the project was oriented on the design and the initial work on the mobile application as the main interface of the caregivers and end users. In retrospect, the most significant success has been to have a strong architectural base of the app even though the backend, machine learning model and sensor infrastructure are still being developed and there for not connected on its entirety. The use of Flutter, Riverpad, and mock data made me consider scalability and modularity early on, so that the product is easy to integrate with the Raspberry Pi and FastAPI backend and cloud database.

The most critical problem that I encountered was the creation of the interface and internal data flow when I did not know the live endpoints of the backend. It involved the need to learn to utilise mock repositories successfully and embrace the concept of clean architecture to ensure that the UI and logic will be unaffected by the introduction of real data. This limitation made me implement better practises sooner than anticipated, like the separation of domain models, repository layers and state management. Lately this was an abstract concept but now I understand the importance of such decisions in the long run maintainability.

The other difficulty was the realisation of how my work fits into the larger IoT-AI pipeline particularly due to the complexity of the edge processing, ML model, and cloud infrastructure. This cooperation

with the rest of the team allowed me to comprehend the interaction of each component and this contributed to my skills in designing an app that is flexible and robust. To illustrate, the pre-planning of API client and repository structures taught me how to pre-plan requirements of the system and how to design interfaces prior to system implementation, a skill that directly translates to the actual engineering project.

Regarding the acquired skills, I have become much more skilled in Flutter, such as state management, navigation architecture, and the creation of reusable UI components. I also got to learn about designing with offline-first behaviour, how to plan to use secure communication with HTTPS, and how to design mobile apps to be able to use modular data sources. These are necessary to any production grade application, and the project provided me practical practise in putting these skills into practise.

Going forward, I can identify several areas that I consider as areas of improvement. First, I will have to finish the integration process with the FastAPI backend after it is deployed and use real sensor summaries and routines instead of mock ones. Second, it will be necessary to apply local database caching with Drift to make the database offline reliable, which I now see is very important to assisted-living applications. Third, I would like to introduce push notifications, which would inform the caregivers about the anomalies as soon as possible even when the app is not active. Lastly, I would like to enhance my knowledge about backend-frontend synchronisation pattern as a way of facilitating long term scalability.

Overall, the technical skills as well as my skills to cooperate within the multidisciplinary system have been improved in this phase of the project. Though the app under consideration is based on fake data, the skeleton is ready, and I believe that further development will go on without any major challenges as the machine-learning and backend are made available.

11. Bibliography

[1] World Health Organization, “Ageing and health,” *World Health Organization*, Oct. 01, 2024. https://www.who.int/news-room/fact-sheets/detail/ageing-and-health

[2] D. Foster, “Adult social care workforce in England,” *House of Commons Library*, Oct. 14, 2024. https://commonslibrary.parliament.uk/research-briefings/cbp-9615/

[3] Fortune Business Insights , “IoMT (Internet of Medical Things) Market Size, Share | Trends, 2026,” *www.fortunebusinessinsights.com*, Nov. 17, 2025. https://www.fortunebusinessinsights.com/industry-reports/internet-of-medical-things-iomt-market-101844

[4] StackCare, “StackCare,” *StackCare*, 2025. https://www.stackcare.co.uk/ (accessed Nov. 12, 2025).

[5] SECOM, “Wellness | Assisted Home Care | SECOM Plc,” *SECOM Security Systems*, Mar. 31, 2025. https://secom.plc.uk/smart-wellness/ (accessed Nov. 12, 2025).

[6] Taking Care, “Taking Care Sense,” *Taking Care Personal Alarms*, 2025. https://taking.care/products/taking-care-sense?srsltid=AfmBOoq_WheFXR8KE6U51_6xFOGMpE4zuEObjfLs6vVdPlZxNoz9KM9g (accessed Nov. 12, 2025).

[7] A. Staff, “Alexa Together launches to help customers remotely care for loved ones,” *US About Amazon*, Dec. 07, 2021. https://www.aboutamazon.com/news/devices/alexa-together-launches-to-help-customers-remotely-care-for-loved-ones

[8] Apple, “Home App,” *Apple (United Kingdom)*, Sep. 29, 2025. https://www.apple.com/uk/home-app/

[9] Home Assistant, “Home Assistant,” *Home Assistant*, 2019. https://www.home-assistant.io/

[10] UL Solutions , “Guide to ETSI EN 303 645 Compliance Services,” *UL Solutions*, 2024. https://www.ul.com/resources/guide-etsi-en-303-645-compliance-services

[11]ISO, “ISO/IEC 30141:2024,” *ISO*, 2024. https://www.iso.org/standard/88800.html

[12] UK Government, “UK General Data Protection Regulation,” *legislation.gov.uk*, 2018. https://www.legislation.gov.uk/eur/2016/679/contents

[13]ISO, “Navigating the ISO 27001 compliance journey,” *Onetrust.com*, 2016. https://www.onetrust.com/resources/iso-27001-compliance-ebook/?ef_id=CjwKCAiAlrXJBhBAEiwA-5pgwvPuzlZXbwRPIsoWke61n1_C3A8AZy9oliUj7LjauWlRyjOBbjNABRoCnCwQAvD_BwE:G:s&s_kwcid=AL (accessed Dec. 08, 2025).

[14] Centers for Disease Control and Prevention, “Health insurance portability and accountability act of 1996 (HIPAA),” *Public Health Law*, Sep. 10, 2024. https://www.cdc.gov/phlp/php/resources/health-insurance-portability-and-accountability-act-of-1996-hipaa.html

[15] C. Aggar, G. Sorwar, C. Seton, O. Penman, and A. Ward, “Smart home technology to support older people’s quality of life: A longitudinal pilot study,” *International Journal of Older People Nursing*, vol. 18, no. 1, Jul. 2022, doi: https://doi.org/10.1111/opn.12489.

[16] Y. Chen, S. Chen, and Y. Ma, “Elderly monitoring system based on multisensor information fusion,” *Advanced Sensor Systems and Applications XIV*, vol. 13243, no. 13243OS, p. 29, Nov. 2024, doi: https://doi.org/10.1117/12.3035780.

[17] CST, “IOT Communication Models,” *CS Taleem*, Oct. 05, 2021. https://cstaleem.com/iot-communication-models (accessed Dec. 08, 2025).

[18] GeeksforGeeks, “What is Pub/Sub Architecture?,” *GeeksforGeeks*, Aug. 20, 2023. https://www.geeksforgeeks.org/system-design/what-is-pub-sub/

[19] KAA IoT, “IoT communication models: overview, types, use cases and implementation strategies,” *Kaaiot.com*, Jun. 06, 2025. https://www.kaaiot.com/iot-knowledge-base/iot-communication-models-overview-types-use-cases-and-implementation-strategies

[20] L. M. Miller, J. Kaye, A. Lindauer, W.-T. M. Au-Yeung, N. K. Rodrigues, and S. J. Czaja, “Remote Passive Sensing of Older Adults’ Activities and Function: User-Centered Design Considerations for Behavioral Interventions Conducted in the Home Setting (Preprint),” *Journal of Medical Internet Research*, vol. 26, pp. e54709–e54709, Aug. 2024, doi: https://doi.org/10.2196/54709.

[21] S. Narayana, R. V. Prasad, V. S. Rao, T. V. Prabhakar, S. S. Kowshik, and M. S. Iyer, “PIR sensors,” *Proceedings of the 14th International Conference on Information Processing in Sensor Networks - IPSN ’15*, 2015, doi: https://doi.org/10.1145/2737095.2742561.

[22]S. Rao and T. Instruments, “Introduction to mmwave Sensing: FMCW Radars,” 2024. Accessed: Dec. 08, 2025. [Online]. Available: https://e2e.ti.com/cfs-file/__key/communityserver-discussions-components-files/1023/Introduction-to-mmwave-Sensing-FMCW--Radars.pdf

[23] A. T. Tran, X. Zhang, and B. Zhu, “The Development of a New Piezoresistive Pressure Sensor for Low Pressures,” *IEEEE Transactions on Industrial Electronics*, vol. 65, no. 8, pp. 6487–6496, Aug. 2018, doi: https://doi.org/10.1109/tie.2017.2784341.

[24] L. Doulos, A. Tsangrassoulis, and F. V. Topalis, “The role of spectral response of photosensors in daylight responsive systems,” *Energy and Buildings*, vol. 40, no. 4, pp. 588–599, Jan. 2008, doi: https://doi.org/10.1016/j.enbuild.2007.04.010.

[25] R. Sarrate, J. Blesa, F. Nejjari, and J. Quevedo, “Sensor placement for leak detection and location in water distribution networks,” *Iwaponline.com*, 2025. https://iwaponline.com/ws/article-abstract/14/5/795/27419/Sensor-placement-for-leak-detection-and-location (accessed Dec. 08, 2025).

[26] V. Merugu, “Door Monitoring System Using Reed Switch,” *SSRN Electronic Journal*, Jan. 2021, doi: https://doi.org/10.2139/ssrn.3917891.

[27] E. Por, M. Van Kooten, and V. Sarkovic, “Nyquist-Shannon sampling theorem 1 Theory 1.1 The Nyquist-Shannon sampling theorem,” May 2019. Available: https://home.strw.leidenuniv.nl/~por/AOT2019/docs/AOT_2019_Ex13_NyquistTheorem.pdf

[28] G. A. Kivman, “Sequential parameter estimation for stochastic systems,” *Nonlinear processes in geophysics*, vol. 10, no. 3, pp. 253–259, Jun. 2003, doi: https://doi.org/10.5194/npg-10-253-2003.

[29] E. A. Robinson and S. Treitel, “PRINCIPLES OF DIGITAL FILTERING,” *Geophysics*, vol. 29, no. 3, pp. 395–404, Jun. 1964, doi: https://doi.org/10.1190/1.1439370.

[30] L. Aubard, G. Verneau, J. C. Crebier, C. Schaeffer, and Y. Avenas, “Power MOSFET switching waveforms: an empirical model based on a physical analysis of charge locations,” *IEEE 33rd Annual IEEE Power Electronics Specialists Conference*, Jun. 2003, doi: https://doi.org/10.1109/psec.2002.1022357.

[31] C. Hooge, “23.2 Faraday’s Law of Induction: Lenz’s Law,” *pressbooks.bccampus.ca*, Aug. 2016, Available: https://pressbooks.bccampus.ca/physics0312chooge/chapter/23-2-faradays-law-of-induction-lenzs-law/

[32] H.-S. Kim, Y.-J. Park, and S.-J. Kang, “Secured and Deterministic Closed-Loop IoT System Architecture for Sensor and Actuator Networks,” *Sensors*, vol. 22, no. 10, pp. 3843–3843, May 2022, doi: https://doi.org/10.3390/s22103843.

[33] Y. Bistritz, “Bounded-Input Bounded-Output Stability Tests for Two-Dimensional Continuous-Time Systems,” *IEEE Transactions on Circuits and Systems I Regular Papers*, vol. 68, no. 5, pp. 2134–2147, Mar. 2021, doi: https://doi.org/10.1109/tcsi.2021.3059839.

[34] R. M. Abdelmoneem, E. Shaaban, and A. Benslimane, “A Survey on Multi-Sensor Fusion Techniques in IoT for Healthcare,” *13th International Conference on Computer Engineering and Systems* , Dec. 2018, doi: https://doi.org/10.1109/icces.2018.8639188.

[35] G. Shafer, “Dempster-Shafer Theory,” 1990. Available: https://fitelson.org/topics/shafer.pdf

[36] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge Computing: Vision and Challenges,” *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, Oct. 2016, doi: https://doi.org/10.1109/jiot.2016.2579198.

[37] MQTT, “MQTT - The Standard for IoT Messaging,” *mqtt.org*, 2024. https://mqtt.org/

[38] V. Selis, “Lecture 7 - M2M Comms for IoT,” *Canvas - University of Liverpool* , 2025. https://liverpool.instructure.com/courses/87504/pages/lecture-7-m2m-comms-for-iot

[39] G. Restuccia, H. Tschofenig, and E. Baccelli, “Low-Power IoT Communication Security: On the Performance of DTLS and TLS 1.3,” *IEEE Xplore*, Dec. 01, 2020. https://ieeexplore.ieee.org/abstract/document/9293085

[40] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation Forest,” *2008 Eighth IEEE International Conference on Data Mining*, Dec. 2008, doi: https://doi.org/10.1109/icdm.2008.17.

[41] K. Sinha, “LSH vs Randomized Partition Trees: Which One to Use for Nearest Neighbor Search?,” *2014 13th International Conference on Machine Learning and Applications*, pp. 41–46, Dec. 2014, doi: https://doi.org/10.1109/icmla.2014.13.

[42] Centre for Advanced Systems in Adaptive Systems , “Welcome to CASAS,” *casas.wsu.edu*, 2009. https://casas.wsu.edu/datasets/

[43] M. Ciliberto, V. Fortes Rey, A. Calatroni, P. Lukowicz, and D. Roggen, “Opportunity++: A Multimodal Dataset for Video- and Wearable, Object and Ambient Sensors-Based Human Activity Recognition,” *Frontiers in Computer Science*, vol. 3, Dec. 2021, doi: https://doi.org/10.3389/fcomp.2021.792065.

[44] F. Schuster and A. Habibipour, “Users’ Privacy and Security Concerns that Affect IoT Adoption in the Home Domain,” *International Journal of Human–Computer Interaction*, vol. 40, no. 7, pp. 1–12, Nov. 2022, doi: https://doi.org/10.1080/10447318.2022.2147302.

[45] S. Bhamare, “What is Flutter: Definition, Benefits, and Limitations | BrowserStack,” *BrowserStack*, Mar. 04, 2025. https://www.browserstack.com/guide/what-is-flutter

[46] Very Good Ventures, “10 Reasons To Choose Flutter for Your Mobile App,” *Verygood.ventures*, 2025. https://www.verygood.ventures/blog/10-reasons-why-to-choose-flutter-for-your-mobile-app (accessed Dec. 09, 2025).

[47] P. Gadient, M. Ghafari, M.-A. Tarnutzer, and O. Nierstrasz, “Web APIs in Android through the Lens of Security,” Jan. 2020, doi: https://doi.org/10.1109/saner48275.2020.9054850.

[48] GeeksforGeeks, “REST API Introduction,” *GeeksforGeeks*, Jun. 06, 2023. https://www.geeksforgeeks.org/node-js/rest-api-introduction/

[49] H. Mishra, “REST-Based Communication API in IoT,” *IoTbyHVM*, May 29, 2025. https://iotbyhvm.ooo/rest-based-communication-api-in-iot/ (accessed Dec. 09, 2025).

[50] R. Furtado, “What is TLS: A Comprehensive Guide to Online Security,” *Vodien Blog*, Aug. 28, 2024. https://www.vodien.com/learn/what-is-tls-transport-layer-security/ (accessed Dec. 09, 2025).

[51] A. Dubey, “Offline-First Architecture in Flutter: Part 1 — SQLite Local Storage and Conflict Resolution,” *DEV Community*, Aug. 09, 2025. https://dev.to/anurag_dev/implementing-offline-first-architecture-in-flutter-part-1-local-storage-with-conflict-resolution-4mdl (accessed Dec. 09, 2025).

[52] GeeksforGeeks, “Explain the Working of HTTPS,” *GeeksforGeeks*, Nov. 26, 2021. https://www.geeksforgeeks.org/html/explain-working-of-https/

[53] Raspberry Pi Ltd, “Raspberry Pi 5,” *Raspberry Pi*. https://www.raspberrypi.com/products/raspberry-pi-5/

[54] A. Piltch, “Raspberry Pi 5’s Wi-Fi Tested: Up to 3x Faster,” *Tom’s Hardware*, Sep. 30, 2023. https://www.tomshardware.com/news/raspberry-pi-5-wi-fi-faster

[55] EMQ Team, “MQTT Ports: Common Ports and How to Configure and Secure Them,” *www.emqx.com*, Dec. 03, 2023. https://www.emqx.com/en/blog/mqtt-ports

[56] DEV Station , “MQTT QoS: Understanding The 3 Levels Of Service – Dev Station Technology,” *Dev Station Technology*, Sep. 18, 2025. https://dev-station.tech/mqtt-quality-of-service-qos/ (accessed Dec. 09, 2025).

[57] YSF, “Rest API VS Fast API,” *DEV Community*, Feb. 21, 2024. https://dev.to/ysf00009/rest-api-vs-fast-api-4d3o

[58] codezup, “Building RESTful APIs with Python & FastAPI,” *Codez Up*, Apr. 04, 2025. https://codezup.com/building-restful-apis-python-fastapi-step-by-step-guide/ (accessed Dec. 09, 2025).

[59] Render, “Fully Managed TLS Certificates – Render Docs,” *Fully Managed TLS Certificates – Render Docs*, 2024. https://render.com/docs/tls

[60] I. Ghat, “A New Era of Simplified Deployment,” *DEV Community*, Aug. 22, 2024. https://dev.to/irfanghat/a-new-era-of-simplified-deployment-51hg (accessed Dec. 09, 2025).

[61] D. Erhabor, “ACID transactions and implementation in a PostgreSQL Database - Aviator Blog,” *Aviator*, Mar. 08, 2023. https://www.aviator.co/blog/acid-transactions-postgresql-database/

[62] H. Lu, “FastAPI ORM Mode: Leveraging ORM Objects with orm_mode,” *Getorchestra.io*, Apr. 04, 2025. https://www.getorchestra.io/guides/fastapi-orm-mode-leveraging-orm-objects-with-orm-mode (accessed Dec. 09, 2025).

[63] PostgreSQL, “5.1. Table Basics,” *PostgreSQL Documentation*, Sep. 26, 2024. https://www.postgresql.org/docs/current/ddl-basics.html

[64] B. Sandu, “What Is a Remote Repository? Explained Simply,” *TMS Outsource*, May 13, 2025. https://tms-outsource.com/blog/posts/what-is-a-remote-repository/ (accessed Dec. 09, 2025).

[65] A. J. Ahmed, “Hybrid Power: Flutter Advantages and Benefits | Toptal®,” *Toptal Engineering Blog*, Jul. 24, 2025. https://www.toptal.com/flutter/hybrid-power-flutter-advantages

[66] Cloudflare, “Why use TLS 1.3? | SSL and TLS vulnerabilities,” *Cloudflare.com*, 2018. https://www.cloudflare.com/en-gb/learning/ssl/why-use-tls-1.3/

[67] S. Mangla, “Offline-First Done Right: Sync Patterns for Real-World Networks,” *Developers Voice | The Software Architects Hub*, Sep. 08, 2025. https://developersvoice.com/blog/mobile/offline-first-sync-patterns/ (accessed Dec. 09, 2025).

[68] Arduino, “Libraries,” *Arduino.cc*, 2024. https://docs.arduino.cc/libraries/

[69] M. Satyanarayanan, “The Emergence of Edge Computing,” *Computer*, vol. 50, no. 1, pp. 30–39, Jan. 2017, doi: https://doi.org/10.1109/mc.2017.9.

[70] Grepow, “What’s the Difference Between LiPo vs NiMH Battery? | Grepow,” *Grepow.com*, 2025. https://www.grepow.com/blog/lipo-battery-vs-nimh-battery-what-is-the-difference.html

[71] Electrical4U, “MOSFET as a Switch | Electrical4U,” *https://www.electrical4u.com/*, Apr. 24, 2024. https://www.electrical4u.com/mosfet-as-a-switch/

Appendices

Appendix A – Machine Learning

Terminal output from the MQTT collector showing successful real-time reception of the three bedroom sensors -https://github.com/miahornett/Iot-and-AI-Home-Assistant-/blob/34d99b1e4f245f41597ddc84556128563e544f05/.vscode/bedroom_sensors_reading.png
CSV training file -https://github.com/miahornett/Iot-and-AI-Home-Assistant-/blob/dfc09108308ad40be52a737af877db0d0d49caf1/.vscode/mqtt_training_20251204_112825.csv
JSON training file -https://github.com/miahornett/Iot-and-AI-Home-Assistant-/blob/dfc09108308ad40be52a737af877db0d0d49caf1/.vscode/mqtt_training_20251204_112825.json

Sensors launch file python script -https://github.com/miahornett/Iot-and-AI-Home-Assistant-/blob/34d99b1e4f245f41597ddc84556128563e544f05/.vscode/mqtt_sensor_test_fixed.py
Manual training python script -https://github.com/miahornett/Iot-and-AI-Home-Assistant-/blob/34d99b1e4f245f41597ddc84556128563e544f05/.vscode/enhanced_mqtt_collector
Model trainer python script -https://github.com/miahornett/Iot-and-AI-Home-Assistant-/blob/34d99b1e4f245f41597ddc84556128563e544f05/.vscode/enhanced_mqtt_trainer
Functional anomaly detection mode python script -https://github.com/miahornett/Iot-and-AI-Home-Assistant-/blob/34d99b1e4f245f41597ddc84556128563e544f05/.vscode/enhanced_mqtt_detector

Appendix B – Sensor And Actuator Subsystem

B.1 Scaled Environment and CAD

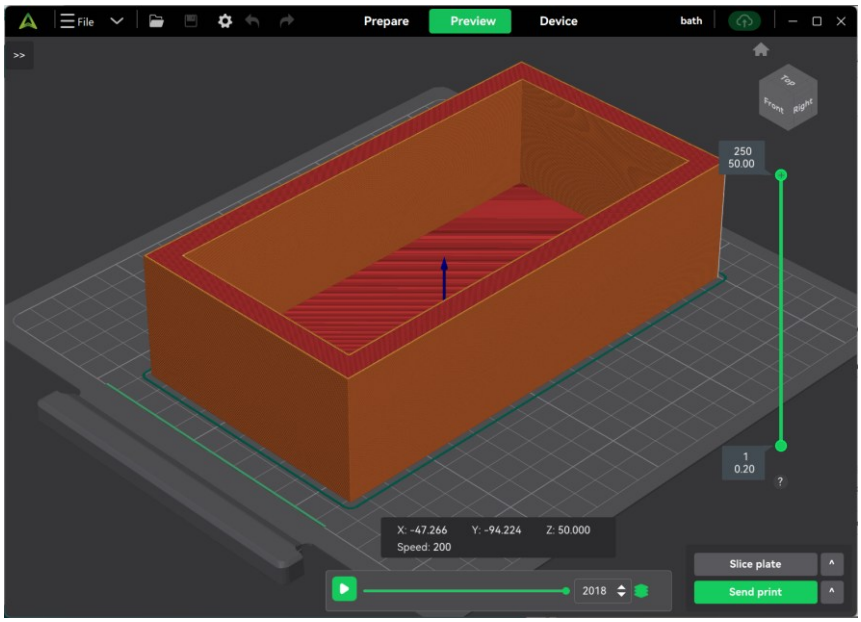


Figure 26 - Bath CAD

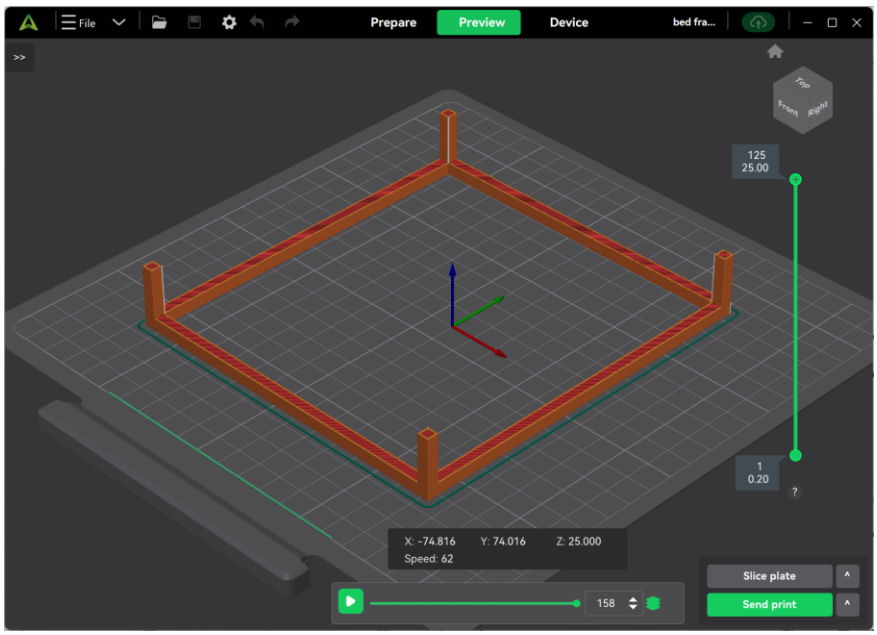


Figure 27 - Bedframe CAD

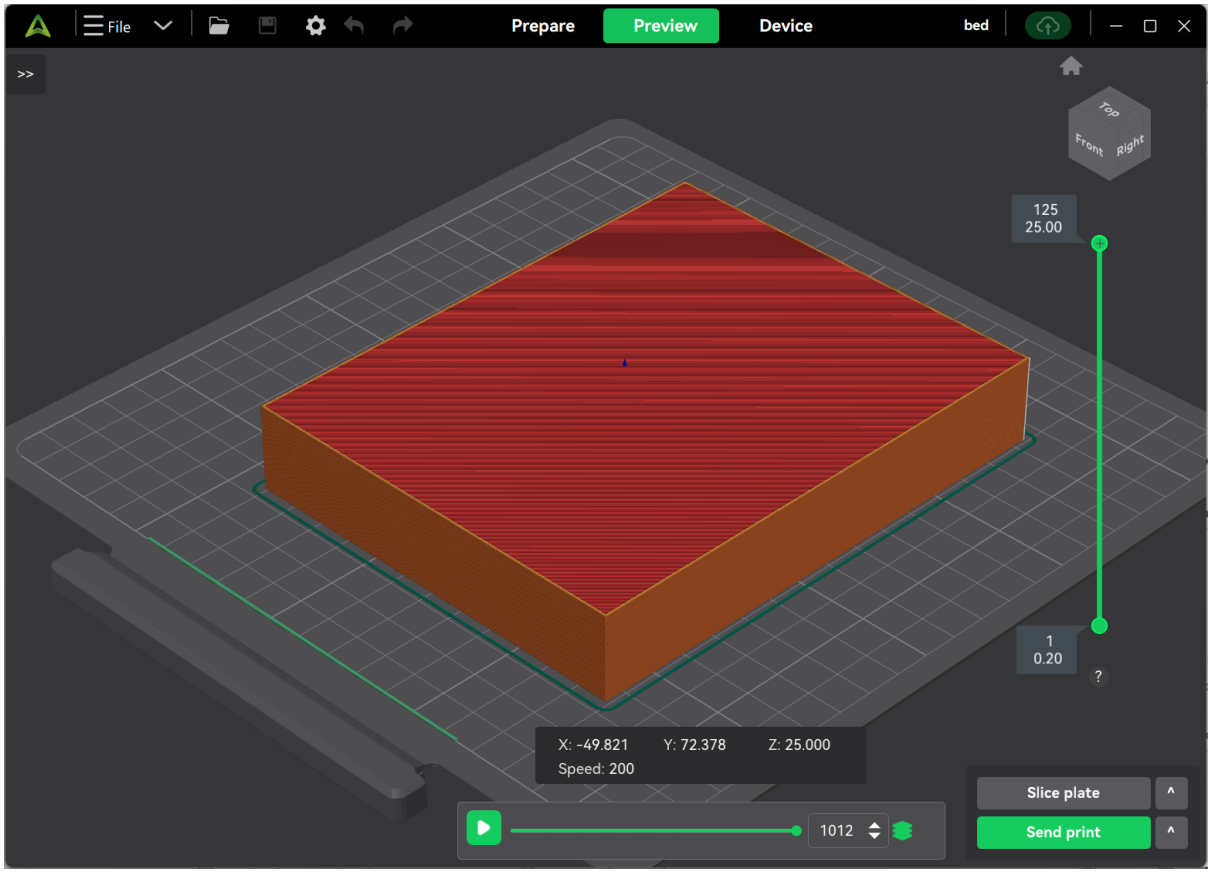


Figure 28 - Bed CAD

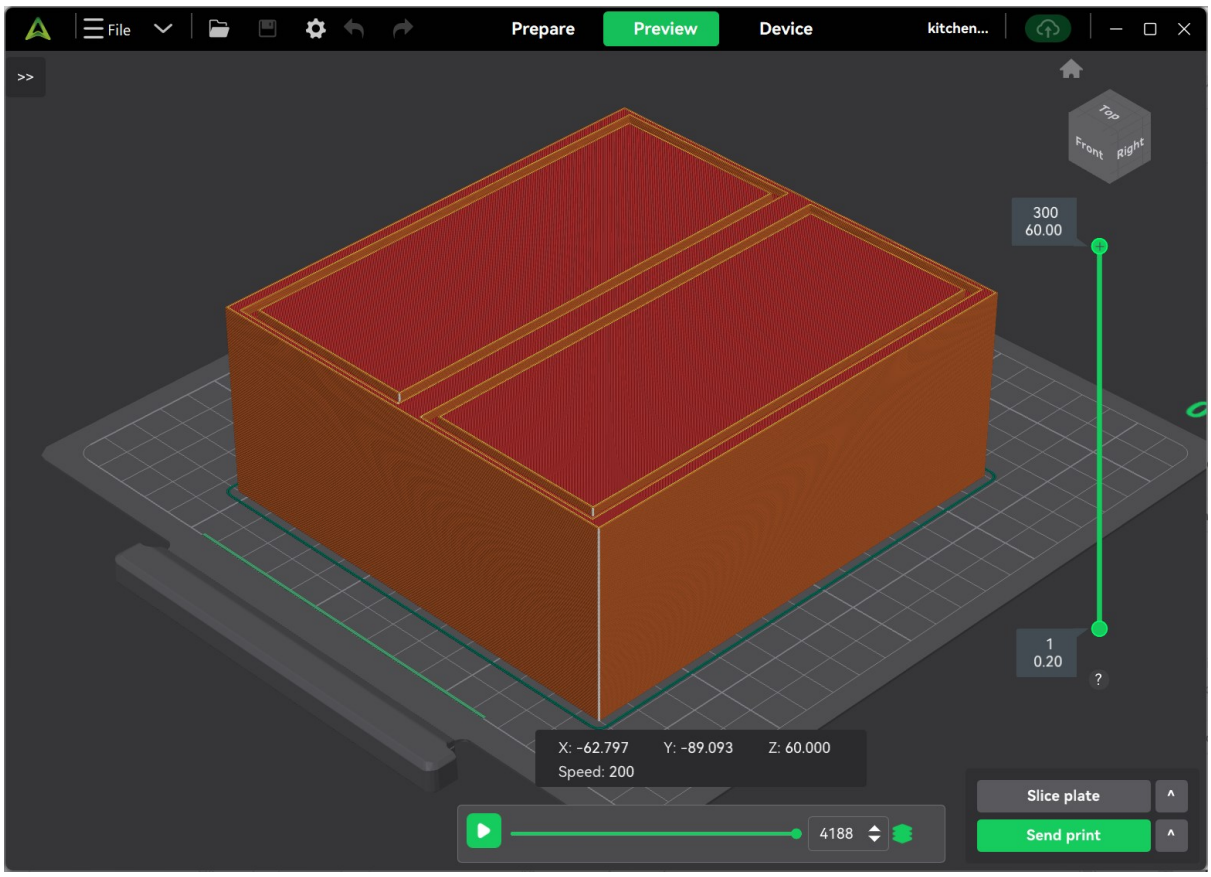


Figure 29 - Wardrobe CAD

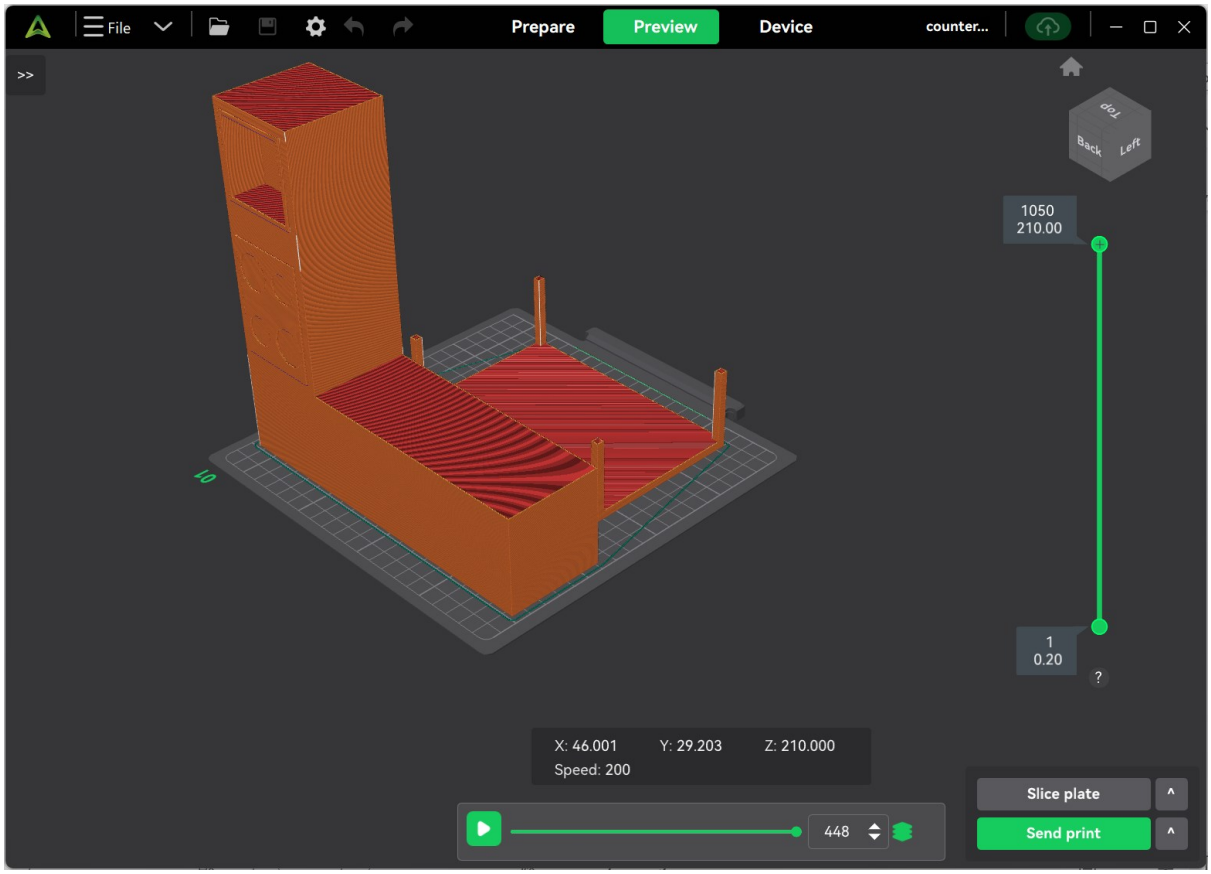


Figure 30 - Kitchen Counter and Table CAD

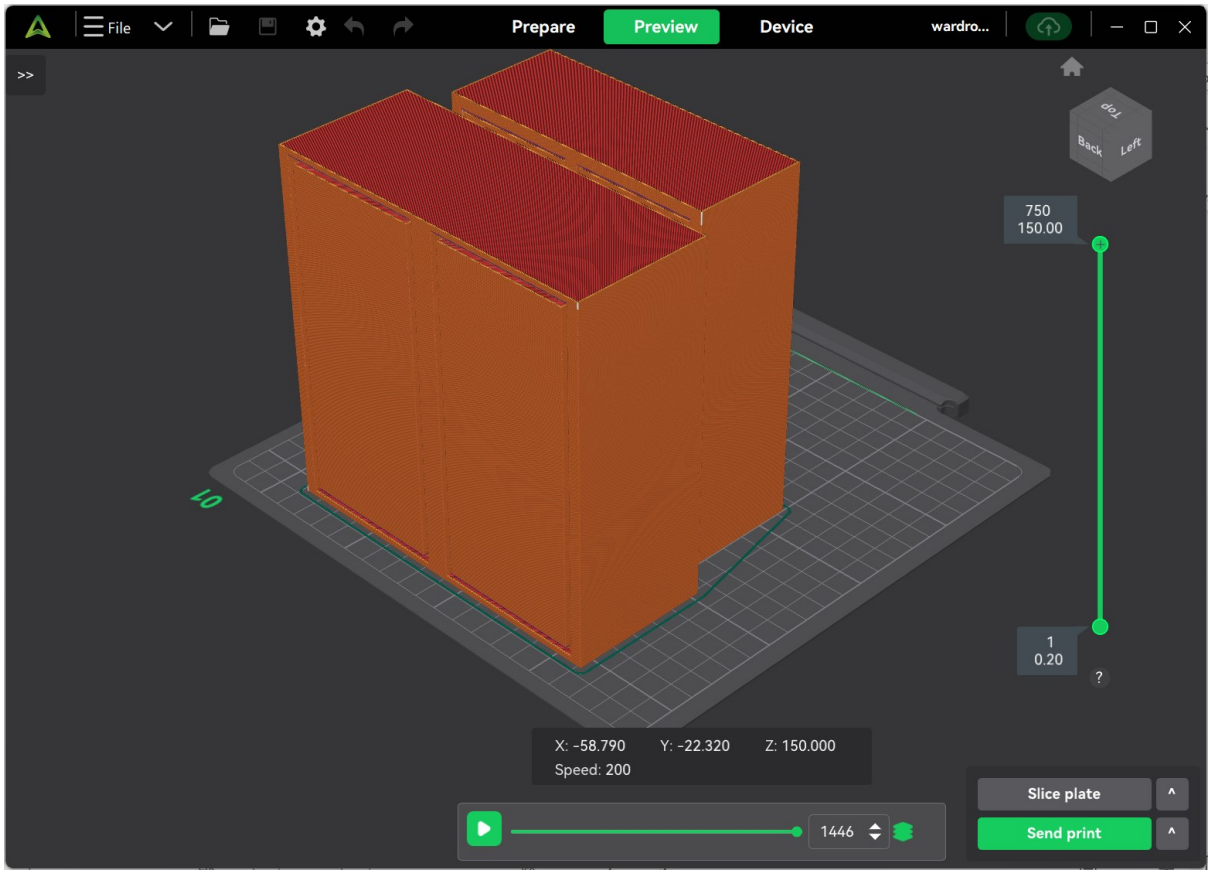


Figure 31 - Cupboard CAD

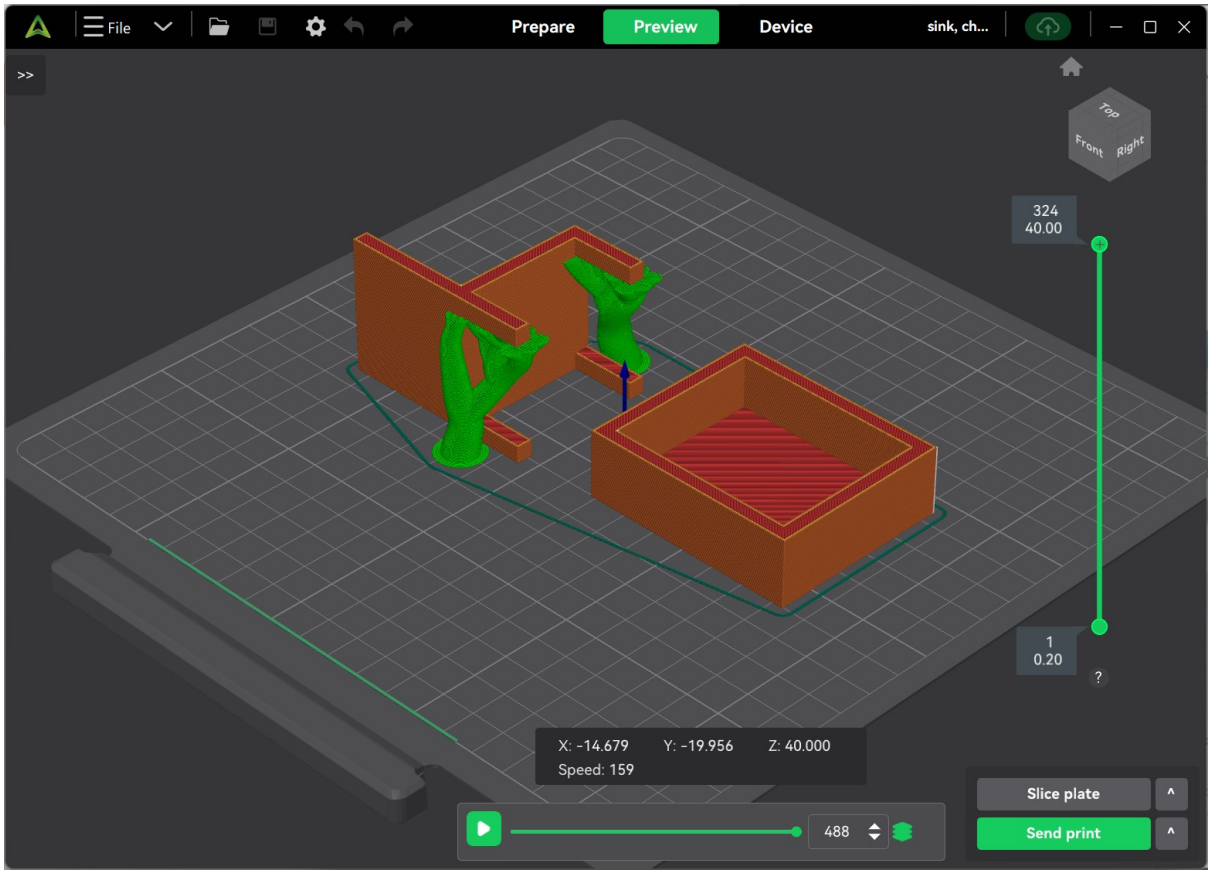


Figure 32 - Kitchen Chair & Sink CAD

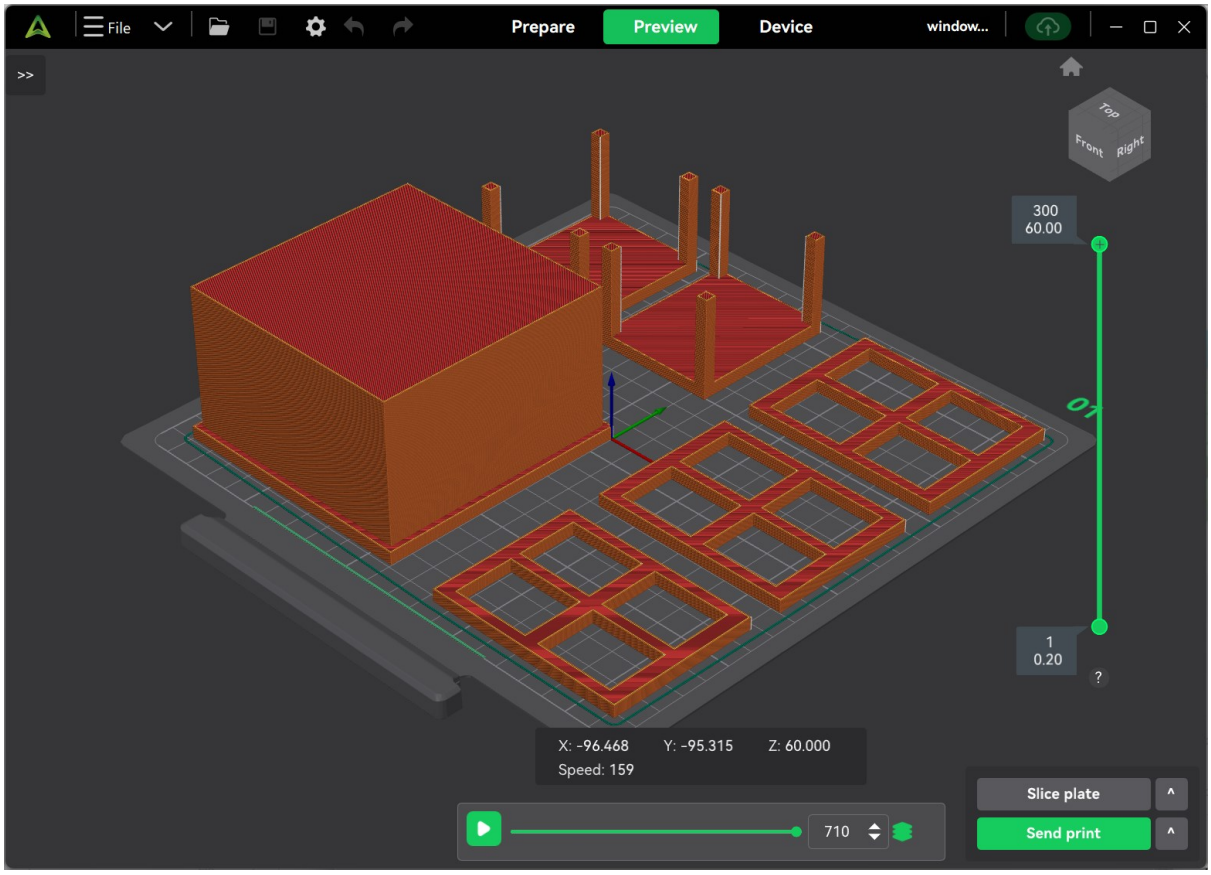


Figure 33 - Kitchen Island, Bed Tables and Window CAD

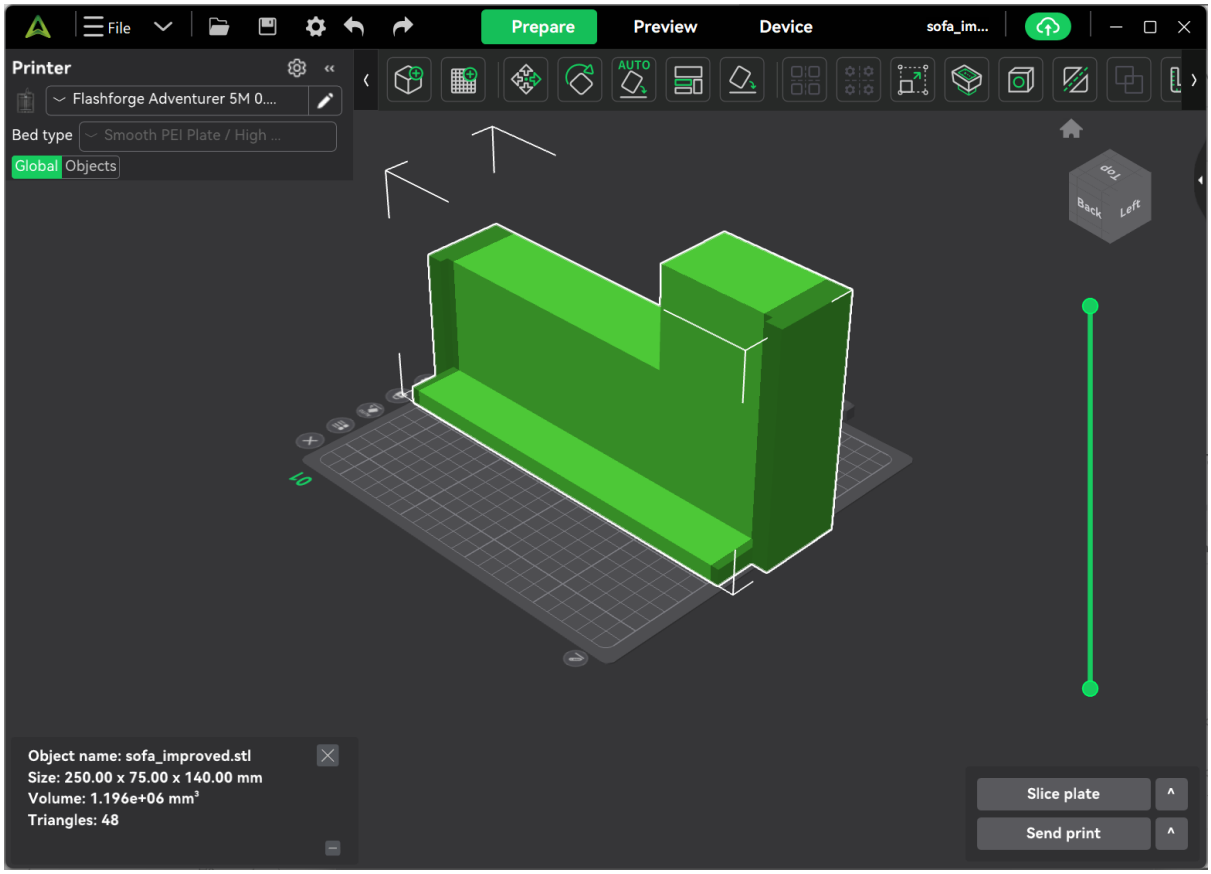


Figure 34 - Sofa CAD

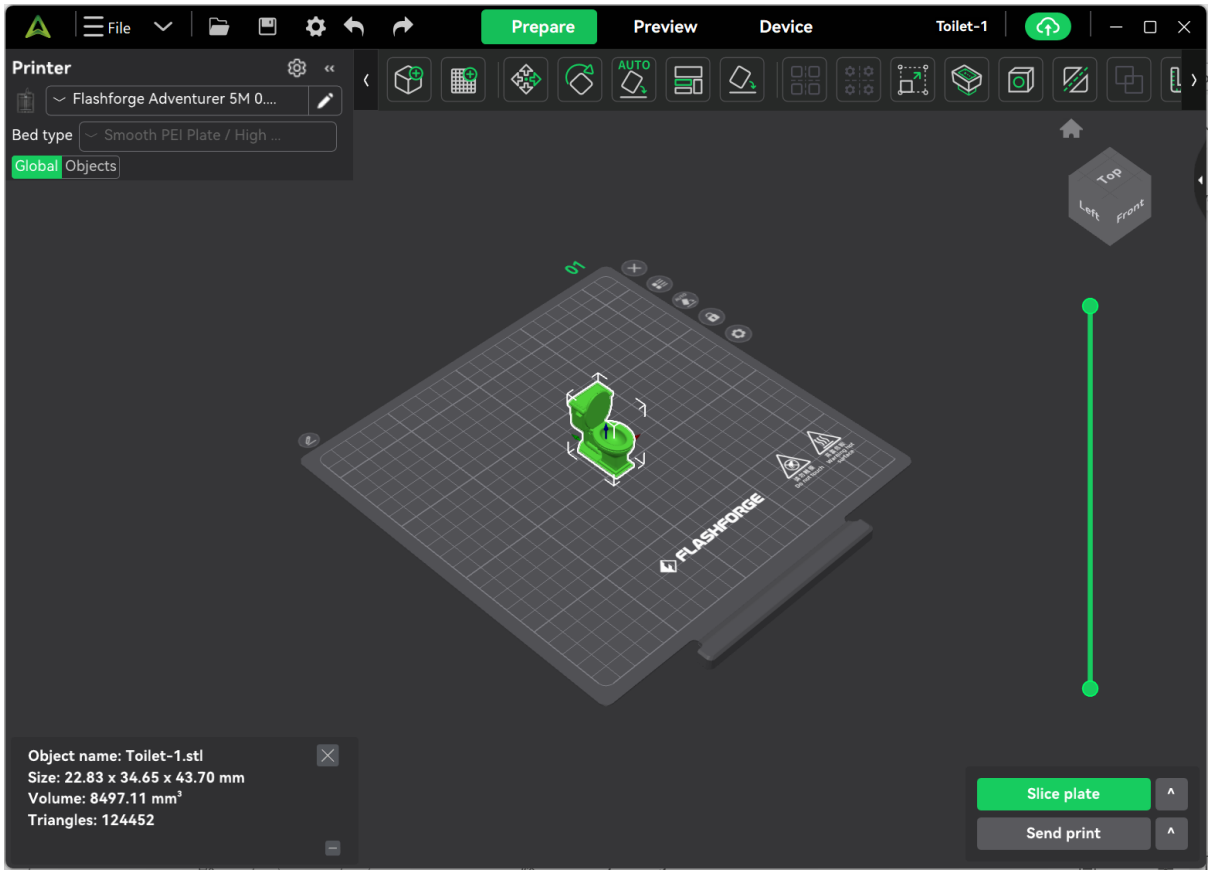


Figure 35 - Toilet CAD

Figure 36 - Small Scale Environment

B.2 PIR

PIR sensor function script - https://github.com/miahornett/Iot-and-AI-Home-Assistant-/blob/ad13d0d13238f9942569f39dc816d65bc1287708/ESP32/PIR_Sensor/PIR.ino

PIR sensor MQTT script - https://github.com/miahornett/Iot-and-AI-Home-Assistant-/blob/ad13d0d13238f9942569f39dc816d65bc1287708/ESP32/PIR_Sensor/PIR_sampling_rate.ino

B.3 mmWave

mmWave function script - https://github.com/miahornett/Iot-and-AI-Home-Assistant-/blob/6b30ad050261a190367d1164a3ee3846dd9d11aa/ESP32/mmWave_Sensor/mmWave_MQTT.ino

mmWave MQTT script - https://github.com/miahornett/Iot-and-AI-Home-Assistant-/blob/9e1ef51c5d47cf7cf03539537077692be9373f55/ESP32/mmWave_Sensor/mmWave_MQTT.ino

B.4 Pressure

Pressure function script - https://github.com/miahornett/Iot-and-AI-Home-Assistant-/blob/9e1ef51c5d47cf7cf03539537077692be9373f55/ESP32/Pressure_Sensor/Code/Calibration

Pressure MQTT Script - https://github.com/miahornett/Iot-and-AI-Home-Assistant-/blob/9e1ef51c5d47cf7cf03539537077692be9373f55/ESP32/Pressure_Sensor/Code/pressure_sampling_rate.ino

Appendix C – Mobile App

Link to the github where you can find the interface of the app: https://github.com/miahornett/Iot-and-AI-Home-Assistant-/tree/main/smart_home_app

Tabs representation with the mock data for visuals aid:

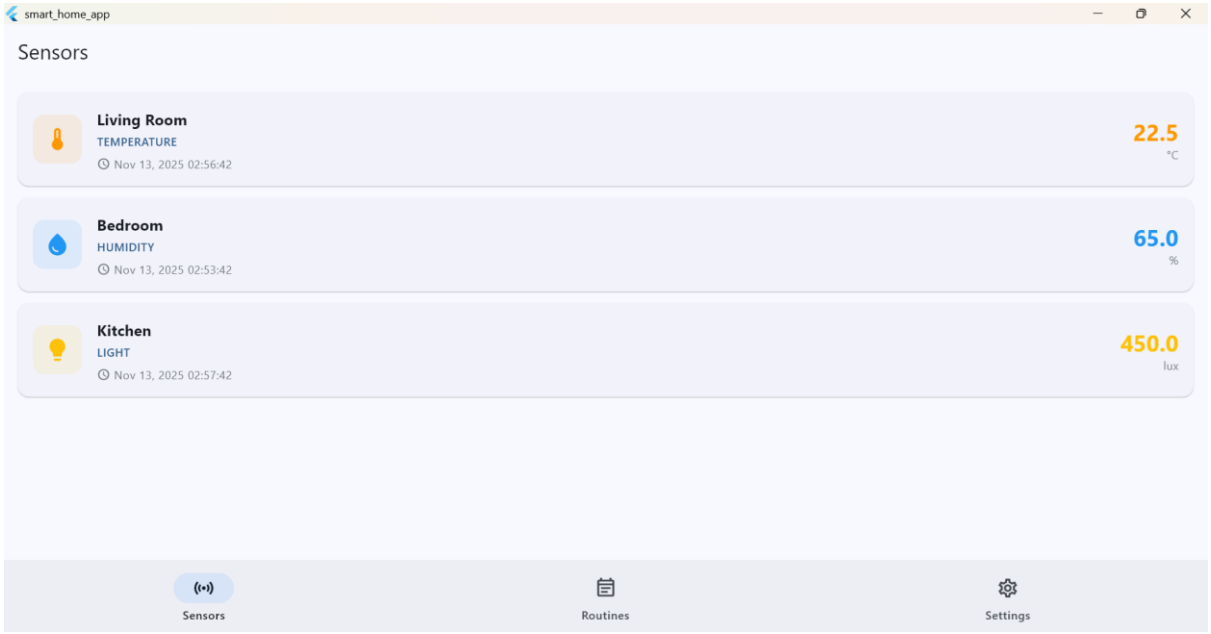


Figure 37 - Sensor Tab

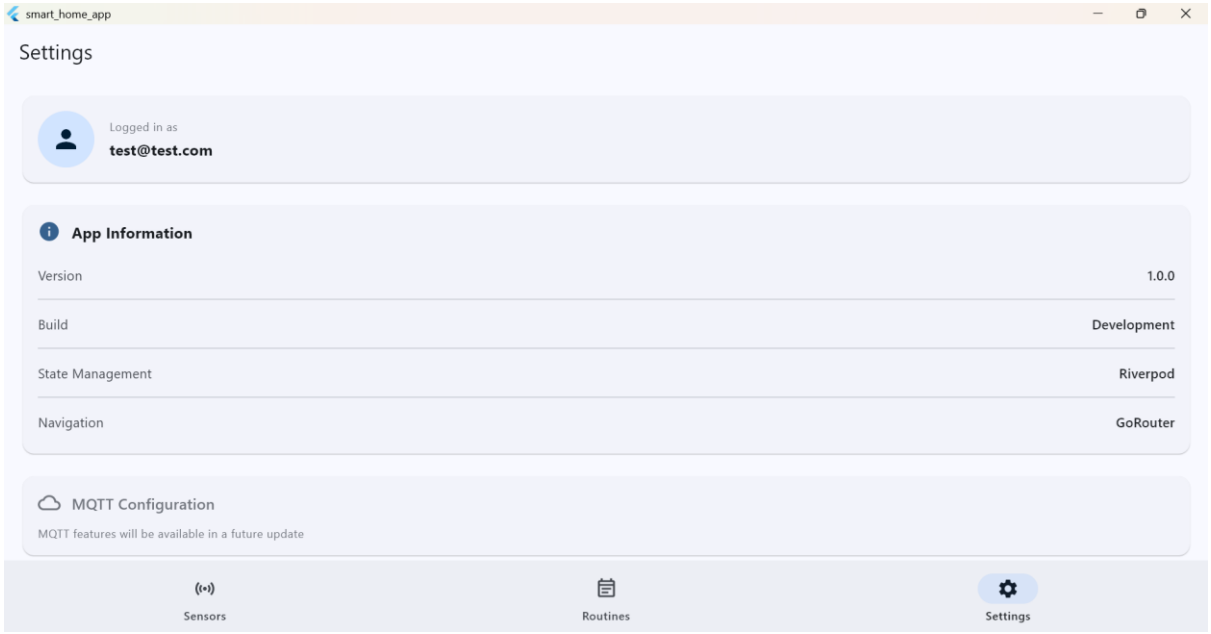


Figure 38 - Setting Tab

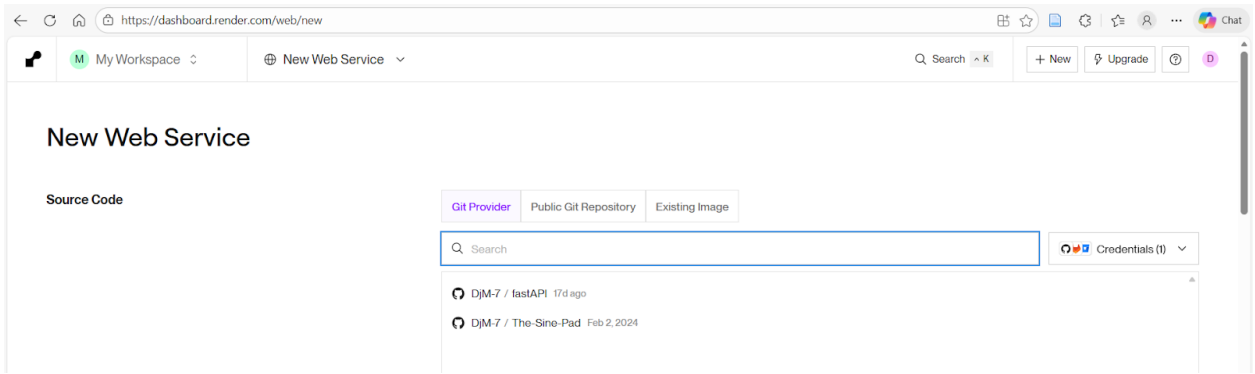
Figure 39 - Routines Tab Overview

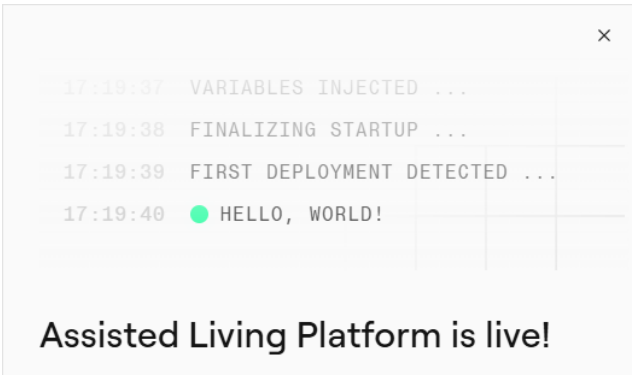
Figure 40 - Routines Tab, Create a New Routine Function

Figure 41 - Routine Tab, Routines Set

Appendix D – Architecture

Cloud Backend Deployment on Render:

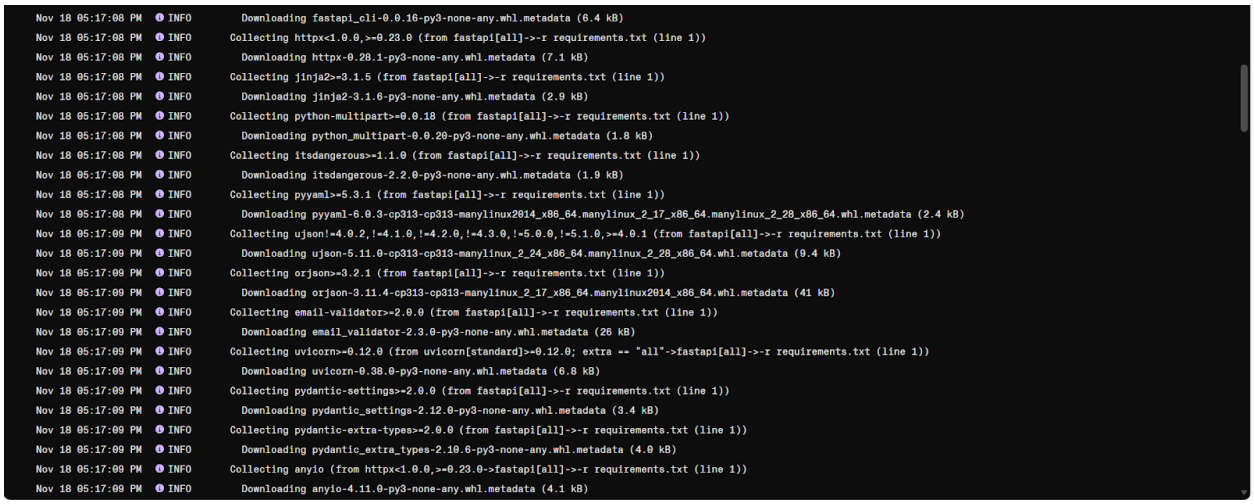


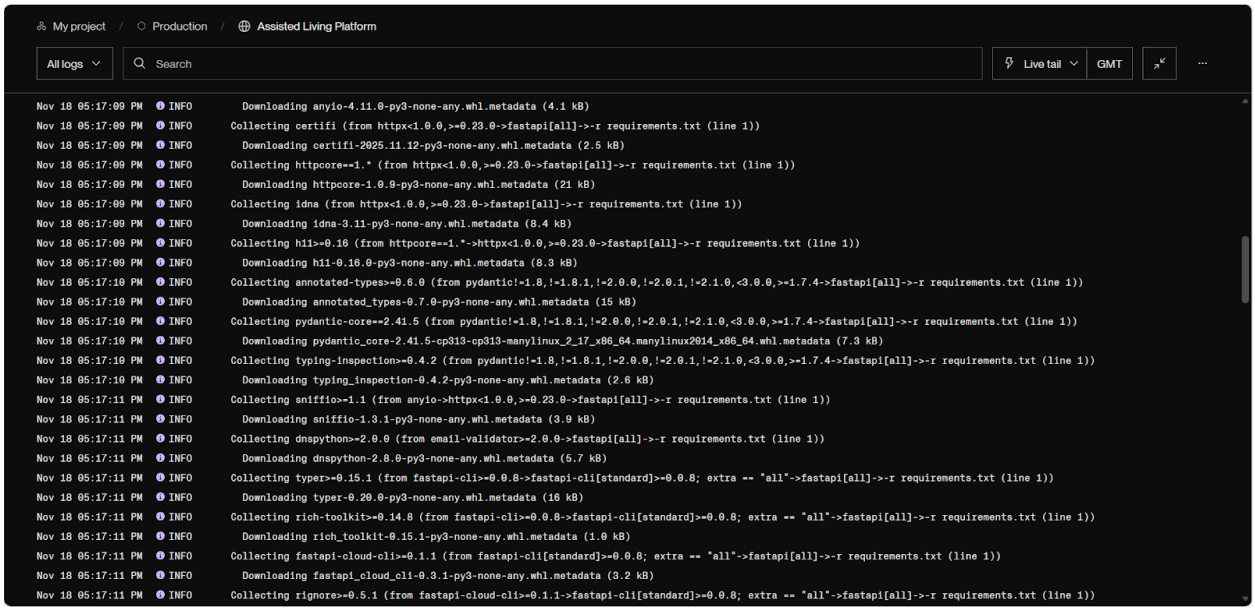


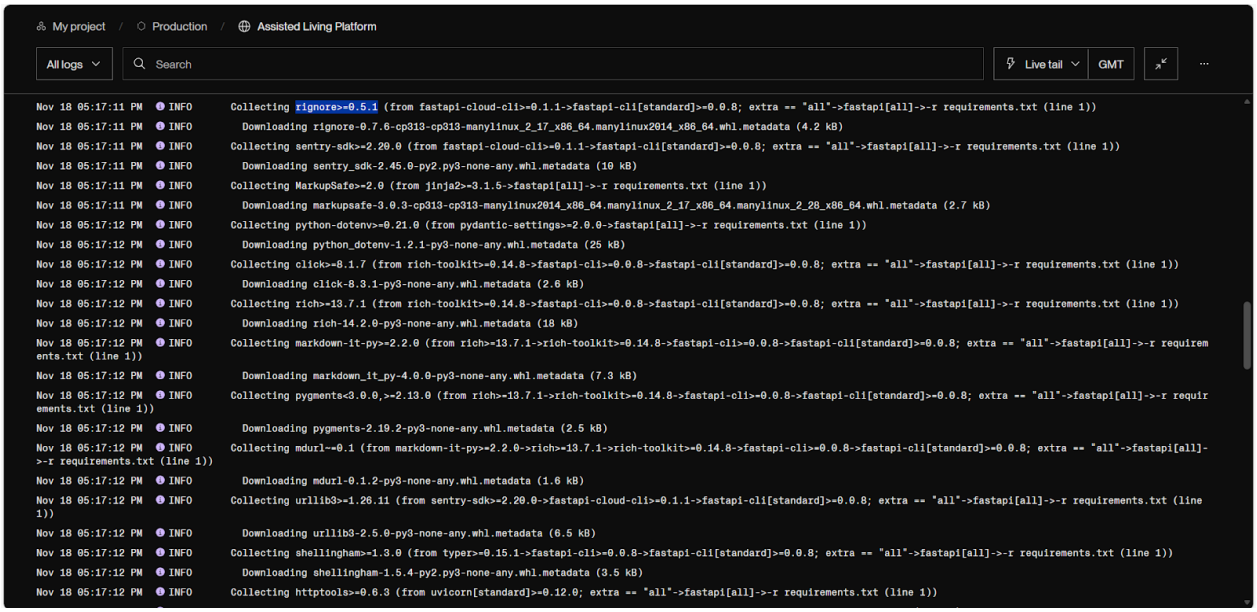
Assisted Living platform can be found at https://assisted-living-platform.onrender.com. Service ID = srv-d4eajs3gk3sc73bkfal0

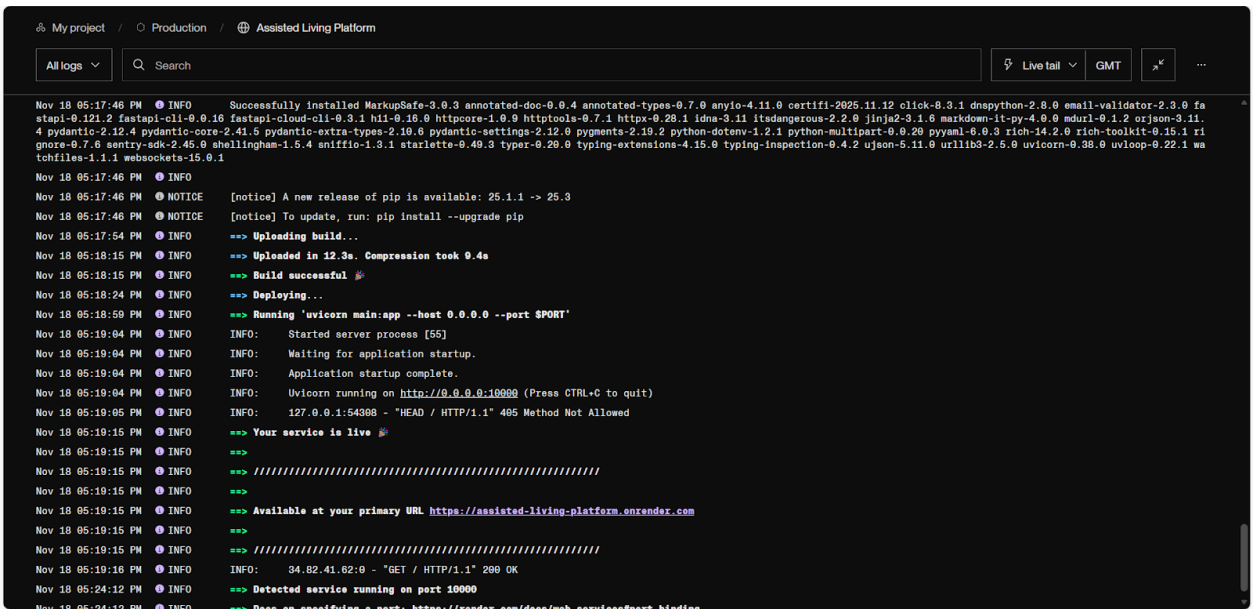
Assisted Living Platform

STATUS	✓ Deployed
RUNTIME	Python 3
REGION	Oregon
UPDATED	19d









The above figures show the logs of the cloud systems being deployed on Render.

Appendix E – Project Plan & Contributions

E.1 – GANTT Charts

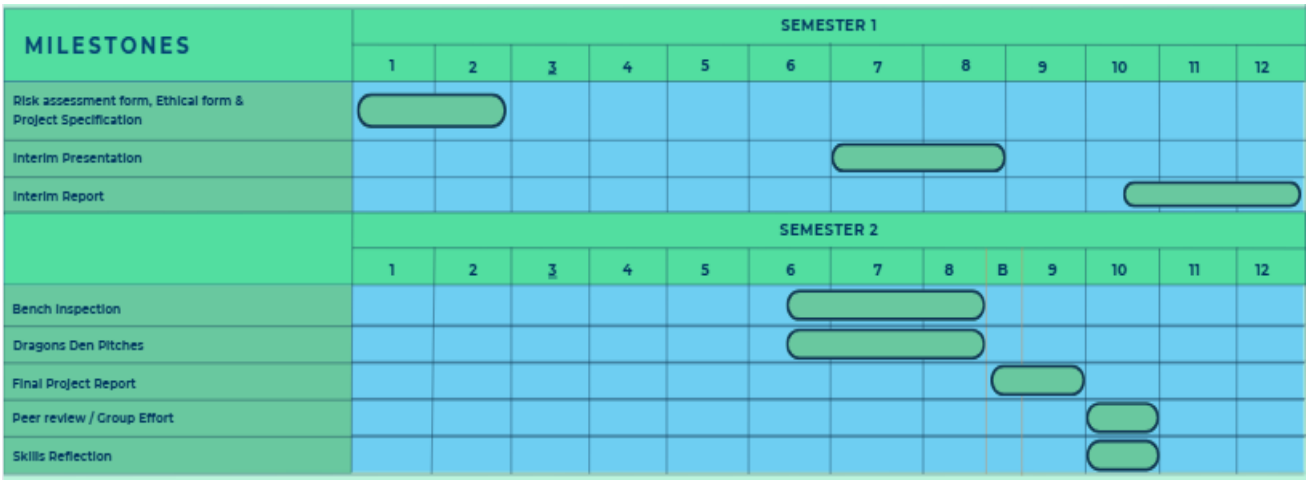


Figure 42 - Project Milestones GANTT Chart

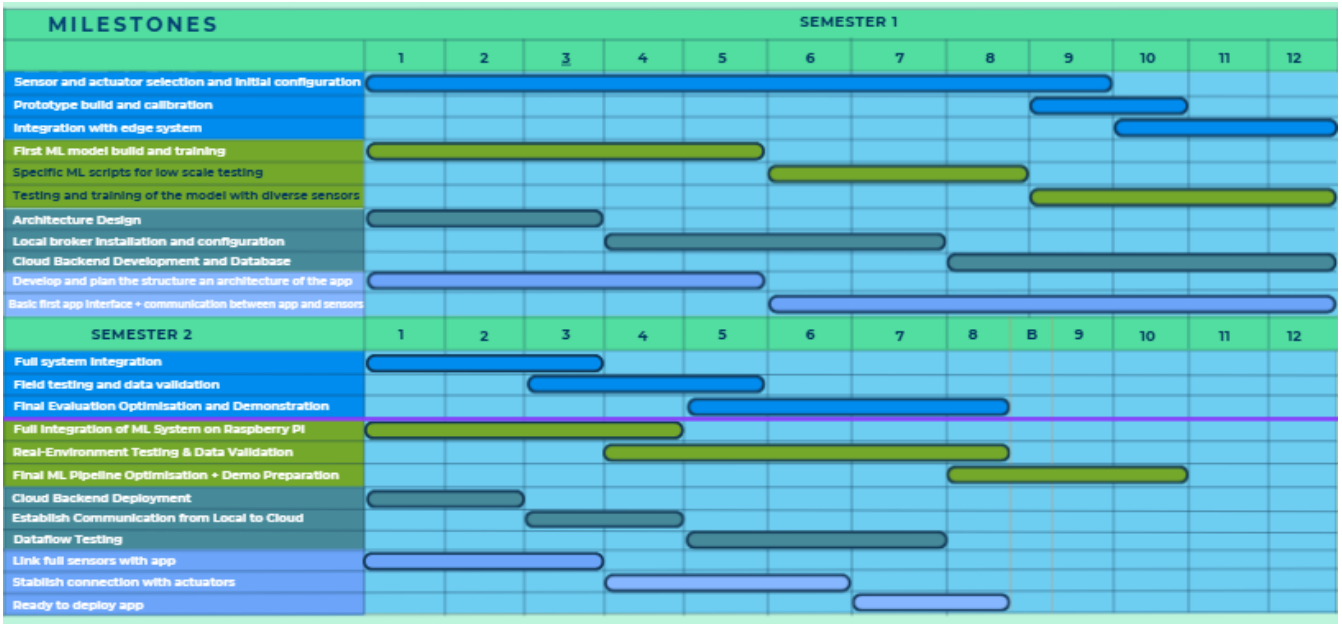


Figure 43 - Project Task GANTT Chart

E.2 Individual Contribution Form

Department of Electrical Engineering and Electronics

MEng Group Project (ELEC450) - Individual Contribution Form

Project Title: IoT and AI for Assisted Living

Student Names / ID No:

1. Mia Hornett - 201551655
2. Daniel Mitchell - 201540943
3. Alvaro Mesa Giner - 20165665
4. Luis Gulliot Lozano - 201514204

Section 1: Roles and Responsibilities

Team member	Assigned Role	Key Responsibilities
Mia	Sensor and Actuator Subsystem	Select sourcing and mounting low-voltage sensors and actuators with discrete enclosures. Implementing and standardising firmware ready for use by Dan Calibrate, debounce and document sensor outputs and define fail safes for actuators Produce and maintain sensors. Work with Alvaro to ensure sensors are calibrated for use in AI systems, work with Dan to ensure sensors are aligned with web-based central interface requirements and Luis to guarantee alerts are related to correct sensors and actuators. Build prototype home for testing of sensors on a small scale
Daniel	Network Architecture, Local and Cloud Systems	- Design of system architecture and choice of communication protocols

		- Configure local Mosquitto Broker on Pi - Local network set up - Establish connection to ESP32 with Mia (sensors) - Deploy FastAPI backend - Work with Luis to receive from frontend app - Work with alvaro to store routine backups in cloud database - PostgreSQL database integration and saving data, user credentials, certificates
Luis	Cross platform app development and communication	Develop and maintain the Flutter-based mobile app as the main user interface. Implement UI structure, state management, and clean architecture for Sensors, Routines, and Settings tabs. Prepare secure HTTPS communication with the FastAPI backend and ensure correct data flow for sensor summaries, alerts, routines, and actuator commands. Coordinate with Dan on API integration, with Álvaro on displaying ML outputs, and with Mia to ensure correct sensor/actuator representation in the UI. Support human-in-the-loop routine entry, implement sensor history and alert displays, and prepare the app for final deployment through testing, optimisation, and cross-platform validation.
Álvaro		

Section 2: Contribution Log and Reflection

Assessment component: Interim Report

Team member	Description of contribution	Key achievements	Challenges & Solutions	Plans for improvement
Mia	2. Industrial Relevance Merged Theory Section 3.2. Principles of Passive Sensing 3.3 Signal Processing & Estimation Theory 3.4. Actuator Drive & Control Physics 3.5. Feedback Mechanisms 3.6. Multi-Sensor Fusion Architecture Merged Design Section 4.2.1 Hardware Design 4.2.2. Firmware 4.5 Prototyping Merged Methodology 5.Methodology 5.1 System Development Framework and Iterative Approach 5.2.1 Scaled Prototype Environment Construction 5.2.2 Sensor & Actuator Hardware 5.2.3. ESP32 Firmware Development & MQTT Integration 5.2.4. Full Scale Hardware Deployment & Optimisation Merged Results 6.2.1 Electrical Characterisation 6.2.2. Firmware Integration Merged Discussion 7.1 Sensor Subsystem & Physical Validation 7.6. Limitations 7.7. Future Work 10.1 Mia 11. Bibliography Appendix B	Sensor and actuator theory section, sensor and actuator designs, sensor and actuator methodology, oscilloscope testing, ESP32 sensor implementation, sensor firmware implementation.	Merging sections without losing key information, conversed with teammates to ensure no key content was lost. Ensuring the report was cohesive, made sure all tenses and persons for each section were uniform	Begin merging report earlier. All write in one document initially to ensure no repetition of information

Daniel	Abstract Introduction Theory: 3.1 IoT Introduction 3.1.2 Secure Client-Server Communication 3.1.3 Offline-First Data Persistence Architecture 3.7. Edge computing in assisted living 3.8 Publish- Subscribe Architecture MQTT 3.15 Security in IoT systems 4.1 Overall system Architecture 4.2 Local System 4.2.3 Local communication and network 4.3 Cloud System 4.3.1 Fast API 4.3.2 Cloud Hosting - Render 4.3.3 Database 4.4 - Bridge between local and cloud systems 5.2.3 MQTT Integration 5.4 Cloud backend development and integration 6.1.1 Broker Configuration 6.1.2 FastAPI Backend and deployment 7.2 Network architecture and scalability 7.7 Future Work 10.2 Dan Appendix D Bibliography (partial)	Network architecture theory/ design/ methodology, Cloud system (backend, database, communication) theory/ design/ methodology, MQTT Broker installation and communication methodology and design, local network testing (data flow)	Creating a system that saw to everyone's needs and fitted the original requirements- Explained the system choices and made sure each group member understood the architecture and its implications	Work more cohesively when making crucial decisions in the future like with the network architecture. Explain why decisions have been made and how they are advantageous over other options.
Luis	Theory: 3.11 Cross platform UI development 3.12 Secure client server communication 3.13 Offline first data persistence architecture 3.14 Human in the loop behavioural Modelling	Completed UI theory, secure communication, offline app architecture, IoT security, and human-in-the-loop modelling. Wrote the mobile app design,	Ensuring theory, design, and discussion sections aligned with the rest of the report was challenging, especially avoiding overlap of content.	Start drafting earlier and use shared terminology with the team to improve consistency. plan to outline sections more clearly before

	3.15 Security in lot System Design: Introduction 4.3.4 Mobile Application 4.6 Summary Methodology: 5.5 Mobile Application development Results and calculations: 6.4 User Application Interface Discussion: 7.2 Network architecture and Scalability 7.4 Privacy by design and Security 7.7 Future work Project plan Reflection: 10.4 Luis Appendices: Appendix C	methodology for app development, UI results. Developed the full Flutter app baseline. Prepared API/repository layers for backend integration.	Maintaining consistent terminology, structure, and narrative flow. Integrating software-focused theory into a hardware-heavy report require explanations clear and relevant.	writing and coordinate more frequently during the editing stage to ensure seamless integration across subsystems. Also include more visuals to improve clarity and reduce dense text.
Álvaro	Industrial Relevance Introduction, Merged Theory Section, Theory Introduction Paragraph, Theory Conclusion Paragraph, 3.6 Multi-Sensor Fusion Architecture 3.7 Edge Computing in Assisted Living Systems 3.8 Publish-Subscribe Architecture MQTT 3.9 Model Theory, Comparison and Justification (this includes Isolation Forest vs others) 3.10 Personalised Routine Learning vs Generic Dataset Approaches 3.11 Cross-Platform UI Development (ties into ML output consumption) 3.12 Secure Client-Server Communication 3.13 Offline-First Data Persistence Architecture	Machine-learning subsystem design, sensor–ML integration testing, development of the ML training script and simulation environment, and contributions to the edge–cloud bridge and ML methodology/results.	Merging sections without losing key information, maintaining accuracy when describing unfinished ML components, managing ethical restrictions by developing a simulation environment, and ensuring multi-sensor data quality through repeated testing and alignment checks.	Further refining the ML training script for higher accuracy, improving multi-sensor calibration for more reliable behavioural features, expanding testing to full-house deployment once ethics approval is granted, and enhancing edge–cloud integration to streamline anomaly reporting and long-term data handling.

	3.14 Human-in-the-Loop Behavioural Modelling Merged Design Section 4.2.4 Machine Learning Architecture 4.4 Bridge between Local and Cloud system, Merged Methodology 5.3 Edge Computing & Machine Learning Pipeline Implementation 5.3.1 Local Data Collection & Synchronisation Pipeline 5.3.2 Feature Extraction & Behavioural Modelling Logic 5.3.4 Simulation Based Behaviour Testing 5.6 System Integration 5.6.1 Local MQTT & Edge ML Pipeline Integration 5.6.2 Edge ML & Cloud Backend Integration 5.6.3 Full Pipeline Trial in Scaled Environment 6.3 Data Acquisition & Machine Learning Pipeline, Results Introduction Paragraph, Results Conclusion Paragraph Merged Discussion 7.3 Data Pipeline (ML readiness analysis) 7.6 Limitations (ML ethical constraints) 7.7 Future Work (ML roadmap expansion) 9. Conclusions 13.Álvaro – Individual Reflection Typographical Corrections, Appendix A – ML Scripts & Training Outputs Referencing			

E.3. Project Specification

https://github.com/miahornett/Iot-and-AI-Home-Assistant-/blob/004a3d773a36a65f11c06328a1fa2f203faa53f7/Project%20Documents/Specifcation%20Report

E.4. Interim Presentation

https://github.com/miahornett/Iot-and-AI-Home-Assistant-/blob/dfb23c39663b290fc83d940c518dafe8467391e6/Project%20Documents/Interim%20Presentation.pdf